국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

최종보고서

보안등급: 일반[O], 보안[ ]

중앙행정기관명	과학기술정보통신부	사업명	SW 컴퓨팅 산업원천기술
전문기관명(해당 시 작성)	정보통신기획평가원	내역사업명(해당 시 작성)	-
공고번호	제2018-0568	총괄연구개발 식별번호(해당 시 작성)	-
		연구개발과제번호	2019-0-00394

기술분류							
국가과학기술 표준분류	EE0110	60%	EE0204	20%	EE0202	20%
ICT기술분류	SWAI0502	100 %	-	%	-	%

총괄연구개발명(해당 시 작성)	국문	-
	영문	-
연구개발과제명	국문	엣지 서비스 지원을 위한 마이크로서비스 기반의 클라우드 엣지 플랫폼 기술 개발
	영문	Development of Cloud Edge Platform Based on Microservice for Edge Service Support

주관연구개발기관	기관명	스트라토(주) (구, 에스피테크놀러지 ㈜)	사업자등록번호	452-86-01859
	주소	(06694) 서울시 서초구 효령로 17	법인등록번호	110111-7575313
연구책임자	성명	김형두	직위	본부장
	연락처 직장전화	02-2101-2884	휴대전화	*******
	연락처 전자우편	kimhd@strato.co.kr	국가연구자번호	1130-9904

연구개발기간	전체	2019. 04. 01 - 2021. 12. 31(2년 9개월)
	단계 1단계	2019. 04. 01 - 2021. 12. 31(2년 9개월)

연구개발비 (단위: 천원)

		정부지원 연구개발비	기관부담 연구개발비		그 외 기관 등의 지원금				합계			연구개발비 외 지원금
					지방자치단체		기타( )					
		현금	현금	현물	현금	현물	현금	현물	현금	현물	합계	
총계		1,834,000	33,334	300,000	-	-	-	-	1,867,334	300,000	2,167,334	-
1단계	1년차	500,000	10,000	90,000	-	-	-	-	510,000	90,000	600,000	-
	2년차	667,000	10,000	90,000	-	-	-	-	677,000	90,000	767,000	-
	3년차	667,000	13,334	120,000	-	-	-	-	680,334	120,000	800,334	-

공동연구개발기관 등 (해당 시 작성)	기관명	책임자	직위	휴대전화	전자우편	비고	
						역할	기관유형
공동연구개발기관	한국전자기술연구원	안재훈	책임	*******	corehun@keti.re.kr	공동	전문연
위탁연구개발기관	-	-	-	-	-	-	-
연구개발기관 외 기관	-	-	-	-	-	-	-

연구개발담당자 실무담당자	성명	김화성	직위	팀장
	연락처 직장전화	02-2101-2448	휴대전화	*******
	연락처 전자우편	goldmars@strato.co.kr	국가연구자번호	1127-2334

이 최종보고서에 기재된 내용이 사실임을 확인하며, 만약 사실이 아닌 경우 관련 법령 및 규정에 따라 제재처분 등의 불이익도 감수하겠습니다.

2022년 3월 1일

연구책임자: 김 형 두 (인)

주관연구개발기관의 장: 채 현 종 (직인)

공동연구개발기관의 장: 김 영 삼 (직인)

과학기술정보통신부장관 귀하

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

최종보고서

보안등급: 일반[O], 보안[ ]

중앙행정기관명	과학기술정보통신부	사업명	사업명	SW 컴퓨팅 산업원천기술	
전문기관명(해당 시 작성)	정보통신기획평가원		내역사업명 (해당 시 작성)	-	
공고번호	제2018-0568	총괄연구개발 식별번호 (해당 시 작성)		-	
		연구개발과제번호		2019-0-00394	

기술분류	국가과학기술 표준분류	EE0110	60%	EE0204	20%	EE0202	20%
	ICT기술분류	SWAI0502	100 %	-	%	-	%

총괄연구개발명 (해당 시 작성)	국문	-
	영문	-
연구개발과제명	국문	엣지 서비스 지원을 위한 마이크로서비스 기반의 클라우드 엣지 플랫폼 기술 개발
	영문	Development of Cloud Edge Platform Based on Microservice for Edge Service Support

주관연구개발기관	기관명	스트라토(주) (구, 에스피테크놀러지 ㈜)	사업자등록번호	452-86-01859
	주소	(06694) 서울시 서초구 효령로 17	법인등록번호	110111-7575313

연구책임자	성명	김형두	직위	본부장
	연락처 직장전화	02-2101-2884	휴대전화	*******
	연락처 전자우편	kimhd@strato.co.kr	국가연구자번호	1130-9904

연구개발기간	전체	2019. 04. 01 - 2021. 12. 31(2년 9개월)
	단계 1단계	2019. 04. 01 - 2021. 12. 31(2년 9개월)

연구개발비 (단위: 천원)	정부지원 연구개발비	기관부담 연구개발비		그 외 기관 등의 지원금				합계			연구개발비 외 지원금
				지방자치단체		기타( )					
	현금	현금	현물	현금	현물	현금	현물	현금	현물	합계	
총계	1,834,000	33,334	300,000	-	-	-	-	1,867,334	300,000	2,167,334	-
1단계 1년차	500,000	10,000	90,000	-	-	-	-	510,000	90,000	600,000	-
1단계 2년차	667,000	10,000	90,000	-	-	-	-	677,000	90,000	767,000	-
1단계 3년차	667,000	13,334	120,000	-	-	-	-	680,334	120,000	800,334	-

공동연구개발기관 등 (해당 시 작성)	기관명	책임자	직위	휴대전화	전자우편	비고	
						역할	기관유형
공동연구개발기관	한국전자기술연구원	안재훈	책임	*******	corehun@keti.re.kr	공동	전문연
위탁연구개발기관	-	-	-	-	-	-	-
연구개발기관 외 기관	-	-	-	-	-	-	-

연구개발담당자 실무담당자	성명	김화성	직위	팀장
	연락처 직장전화	02-2101-2448	휴대전화	*******
	연락처 전자우편	goldmars@strato.co.kr	국가연구자번호	1127-2334

이 최종보고서에 기재된 내용이 사실임을 확인하며, 만약 사실이 아닌 경우 관련 법령 및 규정에 따라 제재처분 등의 불이익도 감수하겠습니다.

2022년 3월 1일

연구책임자: 김 형 두 (인)

주관연구개발기관의 장: 채 현 종 (직인)

공동연구개발기관의 장: 김 영 삼 (직인)

과학기술정보통신부장관 귀하

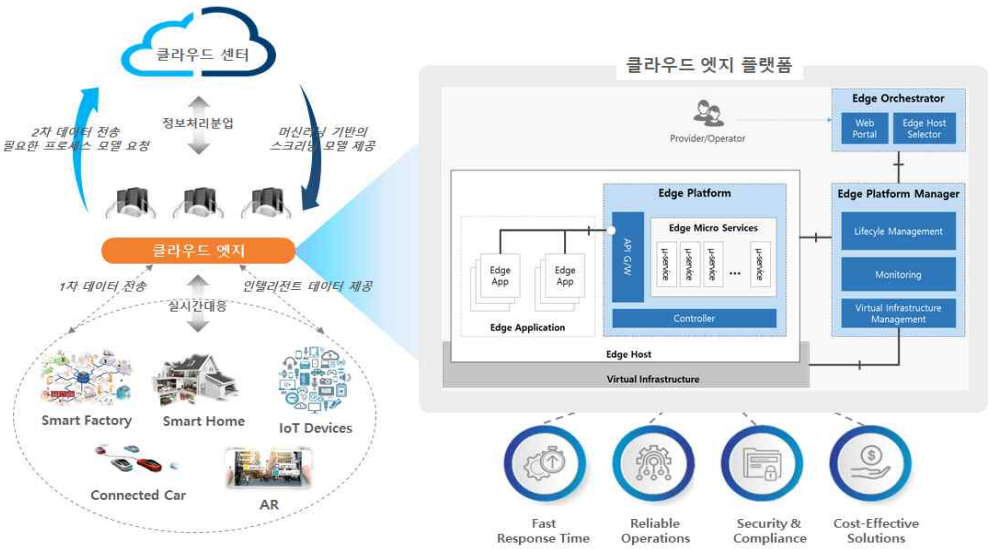
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

< 요 약 문 >

※ 요약문은 5쪽 이내로 작성합니다.

<table>
<tr><td colspan="2">사업명</td><td colspan="3">SW 컴퓨팅 산업원천기술</td><td>총괄연구개발 식별번호
(해당 시 작성)</td><td></td></tr>
<tr><td colspan="2">내역사업명
(해당 시 작성)</td><td colspan="3"></td><td>연구개발과제번호</td><td>2019-0-00394</td></tr>
<tr><td rowspan="2">기술분류</td><td>국가과학기술 표준분류</td><td>EE0110</td><td>60%</td><td>EE0204</td><td>20%</td><td>EE0202</td><td>20%</td></tr>
<tr><td>ICT기술분류</td><td>SWAI0502</td><td>100 %</td><td>-</td><td>%</td><td>-</td><td>%</td></tr>
<tr><td colspan="2">연구개발과제 특성
(해당 시 작성)</td><td colspan="5">기술료비징수 [ ] 사회문제해결 [ ] 혁신도약형 [O] 경쟁형과제 [ ] 표준화연계 [ ]
공개SW [ ] SW자산뱅크 [ ] 연구데이터공개 [ ] 일자리연계 [ ] 규제샌드박스 [ ]
정책지정 [ ] 국제공동 [ ] 사업화연계 [ ] 소재부품장비 [ ] 수요기업 [ ]</td></tr>
<tr><td colspan="2">연구개발과제명</td><td colspan="5">엣지 서비스 지원을 위한 마이크로서비스 기반의 클라우드 엣지 플랫폼 기술 개발</td></tr>
<tr><td colspan="2">전체 연구개발기간</td><td colspan="5">2019. 04. 01 - 2021. 12. 31(2년 9개월)</td></tr>
<tr><td colspan="2">총 연구개발비</td><td colspan="5">총 2,167,334천원
(정부지원연구개발비: 1,834,000천원, 기관부담연구개발비 : 333,334천원,
지방자치단체: 0천원, 그 외 지원금: 0천원)</td></tr>
<tr><td colspan="2">연구개발단계</td><td colspan="3">기초[ ] 응용[ ] 개발[√]
기타(위 3가지에 해당되지 않는 경우)[ ]</td><td>기술성숙도
(해당 시 기재)</td><td>착수시점 기준(5)
종료시점 목표(7)</td></tr>
<tr><td colspan="2">연구개발과제 특성
(해당 시 작성)</td><td colspan="5"></td></tr>
<tr><td rowspan="2">연구개발 목표 및 내용</td><td>최종 목표</td><td colspan="5">▣ 마이크로서비스 기반의 클라우드 엣지 라이프 사이클 관리와 엣지 컴퓨팅 자원 자동 관리 기능을 제공하고, Region 및 클라우드 엣지 플랫폼 간 서비스 라이브 마이그레이션을 지원하는 클라우드 엣지 플랫폼 기술 개발

비전: 엣지 서비스 생태계 조성 및 사업 활성화를 위한 에코 플랫폼化

<목표 클라우드 엣지 플랫폼 기술 구조>

o End Product
- 마이크로서비스 기반 엣지 서비스 관리 및 제어 모듈(SW)</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

		- 엣지 앱 및 서비스 배포를 위한 라이프 사이클 관리 모듈(SW) - 요청 및 스케줄링 Rule 에 따른 클라우드 엣지 자원 자동 관리 모듈(SW) - 엣지 플랫폼 간 이동을 위한 엣지 호스트 관리 모듈(SW) - 엣지 플랫폼 간 서비스 라이브 마이그레이션 제어 모듈(SW) - 엣지 서비스 워크로드에 따른 Live Migration Decision 모듈(SW) - 클라우드 엣지 자원 계층별 워크로드 예측 모듈(SW) - Zone 이동 시, 엣지 플랫폼 간 서비스 라이브 마이그레이션 모듈(SW)
	전체 내용	**주요개발내용** ① 마이크로서비스 기반의 엣지 서비스 관리 기술 개발 • 엣지 환경/특성에 따른 마이크로서비스 기반의 엣지 서비스 조합과 연결 기술 • 엣지 앱의 API 요청에 따른 트래픽 조절과 결함 격리 기술 • 엣지 앱과 엣지 플랫폼 구간 오픈 연동 기술 • 엣지 서비스 API 단위의 권한/허가 및 보안 인증 기술 • 실사용량 측정을 위한 API 단위 호출 및 로깅 기반 미터링 기술 ② 클라우드 엣지 플랫폼 라이프 사이클 관리 기술 개발 • 패키지 및 인스턴스 배포/실행을 위한 라이프 사이클 관리 모델 연구 • 엣지 앱 배포/실행 관리를 위한 네트워크 서비스 디스크립터 기술 • 엣지 플랫폼과 엣지 플랫폼 관리 구간 연동 기술 • Region 및 엣지 플랫폼 간 이동을 위한 엣지 호스트 선택 기술 • 엣지 앱 및 서비스 라이프 사이클 관리를 위한 운용자 및 사용자 포탈 제공 기술 ③ 고확장성 및 고가용성 제공을 위한 클라우드 엣지 자원 자동 관리 기술 개발 • 자동 스케줄링을 위한 자원 워크로드 및 사용량 이벤트 수집 기술 • API 실행을 위한 엣지 서비스 인스턴스 자원 할당 및 스케줄링 기술 • 엣지 앱, 엣지 서비스의 부하 기반 자원 용량 프로비져닝 및 자동 조정 기술 ④ Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발 • 마이크로서비스 기반 Zone In/Out 여부 판단 기술 • 마이크로서비스 기반 Zone 이탈 여부 정보 전달 기술 • Zone 이동 시, 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 ⑤ 클라우드 엣지 컨테이너기반 IoT 응용 서비스 설계 및 프로토타입 개발 • 대규모 IoT 디바이스 데이터 수집 서비스 응용 클러스터 기술 • 컨테이너기반 OneM2M(Mobius 플래폼 or WSO2)기반 IoT 플랫폼 연동 기술 • 클라우드 엣지 컨테이너기반 IoT 응용 서비스 시각화 기술

연구개발성과	▢ **연구개발 결과물의 기능, 규격, 구조 및 특성에 대한 주요 정량적 목표** - 클라우드 엣지 플랫폼의 결과물 검증을 위한 정량적 개발 목표치 계획

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

평가항목(주요 성능 Spec)	단위	개발 목표	최종 실적
1. 라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간	sec	< 14	9.4
2. 자원 자동 스케줄링 관리를 복합분석 정책 메트릭 항목 수	종	5 <=	6
3. 엣지 앱과 엣지 서비스 Life Cycle Management API 수	종	22 <=	25
4. 클라우드 엣지 플랫폼 정보보호 및 보안인증 지원 항목 수	종	30 <=	30
5. 라이브 마이그레이션 수행시 최대 Freeze Time	sec	< 19	13.8
6. 클라우드 엣지간 라이브 마이그레이션 정책 항목 수	종	8 <=	8
7. 워크로드에 따른 Live Migration Decision API 수	종	5	5

연구개발성과 활용계획 및 기대 효과

▢ **활용 계획**

- (자체 사업화) 과제 최종결과물의 수요기관 실증으로 사업화 가능성을 실증하고, IoT 사업자, 국내 이동통신 및 대형 클라우드 서비스 사업자 대상으로 클라우드 엣지 플랫폼 공급 및 응용 서비스 구축 사업화 추진

▢ **기대효과**

- **(기술적 측면)** 4차 산업혁명과 5G 기술 상용화로 클라우드 엣지 기술에 대한 중요성이 부각되고 대부분 글로벌 외산 기업이 클라우드 엣지 기술을 선점하고 있으나 아직 상용화 수준에는 기술적 제약이 있어 글로벌 업체들도 최근에야 대처하기 시작한 상황으로 본 과제에서 제안하는 기술 개발을 통해 기술 선도 가능
- **(경제/산업적 측면)** 외국 클라우드 엣지 서비스를 도입하여 단순 활용하는 국내기업의 해외기술 종속성 개선, 국내 클라우드 SW기업의 기술 및 시장 경쟁력과 글로벌 시장 진출 경쟁력을 확보할 수 있는 미래 핵심 기술
- **(사회적 측면)** 지능정보, 스마트팩토리 등 차세대 융합 서비스와 연계하여 단말과 물리적으로 가까운 위치에 있는 엣지에서 데이터를 처리함으로서 지연시간 최소화를 통해 소비자에 삶의 편의성을 제공하는 다양한 응용 서비스를 실시간 제공

연구개발성과의 비공개여부 및 사유

공개 여부 : 비 공개
사유 : 주관 기관이 기업으로, 자체 사업화 추진 예정

연구개발성과의 등록·기타 건수

논문	특허	보고서 원문	연구시설·장비	기술요약 정보	소프트웨어	표준	생명자원		화합물	신품종	
							생명정보	생물자원		정보	실물
5	9	3	-	-	18	5	-	-	-	-	-

연구시설·장비 종합정보시스템 등록 현황

구입기관	연구시설·장비명	규격(모델명)	수량	구입연월일	구입가격(천원)	구입처(전화)	비고(설치장소)	ZEUS 등록번호
-	-	-	-	-	-	-	-	-

국문핵심어 (5개 이내)	클라우드 컴퓨팅	엣지 컴퓨팅	클라우드 엣지 플랫폼	엣지 앱	엣지 서비스
영문핵심어 (5개 이내)	Cloud Computing	Edge Computing	Cloud Edge Platform	Edge Application	Edge Service

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<목 차>

1. 연구개발과제의 개요.....7

2. 연구개발과제의 수행 과정 및 수행 내용 13

3. 연구개발과제의 수행 결과 및 목표 달성 정도.....80

4. 목표 미달 또는 미흡한 사항에 대한 분석 103

5. 연구개발성과의 관련 분야에 대한 기여 정도.....103

6. 연구개발성과의 관리 및 활용 계획.....104

7. 연구개발성과에 대한 후속연구 및 추가 개발 계획 109

8. 연구개발비 사용실적 111

붙임2. 자체 보안관리 진단표.....114

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1. 연구개발과제의 개요

▣ 엣지 컴퓨팅(Edge Computing) ?

'엣지 컴퓨팅'은 다양한 단말 기기에서 발생하는 데이터를 클라우드와 같은 중앙 집중식 데이터 센터로 보내지 않고 데이터가 발생한 현장 혹은 근거리에서 실시간 처리하는 방식을 의미함. 즉 사용자의 디바이스 근처, 네트워크의 가장자리에서 컴퓨팅을 지원하는 것으로 **'클라우드 엣지' 즉, 클라우드 컴퓨팅과 서비스를 네트워크의 말단부로 확장한 것.**



[Cloud Computing vs. Edge Computing]

엣지 컴퓨팅은 아래와 같은 특징을 가집니다.

◦ 네트워크 의존도를 낮추고 안정적으로 데이터 송수신 가능

- 데이터가 기기에 직접적으로 연결되어 있거나 기기에서 자체적으로 접속
- 가능한 네트워크 노드 상에 위치하고 있기 때문에 네트워크 지연이나 전송 오류로부터 자유로움
- 데이터 해킹과 같은 공격으로부터의 노출을 줄일 수 있음

◦ 클라우드 컴퓨팅보다 빠른 연산과 응답이 가능

- 기기와 데이터의 물리적 거리가 짧고 연산을 위해 다루는 데이터의 양도 클라우드 컴퓨팅에 비해 적기 때문에 상대적으로 더 빠른 연산과 데이터 전송 이 가능(클라우드 컴퓨팅 방식에 비해 10배 빠른 속도 - IEEE 발표 자료에 언급)

◦ 기기에서 발생되는 데이터 대부분은 자체적으로 처리

- 기기에서 수집되는 막대한 데이터 대부분은 엣지 컴퓨팅으로 처리하고 중앙 데이터 센터에는 꼭 필요한 데이터만 전송
- 이를 통해 데이터 부하를 줄이고 전송 단계에서의 보안 위협도 피할 수 있음

◦ 즉 엣지 컴퓨팅을 통해 기존 클라우드 컴퓨팅이 가졌던 한계를 극복 또는 보완

- 4차 산업혁명 시대를 대표했던 네트워크 플랫폼인 클라우드는 엣지 컴퓨팅을

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

통해 더 강력해 질 수 있으며 활용 분야의 확대가 기대됨

- 이는 엣지 컴퓨팅이 클라우드 컴퓨팅을 대체한다는 의미가 아니라 클라우드 컴퓨팅과 공존하며 상호 한계점을 보완할 수 있다는 것을 의미

▣ 엣지 컴퓨팅의 등장 배경

점점 많은 애플리케이션과 워크로드가 클라우드로 빠르게 몰리고 있으며, 산업용 사물인터넷과 자율 주행 자동차 등 실시간 데이터 처리가 중요한 분야는 원격지에 있는 클라우드 컴퓨팅에서 실시간으로 데이터를 분석하고 처리하기에는 효율성이 떨어진다는 것이 최근 전문가들의 의견. 게다가 5G의 확대로 빠른 데이터 처리가 중요한 시대임을 감안할 때도 클라우드의 과부하는 우려스러운 것이 사실. 이젠 물리적 통신 인프라가 막강한 데이터 트래픽 성장을 따라가기가 사실상 불가능한 상황에 놓임에 따라 엣지 컴퓨팅이 클라우드 컴퓨팅 과부하 해결 방안으로 등장하게 됨.

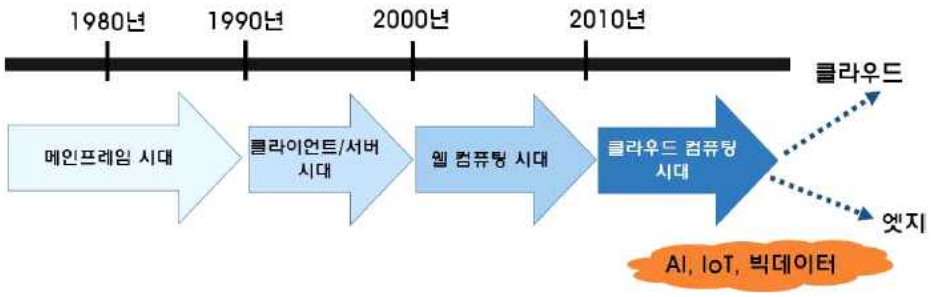
- IoT 기기가 다양해지고 전송하는 데이터양이 방대해 지면서 시간·장소에 구애받지 않고 대한 데이터를 인터넷 상에서 공유·전송할 수 있는 컴퓨팅 기술이 발전을 거듭

※ 세계 IoT 기기 수는 '16년 약 36억 대에서 '21년 250억 대로 32.0%의 CAGR을 기록할 전망(Gartner, '17.12)

※ 세계 데이터 시장 규모는 '16년 16.1ZB(제타바이트: 1021)에서 '25년 163ZB로 10배 이상 증가할 전망(seagate/IDC, '17.4)

- 이에 기존 ICT 자원을 기업 서버나 개인 PC에 소유하던 방식에서 필요한 만큼 자원을 클라우드 공간에 임대하여 사용하고 비용을 지불하는 클라우드 컴퓨팅 방식이 등장
- 클라우드 컴퓨팅 시장은 ICT 시장뿐 아니라 자동차·금융·의료 등 다양한 산업 분야에 확대 적용되며 꾸준히 성장
- 하지만 산업용 IoT·자율주행자동차 등 실시간 데이터 처리가 중요한 분야는 원격지에 있는 클라우드 컴퓨팅에서 데이터를 분석·처리하기는 효율성이 다소 떨어진다는 평가
- 즉, 디바이스 가까운 가장자리에서 컴퓨팅 시스템을 구동하며 보다 빠르고 안정적으로 데이터를 처리할 수 있는 컴퓨팅 기술에 대한 요구가 증대
- 추가적으로, 여러 대기업의 개인정보 유출 사례에서 알 수 있듯이 클라우드 컴퓨팅의 통신 과정에서 발생하는 보안 문제점도 대두
- 이에 따라 디바이스와 가까운 네트워크 '가장자리(Edge)'에서 컴퓨팅을 지원하는 '엣지 컴퓨팅(Edge Computing)'이 클라우드 컴퓨팅 한계를 보완할 수 있는 기술로 부상

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[Computing 환경 변화]

▣ 엣지 컴퓨팅 활용이 증가할 것으로 예상되는 분야

◦ (자율 주행 자동차) 네트워크 지연과 데이터 전송 오류를 최소화하며 안전성 제고

- 차량 장착 센서에서 실시간 수집한 데이터를 Edge에서 빠르게 처리하며 차 간 거리 유지와 주변 도로 상황, 차량흐름 등을 즉시 파악하고 주행 중 돌발 상황 발생 시 신속한 대처 가능
- 기존 클라우드 방식인 중앙 데이터센터에서 이러한 데이터 처리가 이루어진 다면 네트워크 지연이나 데이터 전송 오류 시 큰 사고로 이어질 수 있어 엣 지 컴퓨팅이 가장 유용한 분야로 전망

◦ (스마트팩토리) 클라우드와 엣지 컴퓨팅 조합으로 효율적인 운영 추구

- 신속성을 요구하는 데이터 처리는 엣지에서 연산을 수행하고 고도의 정밀한 분석이 필요한 데이터는 중앙 데이터센터로 전송하여 네트워크 과부하를 줄이고 비용절감 및 서비스 개선 가능

※ 공장 내 온도·습도 조절 등은 엣지에서 처리하고 사고 위험 예측, 기기 수명관리 등은 중앙 데이터센터가 담당

◦ (가상·증강현실) 지연 시간을 줄이며 최적의 몰입경험 구현

- 가상·증강현실 애플리케이션과 콘텐츠를 모두 중앙 데이터센터로 전송할 경 우 속도와 안전성을 보장하기 어렵다는 점에서 엣지 컴퓨팅 기술이 부각
- 특히 네트워크 지연 시간은 몰입 경험에 직접적 영향을 미치기 때문에 가상·증강현실 콘텐츠를 디바이스 가까운 엣지에서 처리하는 것은 큰 장점

◦ (5G) 무선 기지국에 연산 능력을 부여하여 다양한 서비스와 콘텐츠 제공

- 이용자와 가까운 무선 기지국에 대용량 애플리케이션 서버를 탑재해 빠른 전송이 필요한 대용량 데이터를 파악하고 이용자 요청이 있을 때 즉시 전송

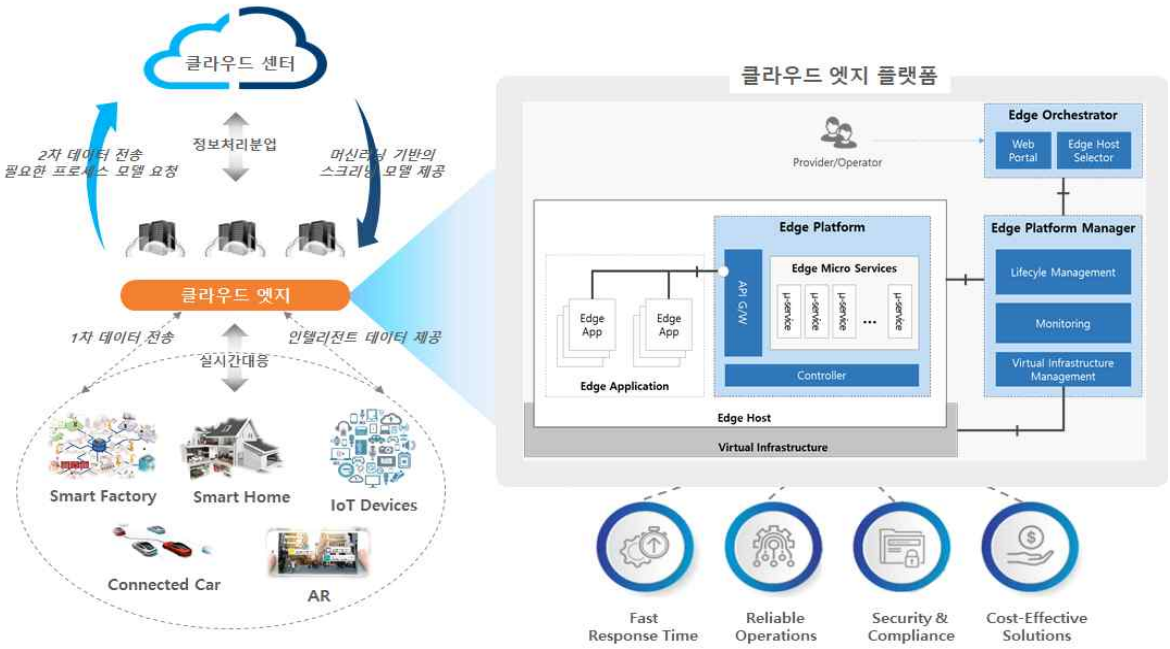
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

과제목표 : 마이크로서비스 기반의 클라우드 엣지 서비스 라이프 사이클 관리 / 엣지 컴퓨팅 자원 자동 관리 기능을 제공하고, Region 및 클라우드 엣지 플랫폼 간 서비스 라이브 마이그레이션을 지원하는 클라우드 엣지 플랫폼 핵심 기술 개발

엣지 서비스 생태계 조성 및 사업 활성화를 위한 에코 플랫폼化

이러한 클라우드 엣지 플랫폼 기술 개발을 위해 아래와 같은 핵심기술 필요

① 마이크로서비스 기반의 엣지 서비스 관리 기술 개발
② 클라우드 엣지 라이프 사이클 관리 기술 개발
③ 고확장성 및 고가용성 제공을 위한 클라우드 엣지 자원 자동 관리 기술 개발
④ Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발
⑤ 클라우드 엣지 컨테이너기반 IoT 응용 서비스 설계 및 프로토타입 개발



[목표 클라우드 엣지 플랫폼 기술 구조]

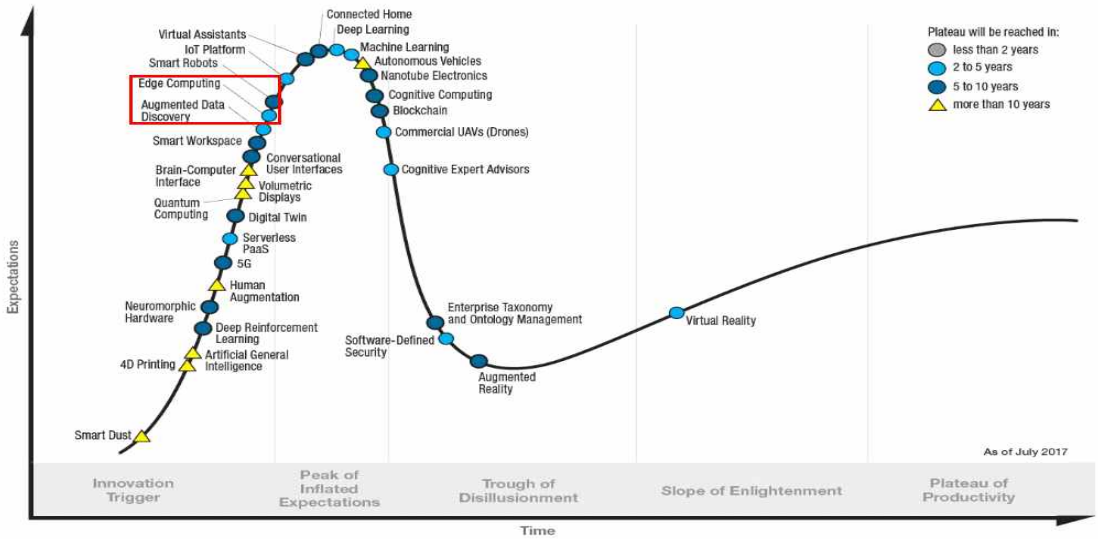
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

▣ **(기술개발의 중요성)** 엣지 컴퓨팅은 클라우드 산업과 자율 주행 자동차, 산업용 사물 인터넷, 스마트 시티, 스마트 팩토리, 스마트 팜, 드론, AR/VR과 같은 다양한 IOT 산업 활성화를 위한 차기 클라우드 서비스 기술로 정부 차원 지원이 시급

- 글로벌 클라우드 대형 기업들은 클라우드 기능을 엣지 기기로 확장할 수 있 는 다양한 솔루션을 제공 중
- 국내는 개념 정립 단계로 글로벌 솔루션 대비 조기 경쟁력 확보를 위한 정부의 전략적 지원 및 기술 확보 방안 마련 필요
- 엣지 컴퓨팅의 한 축인, 모바일 엣지 컴퓨팅이 5G 네트워크 진화의 핵심 기술로 대두

▣ **(기술성)** 분산처리 관리 기술 확보를 통한 차기 클라우드 서비스 경쟁력 제고 필요

- 엣지 컴퓨팅은 클라우드 컴퓨팅 환경에서 분산 처리를 통해 데이터 트래픽을 줄일 수 있고, 산업용 사물 인터넷과 자율 주행 자동차와 같이 실시간 분석/처리가 필요한 서비스를 위한 핵심 기술로 빠른 개발을 통한 경쟁력 확보가 반드시 필요함.
- 포스트에서 2018년 디지털 트랜스포메이션 10대 트렌드로 선정
- 가트너는 2018년 10대 전략 기술 트렌드에서 5번째로 “Cloud에서 Edge로”를 선정
- 가트너가 신기술 관련 보고서 2017년 ‘신기술 하이프 사이클 보고서(Hype Cycle for Emerging Technologies)’를 발표하며, 향후 5~10년간 디지털 경제 내에서 기업의 생존과 번영을 좌우할 주요 기술 트렌드를 선정



※ 출처 : Gartner, Gartner Hype Cycle for Emerging Technologies, 2017

[가트너의 2017년 신기술 하이프 사이클]

▣ **(경제성)** 엣지 컴퓨팅 서비스 시장 규모가 급속도로 성장할 것으로 예상되며, 이에 따라 시장 선점을 위한 글로벌 기업의 시장 경쟁이 가속화되고 있고, 기술 선점을 위한 투자가 막대함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

◦ 전 세계 엣지 컴퓨팅 시장은 2016년 10.9억 달러에서 수요 급증으로 연 평균 35.3% 성장하여 2012년에는 약 49.4억 달러에 이를 것으로 전망 (MarketsandMarkets, 2017.10. 추정)

◦ 모바일 엣지 컴퓨팅 적용 시 3년간 약 54%이상의 TCO 절감 효과 발생 (iGR 미국 소매 업체 TCO 연구 결과)

◦ 막강한 트래픽 증가에 대한 물리적 인프라 확충 대신에 엣지 컴퓨팅을 적용함으로써 인프라 확충 비용 절감 효과 기대할 수 있음

◦ 아마존, MS, GE, IBM 등 세계 유수의 기업들이 엣지 컴퓨팅 기술 개발에 중점을 두고 있음

(단위 : 백만 달러, %)

구분	'16	'17	'18	'19	'20	'21	CAGR
세계시장	1,090	1,470	1,990	2,690	3,650	4,940	35.3

* 출처 : MarketsandMarkets, Edge Computing Market by Component (Hardware, Platform, Solutions), Application (Smart Cities, Location Services, Analytics, Augmented Reality), Organization Size (SME, Large Enterprises), Vertical, and Region - Global Forecast to 2022, 2017.10으로 추정

[엣지 컴퓨팅 기술 분야의 세계 시장 규모 및 전망]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연구개발과제의 수행 과정 및 수행 내용

▣ 주관기관(스트라토) :

- 과제 총괄 및 클라우드 엣지 플랫폼 구조 설계 및 적용
- 마이크로서비스 기반 엣지 서비스의 API 단위 응용 연결기술 개발
- 엣지 앱/서비스 라이프 사이클 관리 기술 개발
- 고확장성 및 고가용성 제공을 위한 클라우드 엣지 자원 자동 관리 기술 개발
- 전체 통합 및 상용 클라우드 시범운영

▢ 다양한 엣지 환경/특성에 따른 엣지 서비스 조합과 연결 기술 개발

○ API 단위 서비스 처리를 위한 마이크로서비스 기반의 엣지 플랫폼 아키텍처 설계 및 개발

- 클라우드 엣지 플랫폼 아키텍처 설계
- Edge Platform Management 구성 모듈 개발

① Portal

- 엣지 앱/서비스 등록 관리(등록/삭제/업데이트)
- Descriptor 생성
- 인증 정보 관리(ClientId/ClientSecret)
- 엣지 앱 서비스 API 이용 관리
- 엣지 서비스 CPU/Memory 사용률 관련 임계치 및 Scale In/Out 정책 설정 관리

② Life cycle Manager

- 엣지 앱/Service 배포 관리(배포/중지/재개/삭제/업데이트/Dependency를 고려한 배포)

③ Resource Manager

- K8S와 연동을 통한 엣지 앱/Service 인스턴스 생성 관리(생성/삭제/업데이트)
- Work Node 사용률/상태 관리
- 엣지 앱/Service 인스턴스 배치 정책에 따른 Work Node 선택
- Self Healing 처리 설정

④ Policy Manager

- 자원 워크로드 정책에 따라 엣지 서비스의 Scale In/Out 트리거링

⑤ Monitoring

- 엣지 서비스의 CPU/Memory 사용률 정보를 수집
- 엣지 서비스 Broker를 통해 CEP로 수집정보 전달

⑥ Logging

- 엣지 앱의 엣지 서비스 API 사용 이력 수집 관리

⑦ CEP

- 알람 정책에 따라 모니터링 관련 알람 발생 관리

- 엣지 서비스 Platform 구성 모듈 개발

① API G/W

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 엣지 앱의 엣지 서비스 API 사용 권한 체크(Access Control)
- 엣지 앱의 일 단위 API별 사용 허용량에 따른 API 사용 제어(Pre-Cotnrol)
- 엣지 서비스 API 요청을 엣지 서비스 Instance로 전달(Routing Control)
- 복수개의 Instance가 존재하는 엣지 서비스 에 대한 API 요청에 대해 Round Robin 방식으로 엣지 서비스 Instance에 균등하게 배분 처리(Load balancing)
- 서비스 사용 제한 또는 권한 체크에 의한 오류 처리(Error Control)
- 엣지 서비스 API 사용 이력 생성, Edge Service Broker를 통해 Logging에 전달(Post Control)
- Registry/Discovery에 등록된 엣지 서비스 Routing 정보를 주기적으로 수집(Registry/Discovery Agent)
- API G/W와 엣지 서비스 간 연동 시 엣지 서비스의 비정상적인 동작에 의한 API G/W의 장애 방지(Circuit Breaker)

② Edge Service Broker

- 엣지 앱/엣지 서비스/Edge Service Platform Management/Edge Service Framework 간 메시징 통로

③ Data Manager

- 엣지 서비스의 데이터 저장 관리(보관 주기 및 암호화 등 지원)

④ Auth Manager

- 엣지 앱의 Edge Service Platform 사용을 위한 인증 정보 발급(ClientId/ClientSecret)
- 엣지 앱의 인증 정보 유효성 체크
- 엣지 앱의 Edge Service Platform Access Token 발급
- 엣지 앱의 Access Token 유효성 체크

⑤ DNS Service

- 엣지 앱의 DNS 정보 관리 및 DNS 서비스 제공

⑥ Registry/Discovery

- 엣지 서비스 Routing 정보 등록 관리
- 엣지 서비스 Instance Health Check

⑦ In/Out Detector

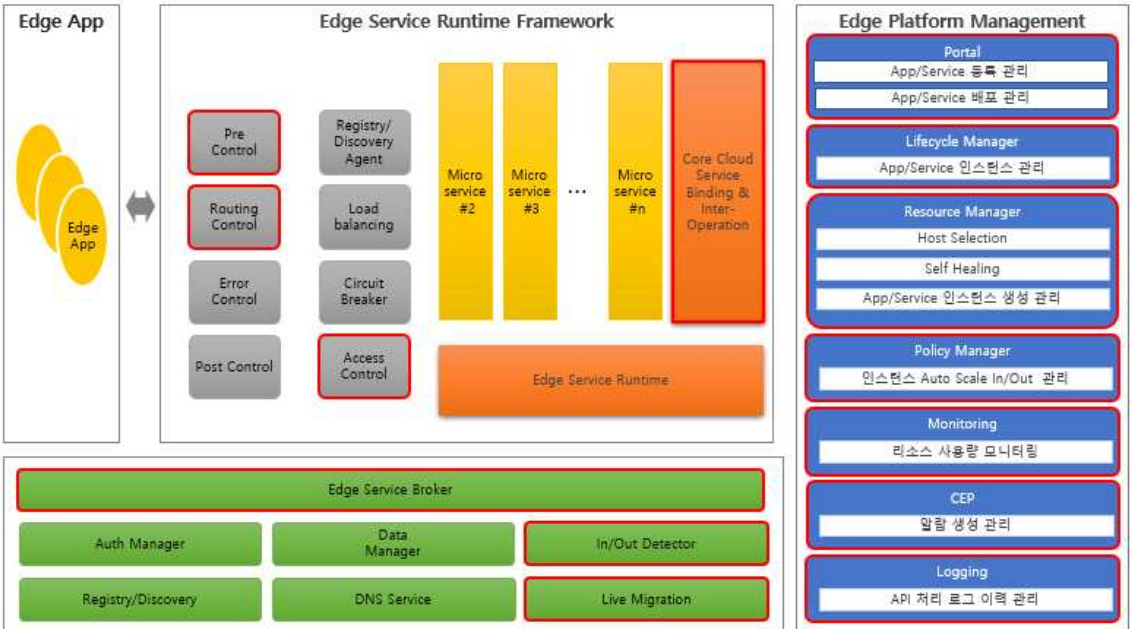
- 엣지 앱의 Edge Service Platform 내 진입/이탈 판단 관리

⑧ Live Migration

- 엣지 서비스/앱의 실시간 Work Node 이동 처리

⑨ Core Cloud Service Binding & Inter-Operation

- Cloud Edge와 Core Cloud간의 서비스 연동을 위한 interface

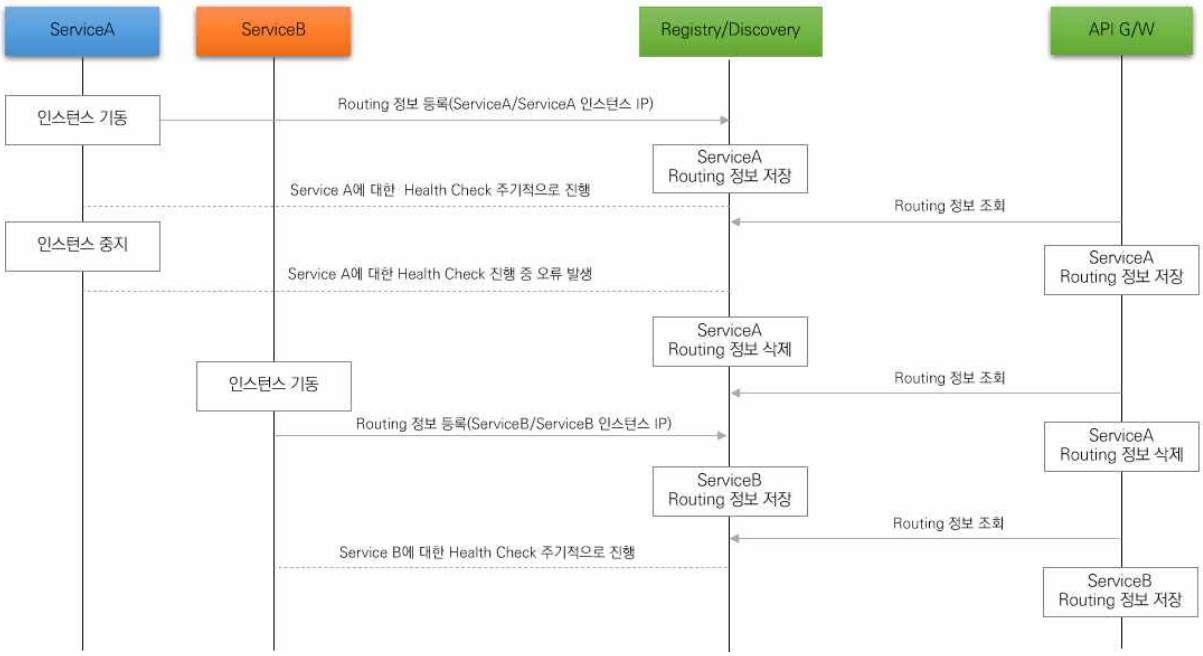


[클라우드 엣지 플랫폼 S/W 아키텍처]

○ 엣지 앱의 API 단위 요청을 엣지 서비스로 연결하는 라우팅 기술 개발

- Client-Server 형태의 서비스 연결 라우팅 기술 개발
- 엣지 서비스 라우팅 정보 관리 모듈 개발

엣지 서비스 라우팅 정보 관리는 엣지 서비스 인스턴스 내 Registry Agent, Registry/Discovery, API G/W내 Registry Agent간에 아래와 같은 과정을 거쳐 진행됨.



[엣지 서비스 라우팅 정보 관리 흐름]

- 엣지 서비스 인스턴스가 기동되면 인스턴스 내에 내장된 Registry Agent가 엣지 서비스 인스턴스의 Routing 정보를 Registry/Discovery에 등록
  (Routing 정보는 service name과 IP(또는 domain)로 구성)

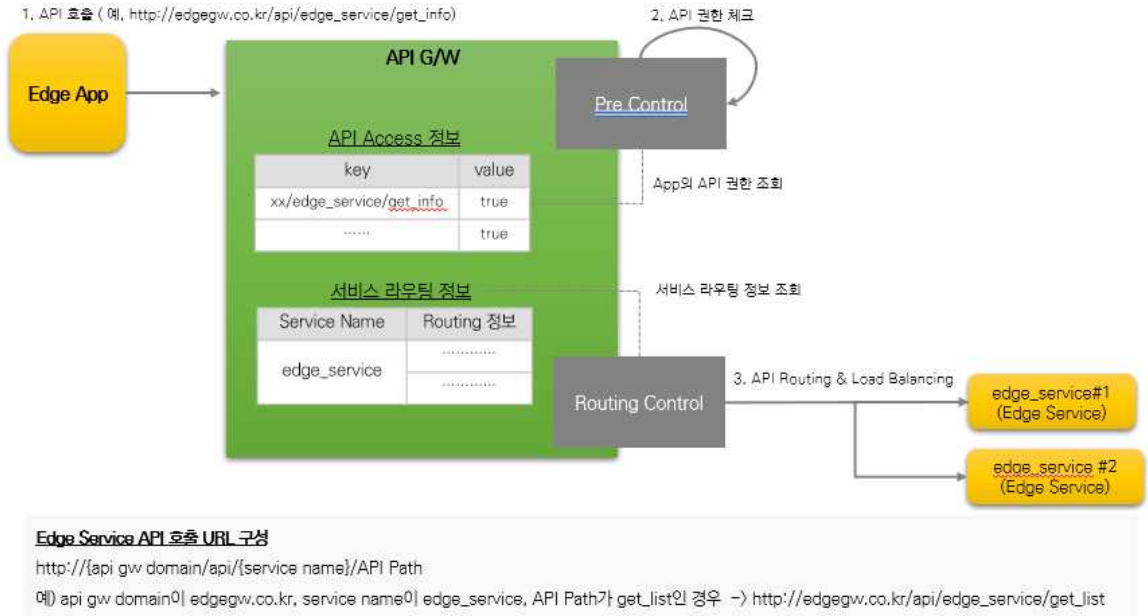
- API G/W내 Registry Agent는 주기적으로 Registry/Discovery에 Routing 정보를 조회하여 엣지 서비스 인스턴스의 최신 Routing 정보를 확보
- Registry/Discovery는 등록된 Routing 정보를 기반으로 주기적으로 등록되어 있는 엣지 서비스 인스턴스에 대해 Health Check를 수행하여 인스턴스 정상 여부를 확인
- Health Check를 수행하여 비정상으로 확인된 인스턴스의 Routing 정보는 자동 삭제 처리

• 엣지 서비스 라우팅 관리 모듈 개발

• 복수 엣지 서비스 인스턴스에 대한 로드밸런싱 기능 개발

엣지 앱의 API 요청에 대한 라우팅 처리는 API G/W내 Registry Agent가 Registry 서버와 주기적인 연동을 통해 수집한 엣지 서비스 라우팅 정보를 이용하여 이뤄지며, 복수 Instance가 존재하는 엣지 서비스에 대해서는 Round Robin 방식의 로드밸런싱을 수행함.

라우팅 처리 과정은 아래와 같이 이뤄짐



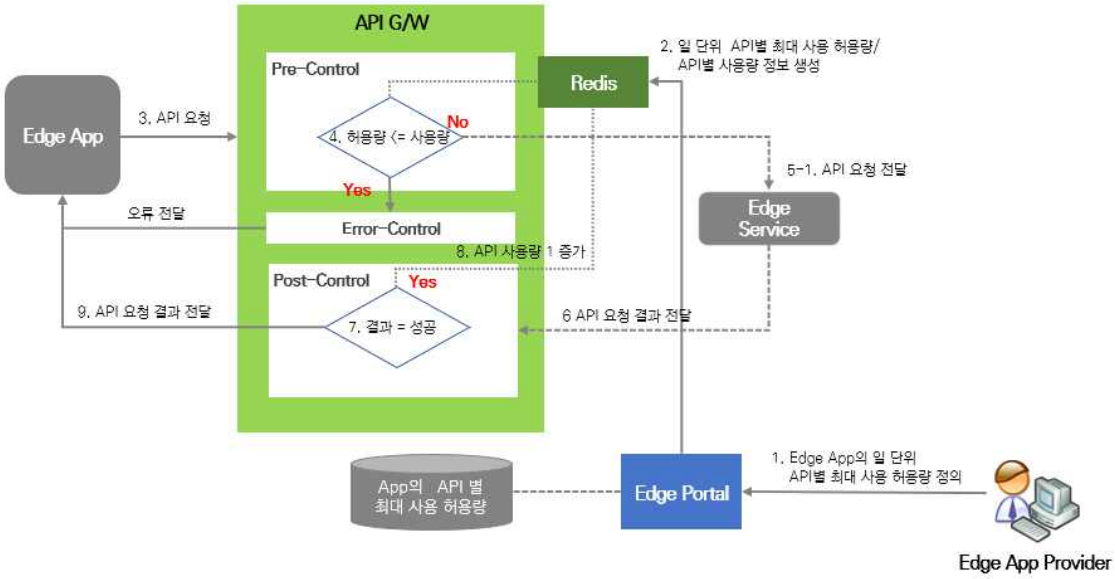
[엣지 서비스 라우팅 처리/Round Robbin 방식의 라우팅 분산 처리]

- 엣지 앱은 엣지 서비스 명이 포함된 API 요청을 API G/W로 요청
  (API 요청 URL : http://{api gw domain}/api/{service name}/{api path})
- API G/W는 API 요청 정보에서 엣지 앱의 clientId와 API 요청 URL 정보를 가지고 해당 앱이 요청한 API에 대한 접근 권한이 있는지를 API Access 정보 DB에서 확인하고, 권한을 갖지 않은 경우 오류처리 수행
- API G/W는 엣지 앱의 요청 URL을 파싱하여 요청된 엣지 서비스의 service name을 확인하고 Routing 정보 DB에서 service name을 기준으로 서비스 인스턴스의 Routing 정보를 확인
- DB에서 확인된 Routing 정보에 따라 해당 서비스 인스턴스로 API 요청을 전달
- 엣지 서비스 인스턴스가 복수 개 존재할 경우 각 서비스 인스턴스에 round-robin 방식으로 API 요청을 분배.
- API G/W는 Routing 정보에 존재하지 않은 서비스 관련 API 요청에 대해서는 오류처리

○ 엣지 앱의 API 요청에 따른 트래픽 조절과 결함 격리 기술 개발

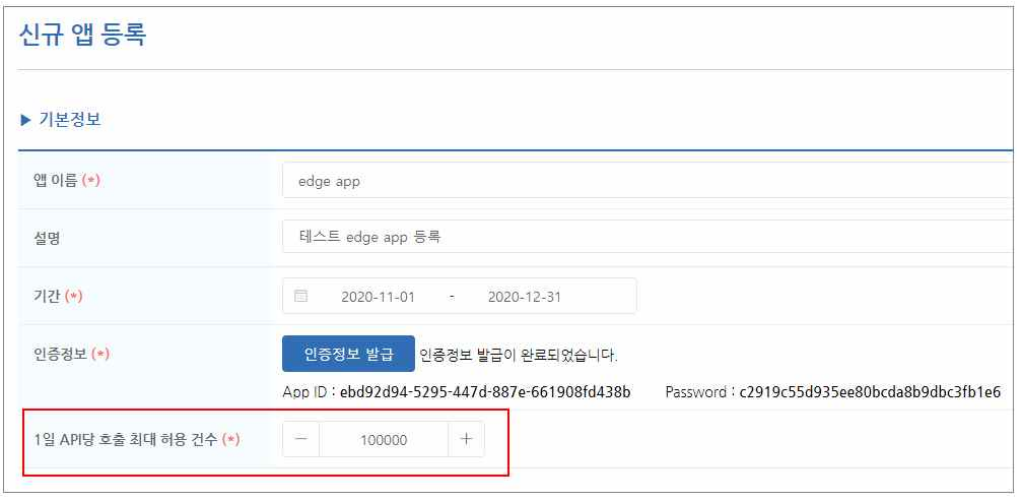
• 엣지 앱의 API별 일 단위 사용 허용량 기준에 따른 트래픽 조절 기능 개발

엣지 앱이 각각의 API를 사용할 수 있는 일 사용 최대 허용량을 등록하고 허용량을 초과한 요청에 대해 실패 처리하는 기능으로 아래와 같은 흐름으로 트래픽 조절이 이루어짐



[엣지 앱의 일별 API 사용 허용량에 따른 트래픽 조절]

- 엣지 앱 Provider가 Portal에서 엣지 앱 등록 시 하루 동안 사용할 수 있는 API 별 최대 사용 허용량을 설정.



[엣지 앱 등록 Portal 화면]

- Edge Portal은 엣지 앱의 API별 일단위 최대 사용 허용량 정보와 엣지 앱의 API별 사용량 정보를 Redis에 등록

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

구분	Key
엣지 앱의 API별 최대 사용 허용량	gw|accessLimit|{엣지 앱의 ClientId}|{method}|/api/{edge service name}/API Path
엣지 앱의 API별 사용량	gw|count|{엣지 앱의 ClientId}|{method}|/api/{edge service name}/API Path

[엣지 앱의 API별 일단위 최대 사용 허용량/사용량 key 구성]

- Post-Control에서는 엣지 앱의 API 요청을 수신하면 엣지 앱의 ClientId와 API 정보를 조합하여 API별 최대 사용 허용량 정보와 API 사용량 정보의 Key를 생성한 후 Key를 이용하여 허용량 정보와 사용량 정보를 Redis에 조회하고 두 값의 크기를 비교

엣지 앱의 ClientId = fc628cc8-6a81-438a-a304-7c6f7444f6c8 API 요청 URL = http://172.16.10.20/api/edge-service/serviceList Method = GET 인 경우	
엣지 앱의 API별 최대 사용 허용량	gw|accessLimit|fc628cc8-6a81-438a-a304-7c6f7444f6c8|GET|/api/edge-service/serviceList
엣지 앱의 API별 사용량	gw|count|fc628cc8-6a81-438a-a304-7c6f7444f6c8|GET|/api/edge-service/serviceList

[ClientId와 API 정보를 조합하여 생성한 허용량/사용량 정보 조회 Key 예]

- 사용량이 허용량을 초과할 경우 Error Control을 통해 엣지 앱에 오류 전달
- 사용량이 허용량 이하일 경우 엣지 서비스로 API 요청을 Routing 처리
- 엣지 서비스 API의 요청 결과를 수신하면 Post-Control에서 결과를 확인
- 결과가 성공인 경우 Redis에 사용량 정보 업데이트(1 증가)
- 엣지 앱에 API 처리 결과 전달

아래 코드는 엣지 앱의 API별 일단위 사용 허용량 기준에 따른 트래픽 조절 필터 코드임

```
@Override
public Object run() throws ZuulException {
  RequestContext ctx = RequestContext.getCurrentContext();

 //서비스 제한값
  String limitKey = createKey(PREFIX_ACCESS_LIMIT, getReqestClientId(),
                              getUriWithHttpMethod());
  int limitCount = listenerService.get(limitKey, Integer.class, 0);

 //서비스 접속 건수
  String countKey = createKey(PREFIX_ACCESS_COUNT, getReqestClientId(),
                              getUriWithHttpMethod());
  int connectionCount = listenerService.get(countKey, Integer.class, 0);

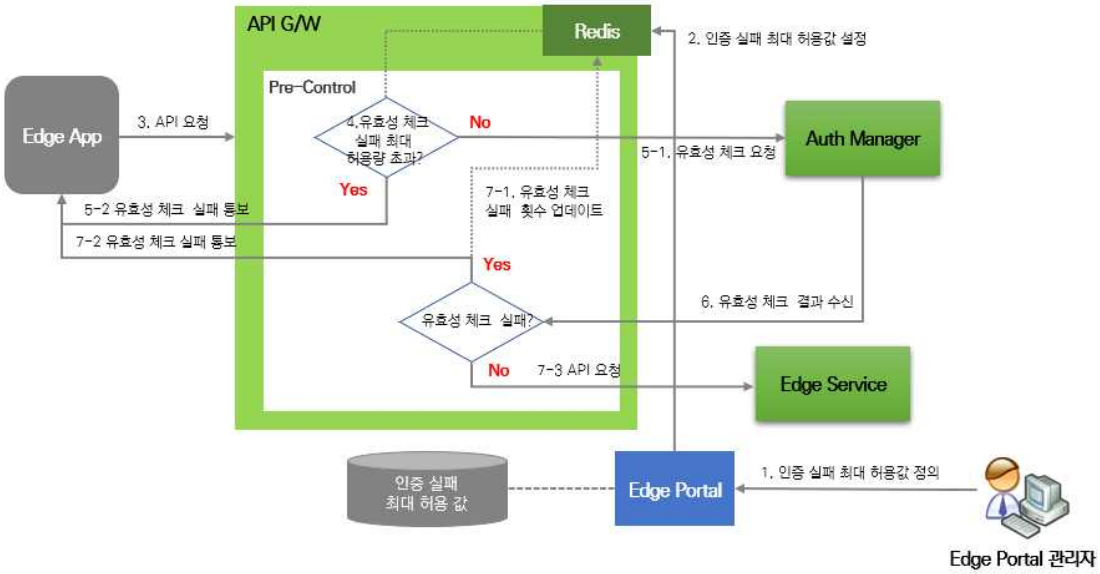
 //접속 횟수를 초과한 경우
```

```
    if(connectionCount >limitCount - 1) {
    return setResponse(ctx,
                    OpenAPIHttpStatus.EXCEEDED_CONNECTIONS_APP_ID_SERVICE_URI);
    }

    return null;
}
```

- 엣지 앱의 Access Token Validation 실패에 따른 트래픽 조절 기능 개발

엣지 앱의 API 요청 시 Header에 전달하는 Access Token에 대한 유효성 검증 실패 횟수가 인증 실패 최대 허용 값에 도달한 이후부터는 엣지 플랫폼의 API 서비스 이용을 제한하는 트래픽 제어 기능으로 아래와 같은 흐름으로 트래픽 조절이 이루어짐



[엣지 앱의 Access Token 유효성 체크 실패에 따른 트래픽 조절]

- Edge Portal 관리자가 관리 Portal에서 인증 실패 최대 허용값을 설정



[인증 실패 최대 허용값 설정 Portal 화면]

- Edge Portal은 인증 실패 최대 허용값 정보를 Redis에 등록(Access Token 유효성 체크 실패의 기준 정보도 인증 실패 최대 허용값 정보 활용)
- Post-Control에서는 엣지 앱의 API 요청을 수신하면 요청 메시지의 Header에 포함된 엣지 앱의 ClientId를 이용하여 엣지 앱의 Token 유효성 체크 실패 횟수

정보 Key를 생성한 후 Redis에서 Token 유효성 체크 실패 횟수와 인증 실패 최대 허용값을 조회하고 두 값의 크기를 비교

(ClientId가 xxxxx인 앱의 Token 유효성 체크 실패 횟수 정보 - key : gw|count Failed CheckToken|xxxxx, Access Token 횟수 정보는 Redis에 해당 앱이 최초 Token 유효성 실패 발생 시점에 생성)

- 실패 횟수가 최대 허용 값을 초과하는 경우 유효성 실패를 앱으로 전달
- 실패 횟수가 최대 허용 값 이하인 경우 Auth Manager로 유효성 체크 요청
- Auth Manger로부터 전달 받은 유효성 체크 결과가 실패인 경우 해당 앱의 유효성 체크 실패 횟수를 업데이트(실패 횟수+1)하고 유효성 체크 실패를 앱으로 전달
- 유효성 체크 결과가 성공인 경우 엣지 서비스로 API 요청을 Routing

아래코드는 엣지 앱의 Access Token 유효성 검증 실패에 따른 트래픽 조절 필터 코드임

```
@Override
public Object run() throws ZuulException {
    RequestContext ctx = RequestContext.getCurrentContext();
    String[] authArray = geTokenAndClientId(ctx);

    if(authArray == null) {
        return setResponse(ctx, OpenAPIHttpStatus.TOKEN_DOES_NOT_EXIST);
    }

    String token = authArray[0];
    String clientId = authArray[1];
    setReqestClientId(clientId);

    String limitedKey = PREFIX_LIMITED_FAILED_CHECKTOKEN;
    String countKey = createKey(PREFIX_COUNT_FAILED_CHECKTOKEN, clientId);

    int failedCount = listenerService.get(countKey, Integer.class, 0);
    int limitedCount = (redisService.exists(limitedKey) > 0) ? listenerService.get(limitedKey,
            Integer.class, 0) : 1;
     //실패 횟수를 초과한 경우
    if(failedCount > limitedCount - 1) {
        return setResponse(ctx, OpenAPIHttpStatus.EXCEEDED_CHECKTOKEN);
    } else {
        ValidTokenResult result = isValidToken(token);
        boolean isValid = result.isValidToken();

        if(isValid) {
            //실패 건수 초기화
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
                redisService.del(countKey);
            } else {
                if(failedCount == 0) {
                redisService.set(countKey, "1");
            } else {
                redisService.incr(countKey);
            }
            return setResponseWithBody(ctx, OpenAPIHttpStatus.INVALID_TOKEN, result.getCause());
            }
        }

        return null;
} private String[] geTokenAndClientId(RequestContext ctx) {
        String authorization = ctx.getRequest().getHeader("Authorization");
        if(authorization == null || !authorization.startsWith("bearer")) return null;
        log.info("authorization: {}", authorization);

        try {
            StringTokenizer stAuth = new StringTokenizer(authorization, " ");
            if(stAuth.countTokens() != 2) return null;
            stAuth.nextToken();
            String tokenStr = stAuth.nextToken();
            String decToken = AES256Util.decrypt(tokenStr, authAesKey);

            String [] stDec =  decToken.split(":");
            if(stDec == null || stDec.length != 2) return null;
            return stDec;

        } catch (UnsupportedEncodingException | GeneralSecurityException e) {
            log.error("", e);
        }

        return null;
}
```

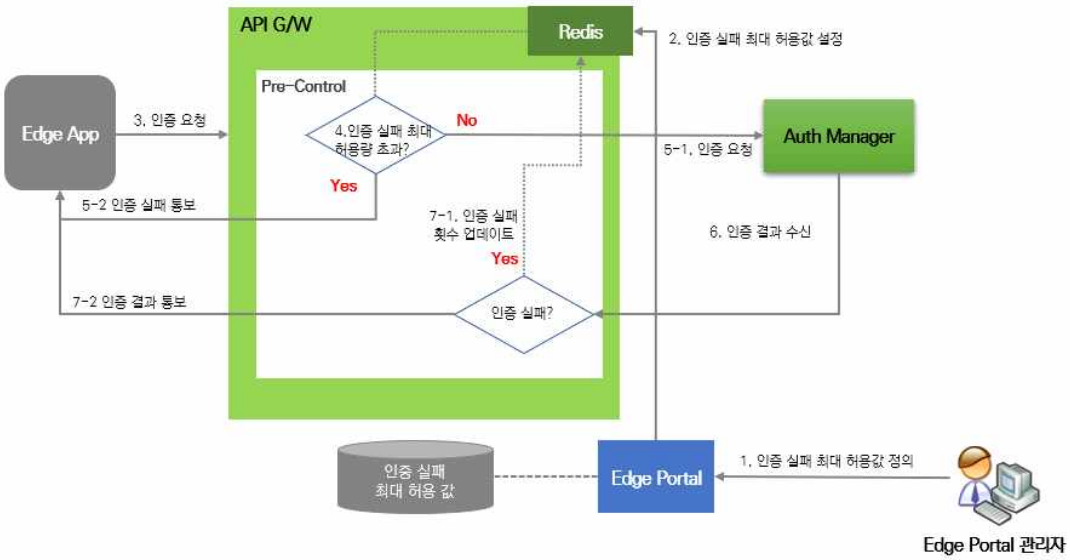
- 엣지 앱의 인증 실패에 따른 트래픽 조절 기능 개발

인증 실패 최대 허용값을 등록하고 인증 실패 횟수가 최대 허용값에 도달한 이후부터는 엣지 플랫폼의 API 서비스 이용을 제한하는 트래픽 제어 기능으로 아래와 같은 흐름으로 트래픽 조절이 이루어짐

- Edge Portal 관리자가 관리 Portal에서 인증 실패 최대 허용값을 설정
- Edge Portal은 인증 실패 최대 허용값 정보를 Redis에 등록(인증 실패 최대 허용값의 key : gw|limitedFailedGetToken)
- Post-Control에서는 엣지 앱의 인증 요청을 수신하면 인증 요청에 포함된 엣지 앱의 ClientId를 이용하여 엣지 앱의 인증 실패 횟수 정보 Key를 생성한 후

Redis에서 인증 실패 횟수와 인증 실패 최대 허용값을 조회하고 두 값의 크기를 비교(ClientId가 xxxxx인 앱의 인증 실패 횟수 정보 key : gw|countFailedGetToken|xxxxx, 인증 실패 횟수 정보는 Redis에 해당 앱이 최초 인증 실패 발생 시점에 생성)

- 실패 횟수가 최대 허용 값을 초과하는 경우 인증 실패 전달
- 실패 횟수가 최대 허용 값 이하인 경우 Auth Manager로 인증 요청
- Auth Manger로부터 전달 받은 인증 결과가 인증 실패인 경우 해당 앱의 인증 실패 횟수를 업데이트(1 증가)
- 엣지 앱에 인증 처리 결과 전달



[엣지 앱 인증 실패에 따른 트래픽 조절]

아래 코드는 엣지 앱 인증 실패에 따른 트래픽 조절 필터 코드임

```
@Override
public Object run() throws ZuulException {
  RequestContext ctx = RequestContext.getCurrentContext();
  HttpServletRequest request = ctx.getRequest();
  String basicAuth = request.getHeader("authorization");

  if(basicAuth == null || !basicAuth.startsWith("Basic")) {
    return setResponse(ctx, OpenAPIHttpStatus.AUTHORIZED_FAIL);
  }else {
    String descryptedAuth = getDecryptedString(basicAuth);
    if(descryptedAuth == null) {
      return setResponse(ctx, OpenAPIHttpStatus.AUTHORIZED_FAIL);
    }else {
      if(decodeHeader(ctx, descryptedAuth, "Authorization")) {
        return setResponse(ctx, OpenAPIHttpStatus.EXCEEDED_GETTOKEN);
      }
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
    }
  }

  return null;
}

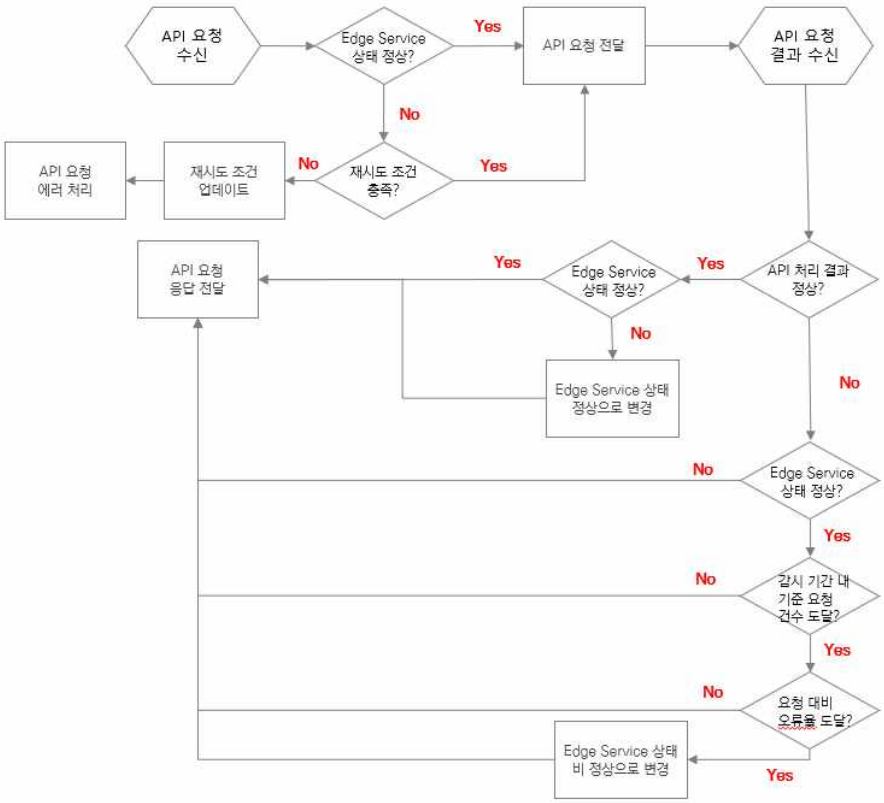
public String getDecryptedString(String basicAuth) {
  StringTokenizer stAuth = new StringTokenizer(basicAuth," ");
  if(stAuth.countTokens() != 2) {
    return null;
  }
  stAuth.nextToken();
  String decryptedAuth = basicAuth = stAuth.nextToken();
  try {
    return decryptedAuth = AES256Util.decrypt(basicAuth, authAesKey);
  } catch(UnsupportedEncodingException ¦ GeneralSecurityException e) {
    return null;
  }
}
protected boolean decodeHeader(RequestContext ctx, String decryptedAuth, String headerName) {
  String clientId = null;
  String clientSecret = null;
  if(decryptedAuth != null &&decryptedAuth.contains(":")) {
    String[] arrDecBasicAuth = decryptedAuth.split(":");
    clientId = arrDecBasicAuth[0];
    clientSecret = arrDecBasicAuth[1];
    setReqestClientId(clientId);


    String limitedKey = PREFIX_LIMITED_FAILED_GETTOKEN;
    int limitedCount = (redisService.exists(limitedKey) >0) ? listenerService.get(limitedKey,
Integer.class, 0) : 1;

    String countKey = createKey(PREFIX_COUNT_FAILED_GETTOKEN, clientId);
    int failedCount = listenerService.get(countKey, Integer.class, 0);
    if(failedCount >limitedCount - 1) {
      log.info("Exceeded the getting token failed count. AppId: {}, Limited Count: {}, Failed Count :
{}", clientId, limitedCount, failedCount);
      return true;
    }
    String StringClient = clientId + ":" + clientSecret;
    byte[] encodedClient = Base64.encodeBase64(StringClient.getBytes());
    String newBasicAuth = new String(encodedClient);
    ctx.addZuulRequestHeader(headerName, "Basic " + newBasicAuth);
  }
  return false;
}
```

• Circuit Breaker를 이용한 엣지 서비스 결함 대비 격리 기술 개발

엣지 서비스들의 서비스 상태를 감시하고 일정 기준 이상 오류 발생 시 해당 엣지 서비스로의 API 요청을 오류처리하여 비정상적인 엣지 서비스로 인한 Edge Platform 서비스 환경 전체 장애 발생을 방지하는 기능으로 아래와 같이 결함 격리가 이루어짐



[Circuit Breaker를 이용한 엣지 서비스 결함 격리]

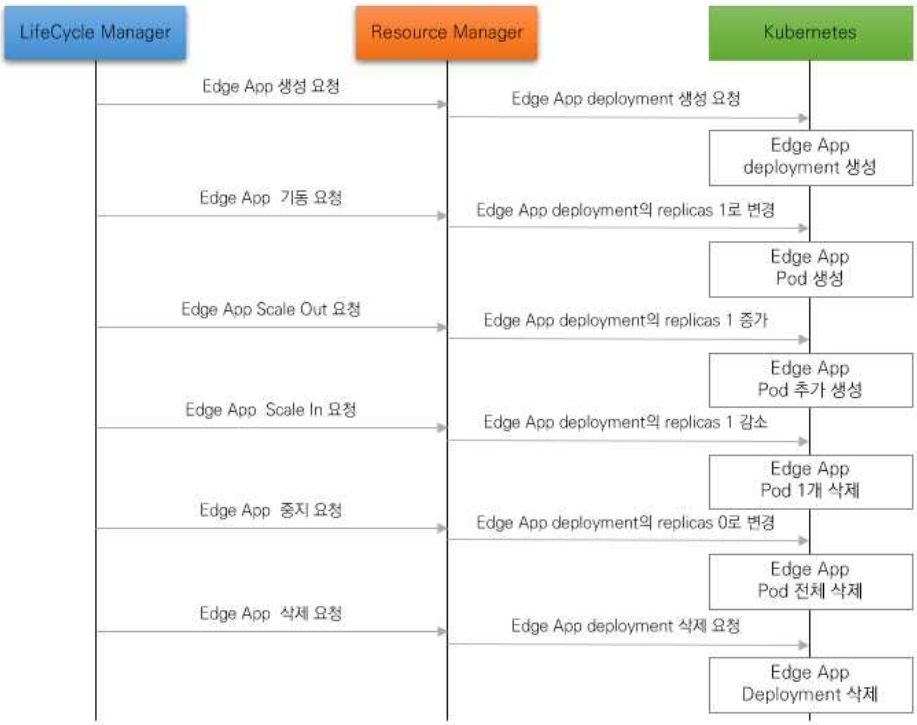
- API G/W로 엣지 서비스에 대한 API 요청 수신 시 Circuit Breaker는 엣지 서비스의 상태 확인
- 엣지 서비스의 상태가 정상인 경우 API 요청을 엣지 서비스로 전달
- 엣지 서비스의 상태가 비정상이고 재시도 조건을 충족하는 경우 API 요청을 엣지 서비스로 전달
- 엣지 서비스의 상태가 비정상이고 재시도 조건을 충족하지 않는 경우 재시도 조건을 업데이트하고 API 요청에 대해 에러 처리
- 엣지 서비스로부터 API 요청에 대한 응답을 수신하면 응답 결과를 확인
- 응답 결과가 정상이고 엣지 서비스 상태가 정상인 아닌 경우에는 엣지 서비스 상태를 정상으로 변경하고 API 요청 응답을 전달
- 응답 결과가 비정상이고 엣지 서비스 상태가 정상인 경우에는 감시 기간 내 기준 요청 건수 도달 여부와 요청 대비 오류율 도달 여부를 확인하고 두 조건을 만족할 경우 엣지 서비스 상태를 비정상으로 변경

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ Kubernetes 기반의 엣지 서비스/앱 컨테이너 관리 구조 개발

• Kubernetes Deployment 기반의 인스턴스 관리 기능 개발

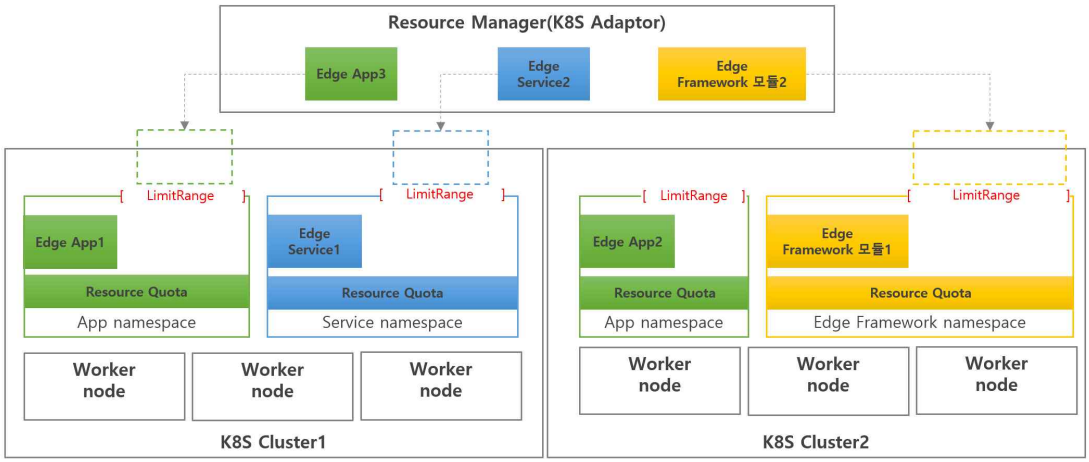
Kubernetes의 Deployment를 활용하여 엣지 앱/서비스, Edge Service Framework 구성 모듈의 인스턴스 생성, 변경, 삭제를 진행하고, deployment의 replicas 변경을 통해 인스턴스의 기동/중지/Scale In/Out을 처리함.



[deployment 기반의 엣지 앱 인스턴스 관리]

• Kubernetes NameSpace 기반의 배포 관리 기능 개발

엣지 앱/서비스, Edge Service Framework 구성 모듈의 리소스 과다 사용으로 인해 Edge Service Framework이나 다른 엣지 앱/서비스에 서비스 장애가 발생하는 일이 없도록 하기 위해 엣지 앱/서비스, Edge Service Framrwork 구성 모듈 배포 시 Cluster와 Namespace를 구분 지정하여 배포하고, Namespace에 Resource Quota와 Limit Range를 설정.



[Cluster와 NameSpace 기반 배포 관리]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ 엣지 앱과 엣지 플랫폼 구간 오픈 인터페이스 개발

• 3rd 엣지 응용을 위한 마이크로 서비스 기반의 플랫폼 표준 오픈 인터페이스 개발

엣지 플랫폼에서 3rd 엣지 앱에게 제공하기 위해 개발된 오픈 인터페이스는 아래와 같음

URL	Method	용도
/api/auth/oauth/api/client	POST	엣지 앱 인증 정보 생성 요청
/api/auth/oauth/token	POST	엣지 앱 인증 및 인증 토큰 요청
/api/{ServiceName}/{API Path}	POST	엣지 앱의 엣지 서비스 API 사용 권한 조회
/applications/start	POST	엣지 앱 기동 등록
/applications/terminate	POST	엣지 앱 종료 요청
/services	GET	엣지 서비스 목록 조회
/services/{ServiceName}	GET	특정 엣지 서비스 정보 조회
/services/available/{ServiceName}	GET	특정 엣지 서비스 가용성 조회
/services/subscriptions	POST	엣지 서비스 Routing 정보 등록
/applications/dns_rules	GET	DNS Rule 목록 조회
/applications/dns_rules/{dnsRuleId}	GET	특정 DNS Rule 정보 조회
/applications/time	GET	엣지 플랫폼 시간 조회

<u>엣지 앱과 Edge Service Platform간 API 예)</u>

– 엣지 앱 인증 정보 생성 요청 API

- Method: POST
- URI: /api/auth/oauth/api/client
- Request 파라미터

Name	위치	Type	설명
Content-Type	header	String	application/json
scope	body	String	인증정보 사용 범위
authorizedGrantTypes	body	String	인증 방식
expireDate	body	String	인증 정보 만료 기간

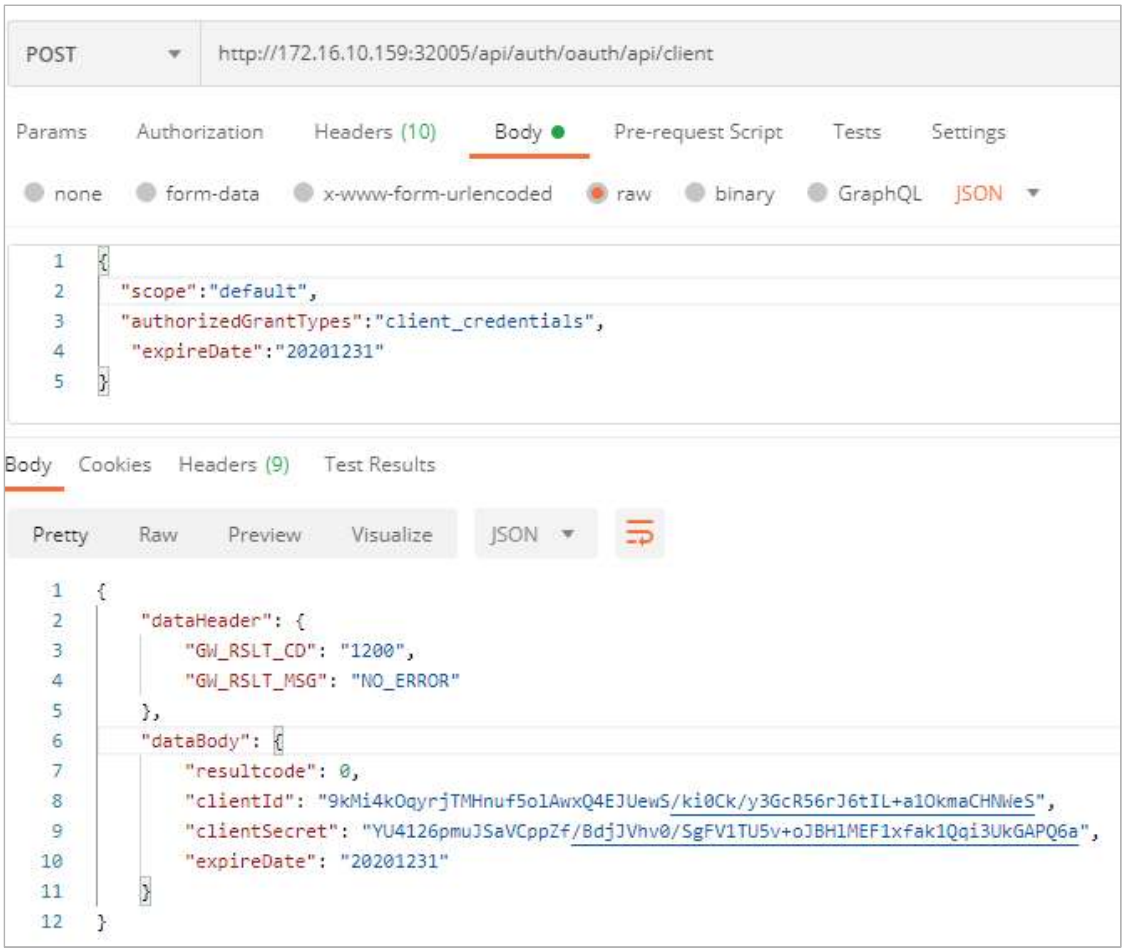
- Response 파라미터

Name	Type	설명
dataHeader	Object	data header
.GW_RSLT_CD	String	Gateway의 API 처리 결과 코드
.GW_RSLT_MSG	String	Gateway의 API 처리 결과 메시지
dataBody	Object	data body
resultcode	String	API 처리 결과 코드

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

.clientId	String	새로 생성된 엣지 앱의 ID
.clientSercet	String	새로 생성된 엣지 앱의 비밀번호
.expireDate	String	엣지 앱의 ID와 비밀번호의 만료 날짜

- API 실행 예



– 인증 및 인증 토큰 요청

- Method: POST
- URI: /api/auth/oauth/token?grant_type=credentials
- Request 파라미터

Name	위치	Type	설명
Authorization	header	String	“Basic" + " " + Base64Encode(aes256Encrypt(clientId + ":" + clientSecret))
grant_type	parameter	String	인증 방식

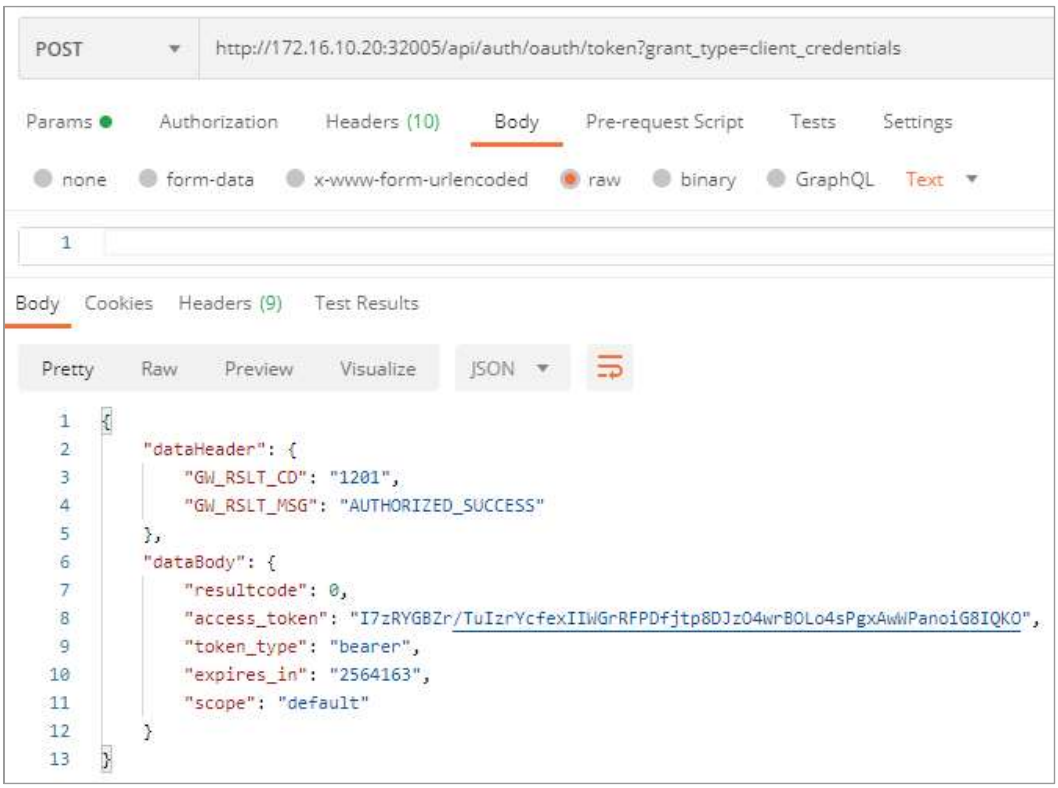
- Response 파라미터

Name	Type	설명
dataHeader	Object	data header
.GW_RSLT_CD	String	Gateway의 API 처리 결과 코드

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

.GW_RSLT_MSG	String	Gateway의 API 처리 결과 메시지
dataBody	Object	data body
.resultcode	String	API 처리 결과 코드
.access_token	String	엣지 앱이 API 요청 시 사용할 Access Token
.token_type	String	Token 타입
.expires_in	String	Token 유효
.scope	String	인증정보 사용 범위

▪ API 실행 예



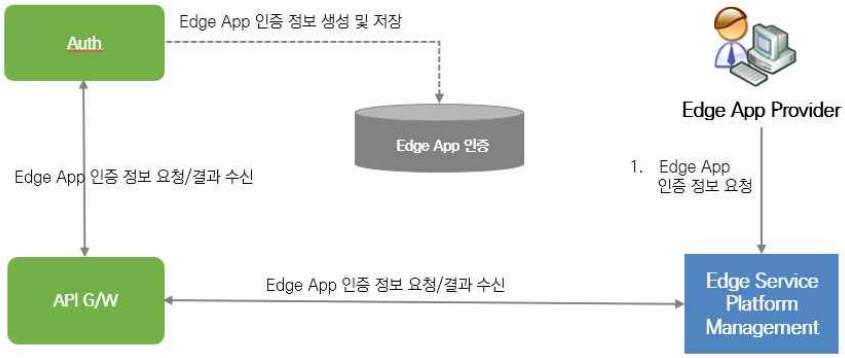
▢ 엣지 서비스 API 단위의 권한/허가 및 보안 인증 기능 설계 및 개발

○ OAuth2 기반의 인증 기능 설계 및 개발

• OAuth2 기반의 인증 서버와 연동을 통한 인증 정보 생성 기능 설계 및 개발

엣지 앱의 인증 정보는 App별로 할당되는 Unique한 ID(AppID)와 해당 ID에 대한 Password로 구성되며 Auth Server에서 생성 및 발급.

엣지 앱의 인증 정보 생성 및 발급은 아래와 같이 진행됨.



[엣지 앱 인증 정보 발급 과정]

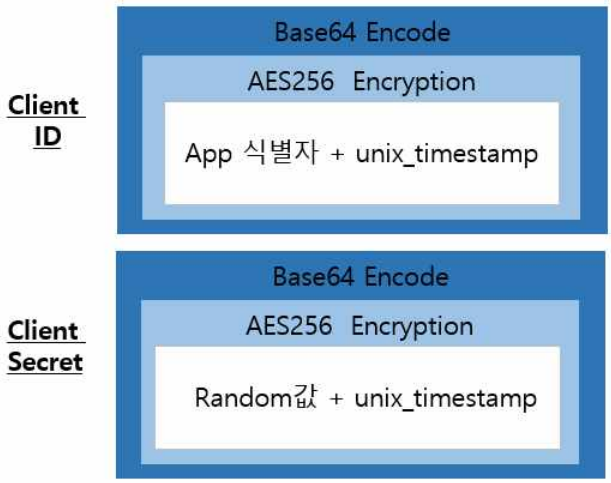
- 엣지 앱 Provider는 Edge 포털에서 엣지 앱 인증 정보 발급을 요청하고, 해당 요청은 API G/W를 거쳐 Auth Server로 전달.
- Auth Server에서는 Unique한 엣지 앱의 인증 정보(AppID, Password)를 생성하고 엣지 앱 인증 DB에 저장한 후 결과를 API G/W를 거쳐 Edge 포털로 전달.
- 엣지 앱 Provider는 엣지 앱 개발 시 엣지 앱 인증 정보를 엣지 앱 내에 포함시켜 인증에 활용

• 엣지 앱 정보 유효성 체크 및 Access Token 발급 기능 설계 및 개발

엣지 앱은 Edge Platform 내 엣지 서비스 API를 사용하기 위해 인증이 필요하며 인증후 서비스 접근에 사용할 Access Token을 발급 받으며 이후에는 해당 Token을 이용하여 API 서비스 요청 시 인증을 받음.

엣지 앱에 대한 인증 처리는 아래와 같이 진행됨.

- 엣지 앱은 인증 정보를 포함한 인증 요청을 API G/W로 전달.

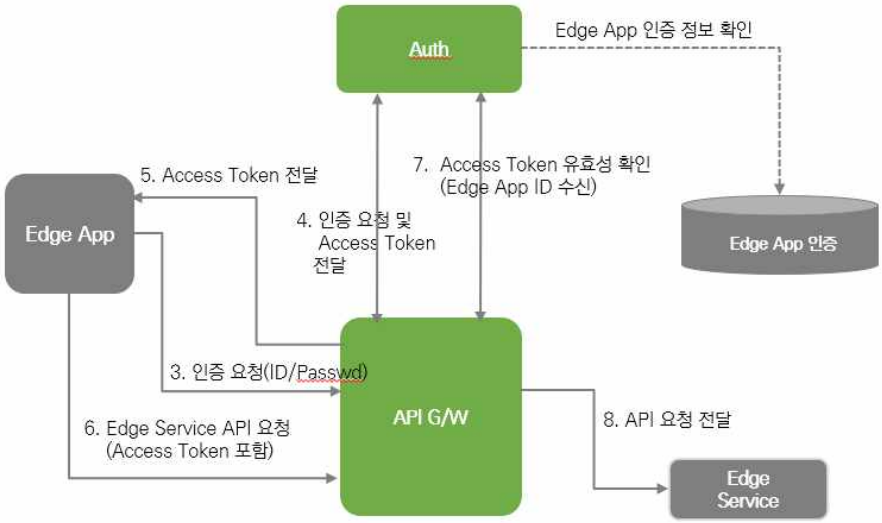


[인증 정보 전송 데이터 구성]

- API G/W는 인증 요청을 Auth 서버로 전달하고 Auth 서버는 인증 후 Access Token을 생성하여 전달
- API G/W는 Access Token을 엣지 앱으로 전달.
- 엣지 앱은 Access Token을 포함한 엣지 서비스 API 요청을 API G/W로 보냄
- API G/W는 Access Token을 Auth 서버에 전달하여 유효성 검증을 받고 유효성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

검사를 통과한 엣지 서비스 API 요청을 엣지 서비스 Instance로 Routing 처리



[OAuth2 기반의 인증 처리 흐름]

○ 엣지 앱의 엣지 서비스 API 단위 소유 및 사용 권한/허가 설정 코어 고도화

• 엣지 앱의 엣지 서비스 API 단위 접근 권한 정보 관리 구조 설계 및 개발

엣지 포탈을 통해 엣지 앱 등록 시, 사용할 API를 선택하면 해당 API의 사용 권한 정보가 메모리 DB(Redis)내에 Key&Value 형태로 등록되며, 사용 권한 정보는 해당 앱의 API 이용 정보 변경/삭제나 사용 기간 만료 시에 삭제 처리됨



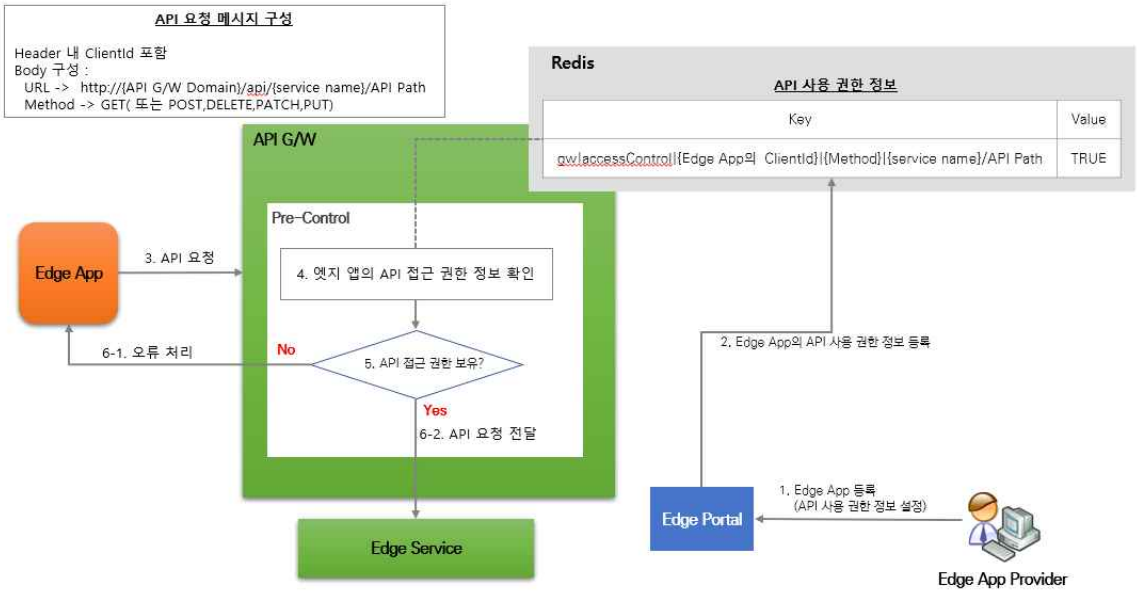
[엣지 포탈 내 엣지 앱의 API 사용 설정 화면]

gw|accessControl|{Edge App의 ClientId}|{Method}|{service name}/API Path

[엣지 앱의 API 사용 권한 정보 Key 구성 정보]

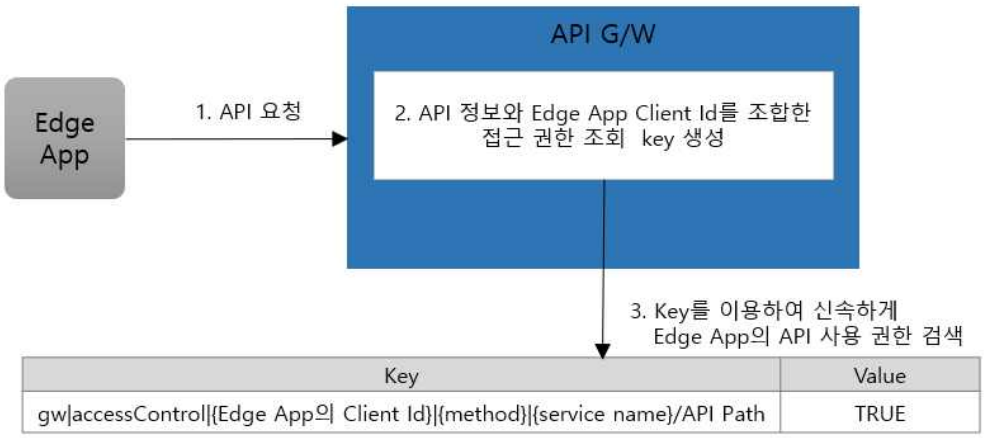
• 엣지 앱의 엣지 서비스 API 접근 권한 확인 기능 개발

엣지 앱에 대한 서비스 API 접근 권한 등록과 API 요청에 대한 접근 권한 체크 처리 흐름은 아래와 같음



[API 사용 권한 등록 및 권한 체크 흐름]

- 엣지 앱 Provider는 앱 등록 시 사용할 엣지 서비스 API 정보를 설정
- 엣지 포탈은 앱 등록 시 선택한 API 정보와 엣지 앱의 ClientId를 조합하여 메모리 DB(Redis)에 Key&Value 형태로 엣지 앱의 API 권한 정보를 등록
- 엣지 앱은 API 요청 시 자신의 ClientId를 포함하여 전달
- API G/W는 엣지 앱의 ClientId와 API 요청 URL, Method를 조합하여 API 사용 권한 정보 Key를 생성한 후 해당 키로 엣지 앱의 API 사용 권한을 메모리 DB에서 조회



App의 API 접근 권한 정보 DB 구조

[메모리Key-Value 구조의 엣지 앱 API 접근 권한 체크]

- 엣지 앱이 API 사용 권한을 갖고 있는 경우, API 요청을 엣지 서비스로 Routing 하고 권한이 없을 경우, 오류 처리

아래코드는 엣지 앱의 엣지 서비스 API 사용 권한 체크 코드임

```
@Override
public Object run() throws RoutingException {
```

```
            RequestContext ctx = RequestContext.getCurrentContext();

            String statusKey = getStatusKey();
            boolean status = Boolean.parseBoolean(listenerService.get(statusKey));

            if(!status) {
                    setNextFilter(ctx, false);

                    OpenAPIHttpStatus notAllowedStatus =
                notAllowedStatus(OpenAPIHttpStatus.ACCESS_NOT_ALLOWED_APP_ID_SERVICE_URI);
            return setResponse(ctx, notAllowedStatus);
            }

            return null;
    }
```

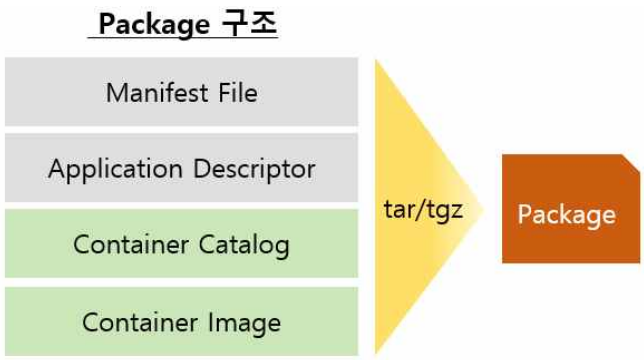
▢ 클라우드 엣지 플랫폼 라이프 사이클 관리 기술 개발

○ 패키지 및 인스턴스 배포/실행을 위한 라이프 사이클 관리 모델 설계 및 개발

- 엣지 앱/서비스 패키지 구조 설계 및 적용

엣지 플랫폼 컴포넌트, 엣지 앱, 엣지 서비스 Package는 아래 4가지 요소를 TAR로 묶어서 압축한 형태임

  - Manifest File : Package의 파일 구성 정보를 포함
  - Application Descriptor : Application의 ID 정보를 포함하여 해당 Application의 상세 구성 정보를 포함
  - Container Catalog : Container를 생성하는데 필요한 Spec 정보를 포함
  - Container 이미지 : application의 Container 배포 이미지



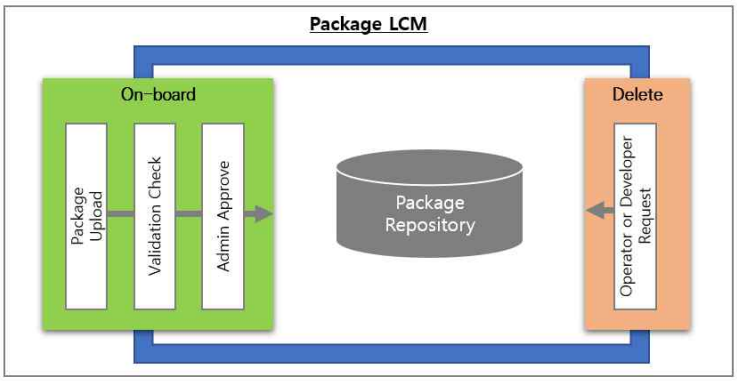
파일 종류	포맷	설명
Manifest file	JSON	package의 파일 구성 정보
Application Descriptor	XML	상세 구성 정보
Container Catalog	XML	K8S Catalog YAML 파일
Container Image	Binary	Container 이미지

[엣지 플랫폼, 엣지 앱/서비스 패키지 구조]

- 엣지 앱/서비스 패키지 라이프 사이클 모델 설계 및 적용

패키지 등록 시, 패키지 라이프 사이클 관리는 On-board와 Delete의 두 단계로

구분

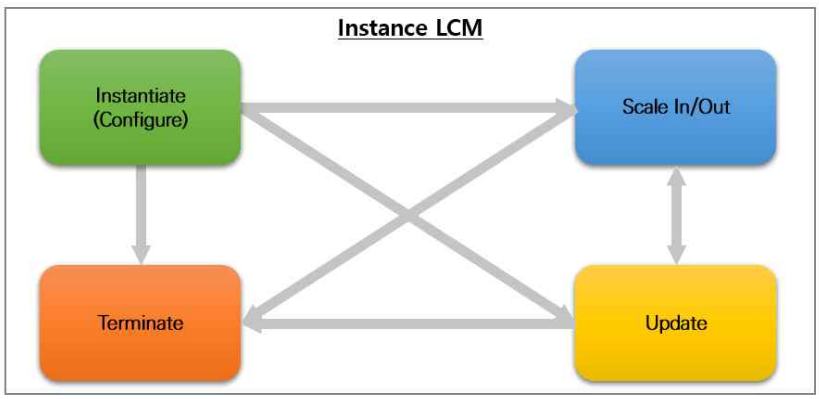


[패키지 라이프 사이클 관리 모델]

- On-board: 패키지 등록 시, 정해진 규격에 맞게 구성되었는지 Validation 체크 진행 후 관리자의 승인을 거쳐 Package Repository에 등록.
- Delete: 해당 패키지가 더 이상 사용할 필요가 없게 되는 경우, 개발자 또는 운영자가 엣지 포탈에서 해당 패키지의 삭제를 실행시키면 Package Repository에서 삭제 처리

• 엣지 앱/서비스 인스턴스 라이프 사이클 모델 설계 및 적용

인스턴스 라이프 사이클 관리는 Instantiate, Scale Out/In, Update, Terminate 총 4단계로 구분



[인스턴스 라이프 사이클 모델]

- Instantiate: Package를 이용하여 Application(Edge Platform Component/엣지 앱/엣지 서비스)의 Container를 생성하고 Application Instance가 기동 되고 Application 실행 관련 환경 설정을 진행.
- Scale Out/In: Application의 서비스 운영 중 필요에 따라 추가적인 Instance 생성이나 Instance 삭제를 진행.
- Update: Application의 실행 Container Spec 변경이나 Application의 Container 이미지 변경이 필요할 경우 그에 맞게 업데이트를 진행
- Terminate: Application의 서비스를 종료하고자 할 경우, Application Instance 삭제를 진행. 삭제 시 Application 관련해서 Edge Platform에 설정한 환경 설정(예, DNS Rule, 엣지 서비스 API 접근 권한 등)을 초기화.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 엣지 앱 및 엣지 서비스 라이프 사이클 관리 모듈 개발
  - 엣지 포탈을 통한 배포 요청에 의한 엣지 앱/서비스 인스턴스 생성/기동/중지/삭제/Scale In&Out
  - 자원 자동 스케줄링에 의한 Scale In&Out
  - LiveMigration에 의한 인스턴스 이동 정보 관리
  - 엣지 서비스간 Dependecy를 고려한 라이프 사이클 관리
  - 엣지 앱이 사용하는 엣지 서비스를 고려한 라이프 사이클 관리

아래 코드는 엣지 서비스간 Dependency를 고려한 라이프 사이클 관리 코드임

```
@ApiOperation(value = "배포 실행")
@RequestMapping(value = "/deploy", method = RequestMethod.POST)
public ResponseEntity<?> deployCreatedRunningContainer(@RequestBody RequestDeploy
                                                       requestDeploy)
  throws JSONException, InterruptedException {

  ArrayList<String> deployTargetDockerContainerIds = new ArrayList<String>();
  Deployment deployment = new Deployment();
  deployment.setId(requestDeploy.getDeploymentId());

  List<DeploymentContainer> deploymentContainers =
                          deploymentService.findDeploymentContainers(deployment);

  for (DeploymentContainer deploymentContainer : deploymentContainers) {
    List<DeploymentContainerDependency> DeploymentContainerDependencies =
                        deploymentService.findDeploymentContainerDependency(deploymentContainer);
    for (DeploymentContainerDependency deploymentContainerDependency :
                                  DeploymentContainerDependencies) {
      ResponseEntity<String> response = dockerService.stateContainer
                (deploymentContainerDependency.getDockerContainerId());

      JSONObject jsonObj = new JSONObject(response.getBody());
      String dockerStatus = (String) jsonObj.get("Status");

     if (dockerStatus.equals(DockerContainerStatus.CREATED.getValue())
         || dockerStatus.equals(DockerContainerStatus.EXITED.getValue())) {
        if (dockerStatus.equals(DockerContainerStatus.CREATED.getValue())) {
deploymentService.updateDeploymentRequestedAtByDeploymentContainer(
 deploymentContainerDependency.getDependencyDeploymentContainerId());
        }
     deployTargetDockerContainerIds.add(deploymentContainerDependency.getDockerContainerId());
        } else {
        if(!dockerStatus.equals(DockerContainerStatus.RUNNING.getValue())) {
          return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
          }
       }
    }

    deployTargetDockerContainerIds.add(deploymentContainer.getDockerContainerId());
  }
  deploymentService.updateDeploymentRequestedAt(requestDeploy.getDeploymentId());
```

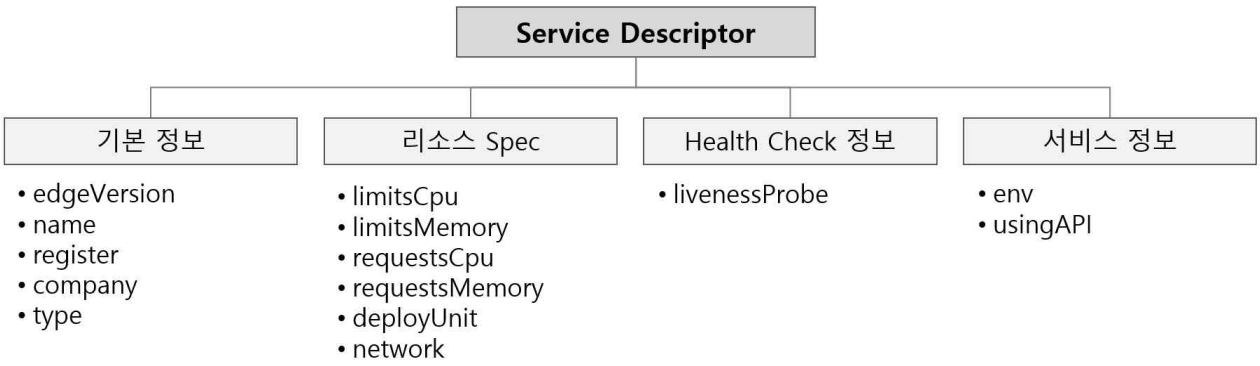
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
    bulkContainerStart(deployTargetDockerContainerIds);
    return ResponseEntity.ok(deployTargetDockerContainerIds);
}
```

○ 엣지 앱 배포/실행을 위한 네트워크 서비스 디스크립터 포맷 설계 및 개발

• 엣지 서비스/앱 서비스 디스크립터 설계 및 적용

– 엣지 서비스/앱 서비스 디스크립터 설계



[서비스 디스크립터 구성]

서비스 디스크립터는 아래와 같은 요소 포함

Attribute	Description
edgeVersion	엣지 플랫폼 버전
name	엣지 앱/서비스 이름
register	엣지 앱/서비스 등록자
company	엣지 앱/서비스 개발사
type	app or service
limitsCpu	엣지 앱/서비스 인스턴스가 사용할 수 있는 CPU 최대 크기
limitsMemory	엣지 앱/서비스 인스턴스가 사용할 수 있는 메모리 최대 크기
requestCpu	엣지 앱/서비스 인스턴스가 사용해야하는 CPU 최소 크기
requestMemory	엣지 앱/서비스 인스턴스가 사용해야하는 메모리 최소 크기
deployUnit	엣지 앱/서비스 배포 시 생성되어야 하는 컨테이너 수
network	엣지 앱/서비스의 network 종류(cluster ip or node port)
livenessProbe	엣지 앱/서비스의 정상 기동 여부 체크 방법
env	엣지 앱/서비스에서 사용할 환경 변수
usingAPI	엣지 앱이 사용할 엣지 서비스 API 정보

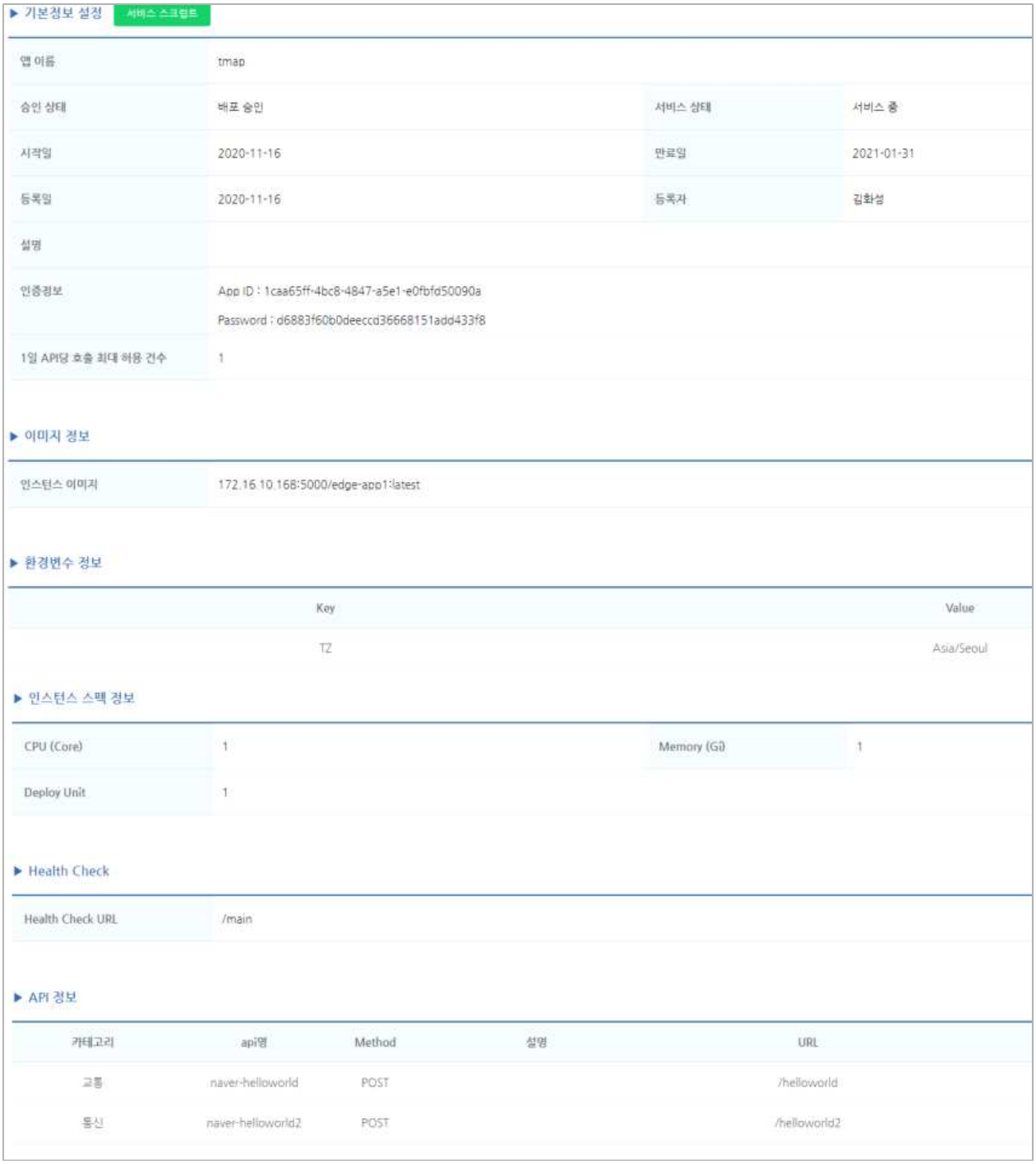
– 엣지 서비스/앱 등록 시 디스크립터 규칙에 맞게 구성된 디스크립터로 등록할

경우에도 인스턴스 기동 및 엣지 앱의 엣지 서비스 API 이용이 정상적으로 이뤄어질 수 있도록 하는 기능 개발

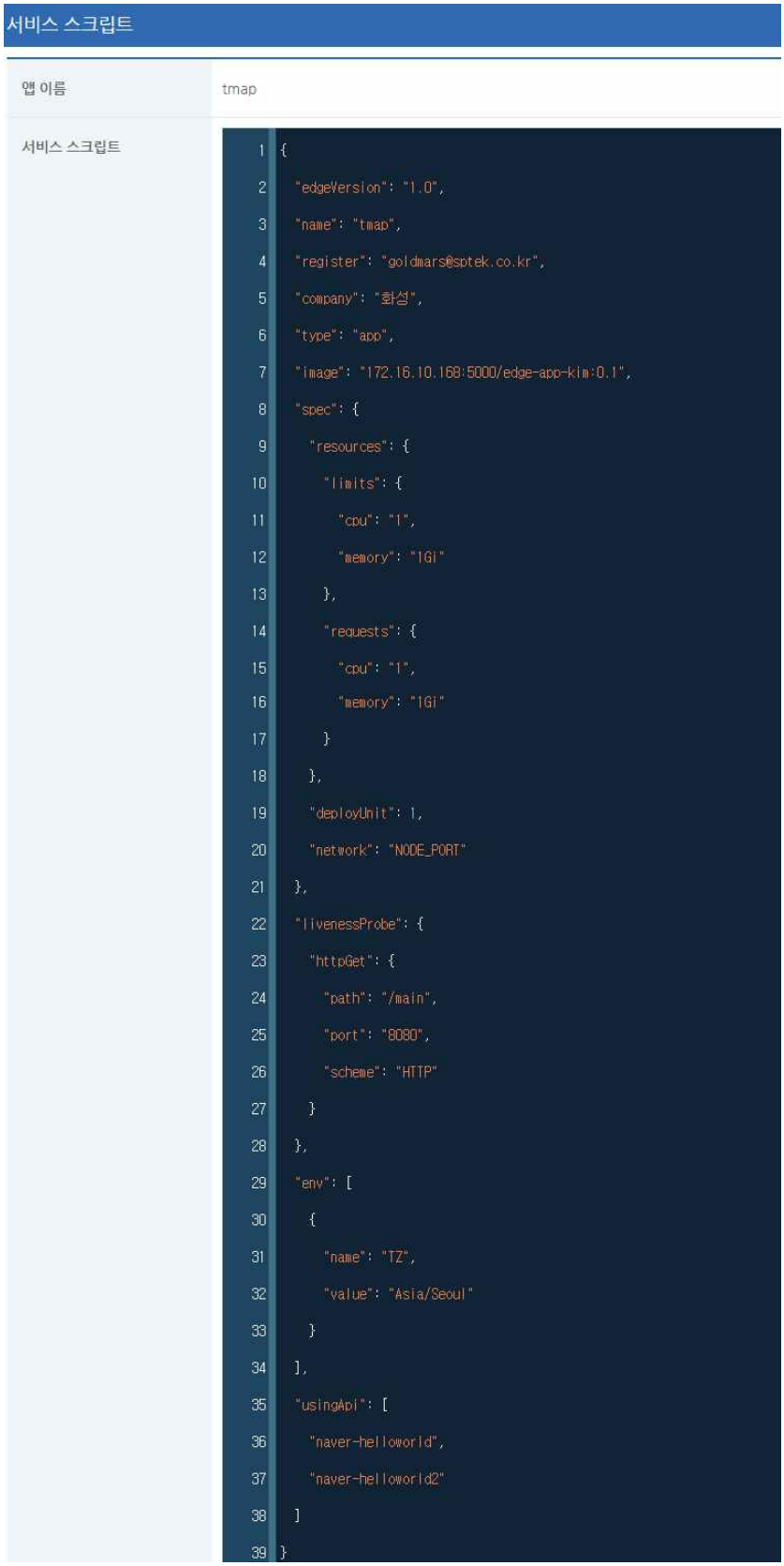
- 서비스 디스크립터 자동 생성 기능 개발

디스크립터 작성에 익숙하지 않은 개발자들을 위해 엣지 포털에서 정보 입력하면 해당 정보를 이용하여 디스크립터를 자동 생성

아래는 엣지 포탈에 등록된 엣지 앱 정보와 앱을 등록하여 자동 생성된 서비스 디스크립터 정보임



[엣지 포탈 내 엣지 앱 상세 정보 화면]

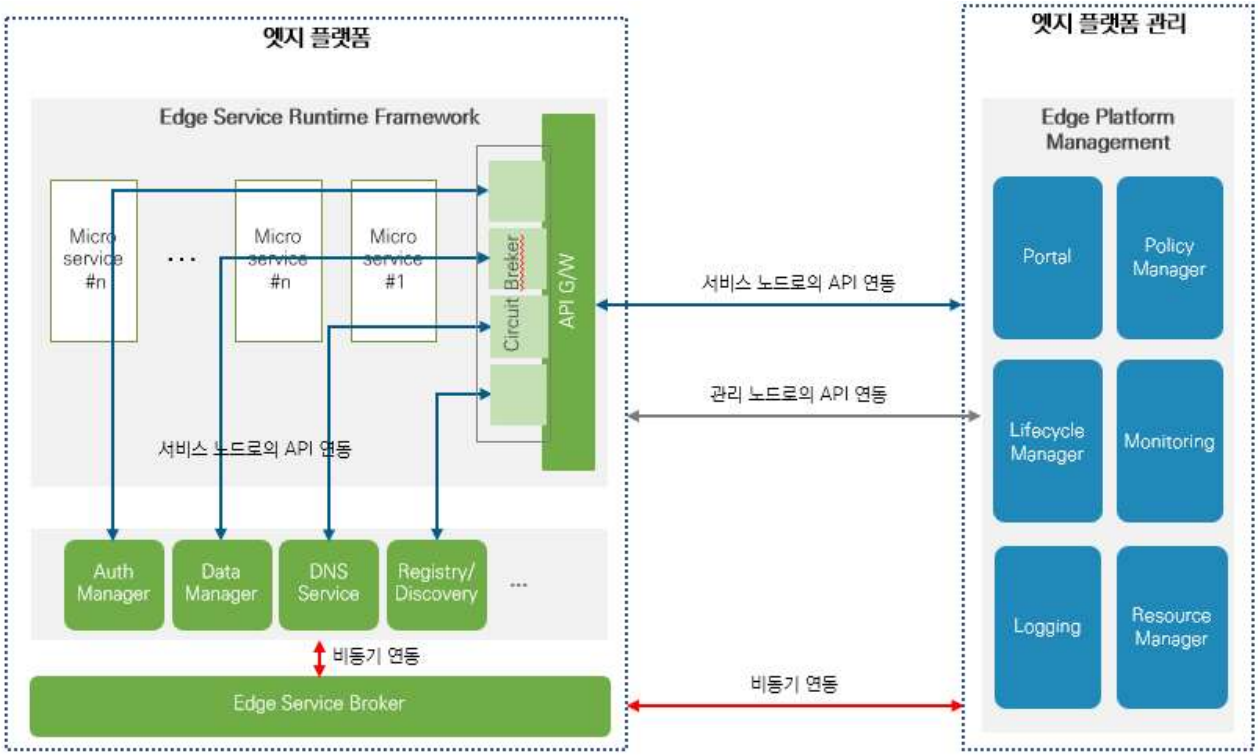


[엣지 포탈 내 엣지 앱의 서비스 디스크립터 상세 화면]

○ 엣지 플랫폼과 엣지 플랫폼 관리 구간 오픈 인터페이스 개발

- 엣지 플랫폼과 엣지 플랫폼 관리 간 운영 형상 구성 완료

엣지 플랫폼과 엣지 플랫폼 관리간 연동에 Rest API와 Queue 기반의 오픈 인터페이스 개발 적용하여 상호 종속성을 배제함.



[엣지 플랫폼과 엣지 플랫폼 관리간 연동 구성]

- 엣지 포탈에서 Auth Manager에 인증 정보 생성을 요청하는 것과 같이 엣지 플랫폼 관리 모듈에서 엣지 플랫폼 모듈로의 연동은 Rest API 기반으로 API G/W를 통해 진행
- 엣지 플랫폼 관리 모듈에서 엣지 플랫폼 모듈로의 연동 시 엣지 플랫폼 모듈 의 발생으로 인한 엣지 플랫폼 관리 모듈의 장애를 예방하기 위해 API G/W에 Circuit breaker 적용.
- LiveMigration에서 라이브마이그레이션 결과를 LifeCycle Manager에서 전달하는 것과 같이 엣지 플랫폼 모듈에서 엣지 플랫폼 관리 모듈로의 연동은 Rest API 기반으로 연동되며 플랫폼 관리 모듈로 직접 요청하는 방식을 적용
- API 사용 이력 전달이나 엣지 앱/서비스 인스턴스 등의 리소스 사용 정보 모니터링 정보 전달과 같이 비동기식으로 연동이 진행되는 경우는 Edge Service Broker에서 제공하는 Publish/Subscribe 방식의 Message Queue 연동을 적용

• 엣지 앱/서비스 인스턴스 관리 인터페이스 개발
  - 엣지 앱/서비스 생성
  - 엣지 앱/서비스 시작
  - 엣지 앱/서비스 업데이트
  - 엣지 앱/서비스 Scale Out
  - 엣지 앱/서비스 중지
  - 엣지 앱/서비스 삭제

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 엣지 앱/서비스 Scale In
- 엣지 앱/서비스 전체 인스턴스 정보 조회
- 엣지 앱/서비스 인스턴스 정보 상세 조회
- 엣지 앱/서비스 인스턴스 상태 조회

엣지 앱/서비스 인스턴스 관리 인터페이스 예)

엣지 앱 업데이트 API

- Method: POST
- URI: /api/apps/{app_id}/upgrade
- Request 파라미터

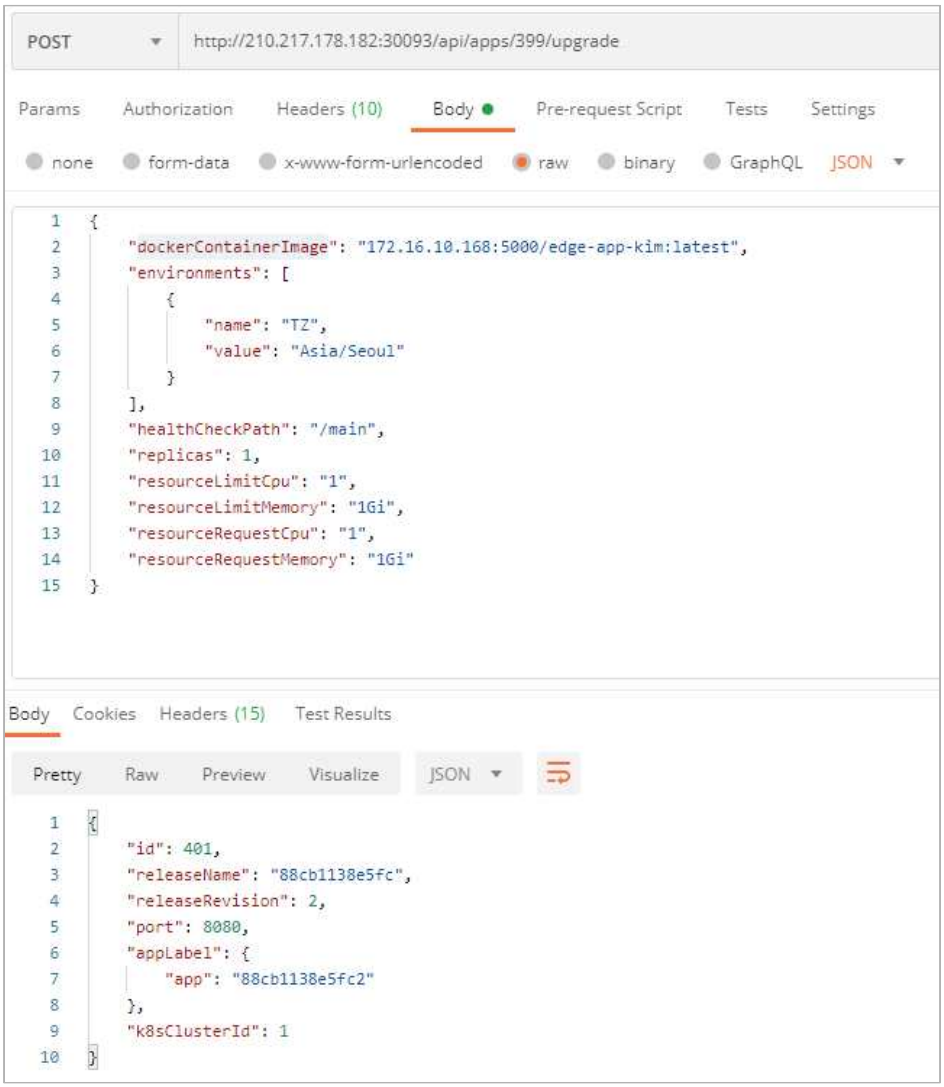
Name	위치	Type	설명
Content-Type	header	String	application/json
X-LCM-USER-ID	header	String	API 요청자 구분
app_id	Query	String	LifeCycle Manager에서 관리하는 엣지 앱/서비스의 Id
dockerContainerImage	body	String	엣지 앱/서버의 컨테이너 이미지
environments	body	Object	환경 변수 정보
.name	body	String	환경 변수 이름
.value	body	String	환경 변수 값
healthCheckPath	body	String	엣지 앱/서버의 상태 체크를 위한 http 호출 경로
replicas	body	String	엣지 앱/서버의 생성할 인스턴스의 수
resourceLimitCpu	body	String	인스턴스에서 사용할 수 있는 CPU 최대 사이즈
resourceLimitMemory	body	String	인스턴스에서 사용할 수 있는 메모리 최대 사이즈
resourceRequestCpu	body	String	인스턴스에 할당되어야하는 CPU 최소 사이즈
resourceRequestMemory	body	String	인스턴스에 할당되어야하는 메모리 최소 사이즈

- Response 파라미터

Name	Type	설명
id	String	새로 할당한 엣지 앱/서비스의 관리 id
releaseName	String	LifeCycleManager에서 관리하는 엣지 앱/서비스의 관리 이름
releaseVerison	String	엣지 앱/서비스의 배포 버전
port	String	엣지 앱/서비스 연동 시 사용할 포트 번호

appLabel	Object	엣지 앱/서비스 인스턴스를 Kubernetes에서 조회 시 사용할 참고 정보
.app	String	엣지 앱/서비스 인스턴스를 Kubernetes에서 조회 시 사용할 참고 정보
k8sClusterId	String	엣지 앱/서비스 인스턴스가 배포된 Kubernetes Cluster 정보

▪ API 실행 예



- 엣지 플랫폼 인프라 정보 관리 인터페이스 개발
  - 엣지 플랫폼 Cluster 전체 조회
  - 엣지 플랫폼 Node 전체 조회
  - 엣지 플랫폼 Node 동기화
  - 엣지 플랫폼 Name Space 전체 조회
  - 엣지 플랫폼 Name Space 정보 조회
  - 엣지 플랫폼 Name Space 생성
  - 엣지 플랫폼 Name Space 삭제
  - 엣지 플랫폼 Name Space 내 Limit Range 목록 조회

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 엣지 플랫폼 Name Space 내 특정 Limit Range 목록 조회
- 엣지 플랫폼 Limit Range 생성
- 엣지 플랫폼 Limit Range 삭제
- 엣지 플랫폼 Name Space 내 리소스쿼터 목록 조회
- 엣지 플랫폼 Name Space 내 특정 리소스쿼터 정보 조회
- 엣지 플랫폼 Name Space 내 리소스 쿼터 생성
- 엣지 플랫폼 Name Space 내 리소스 쿼터 삭제

엣지 플랫폼 인프라 정보 관리 인터페이스 예)

Edge 엣지 플랫폼 Cluster 전체 조회 API

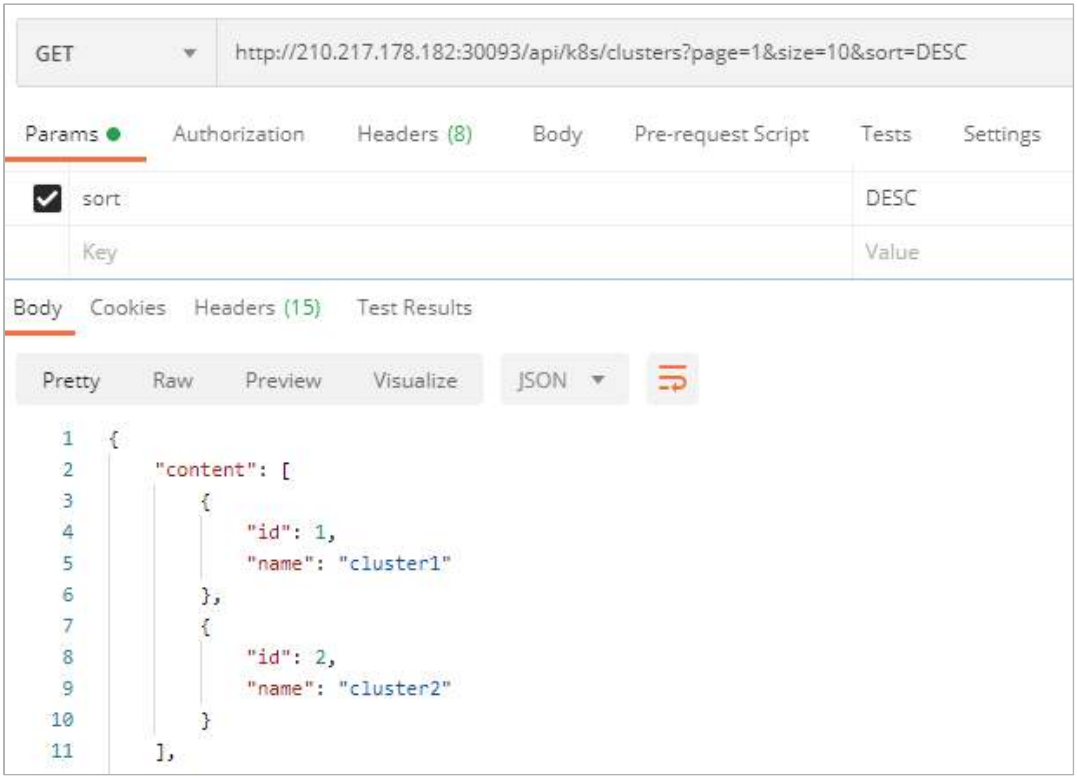
- Method: GET
- URI: /api/k8s/clusters?page=1&size=10&sort=DESC
- Request 파라미터

Name	위치	Type	설명
Content-Type	header	String	application/json
X-LCM-USER-ID	header	String	API 요청자 구분
page	query	String	kluster 정보 목록 페이지
size	query	String	kluster 목록의 항목 수
sort	query	String	kluster 목록 정렬 방법

- Response 파라미터

Name	Type	설명
content	Object	cluster 정보 목록
.id	String	cluster Id
.name	String	cluster 이름

- API 실행 예



▢ 정책 기반의 클라우드 엣지 자원 자동 관리 기술 개발

○ 자동 스케줄링을 위한 자원 워크로드 및 사용량 이벤트 수집 기술 개발

- 모니터링 및 이벤트 정책 관련 메트릭 정의
  - 엣지 앱/서비스 인스턴스의 CPU 사용률
  - 엣지 앱/서비스 인스턴스의 메모리 사용률
  - 엣지 앱/서비스 인스턴스의 Network I/O
- 엣지 앱/서비스 인스턴스의 리소스 사용량 관련 이벤트 정책 관리 기능 개발

  엣지 앱/서비스 인스턴스의 CPU/메모리 사용률, Network I/O 관련 인스턴스의 Scale In/Out 처리를 위한 이벤트 정책 관리 기능을 개발

  - 이벤트 정책 구성

  엣지 서비스에 대해 10초 간격으로 CPU 사용률을 체크하여 1분 동안 CPU 사용률이 90%를 초과한 경우가 연속적으로 3회 발생하면 Scale Out 이벤트를 생성하고, CPU 사용률이 10%에 미달하는 경우가 연속적으로 2회 발생하면 Scale In 이벤트를 생성하는 이벤트 정책

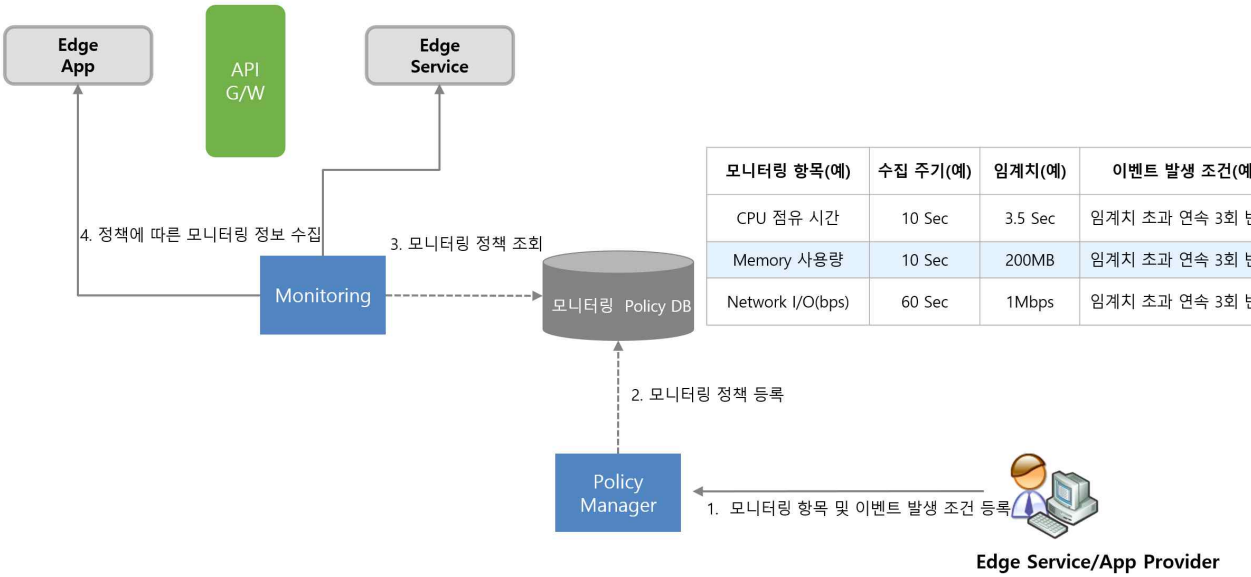
Event명	모니터링 대상	모니터링 정보	수집주기	임계치	판단 주기	발생 횟수	연속 여부
ScaleOut_CPU	EdgeService	CPU	10초	90 〉	1분	3회	연속
ScaleIn_CPU	EdgeService	CPU	10초	10 〈	1분	3회	연속

[이벤트 정책 구성 예]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

– 모니터링 및 이벤트 정책 등록 관리 흐름

모니터링 및 이벤트 정책 등록 관리는 아래와 같은 순서로 진행됨



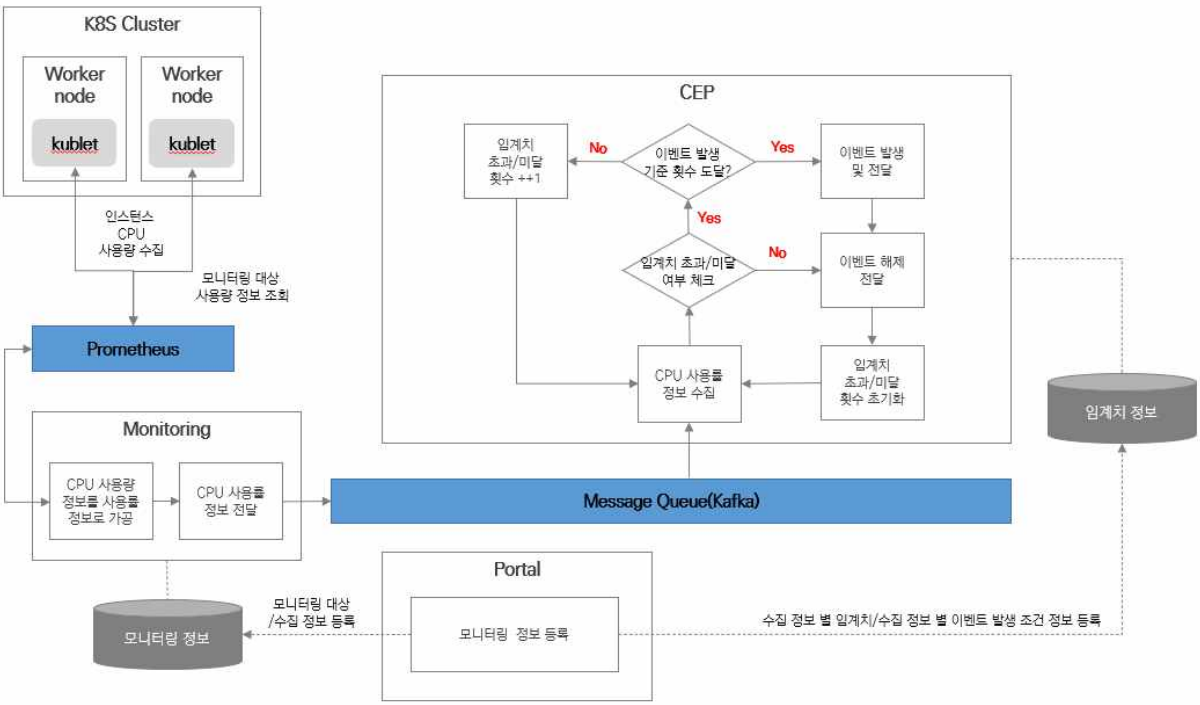
모니터링 항목(예)	수집 주기(예)	임계치(예)	이벤트 발생 조건(예)
CPU 점유 시간	10 Sec	3.5 Sec	임계치 초과 연속 3회 반복
Memory 사용량	10 Sec	200MB	임계치 초과 연속 3회 반복
Network I/O(bps)	60 Sec	1Mbps	임계치 초과 연속 3회 반복

[자원 워크로드 및 사용량 이벤트 정책 관리]

- 엣지 서비스/앱 개발자에 의해 모니터링 및 이벤트 정책 등록
- Policy Manager를 통해 모니터링 Policy DB에 저장
- 모니터링 Policy DB에 변경 사항 발생 시 Trigger에 의해 Monitoring에 전달
- Monitoring은 정책에 따라 모니터링 정보 수집 및 이벤트 처리

• 리소스 사용량 수집/분석 기능 개발

자동 스케줄링을 위한 리소스 사용량 수집 분석 및 이벤트 처리는 아래와 같은 과정을 거쳐서 진행됨



국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[모니터링 정보 수집 및 이벤트 발생 처리 흐름]

- 엣지 포탈에서 모니터링 대상(엣지 앱/엣지 서비스 인스턴스 등), 수집 정보(CPU 사용률, 메모리 사용률, Network I/O 등), 수집 정보별 임계치 정보, 수집 정보별 이벤트 조건 정보를 등록하여 DB에 저장
- Prometheus를 이용하여 K8S Cluster의 Work Node내 인스턴스들의 CPU/메모리/Network 사용량 정보 수집
- Monitoring에서는 모니터링 정보 DB의 정보를 참고하여 Prometheus를 통해 수집되는 정보 중 모니터링 대상의 수집 정보를 선별한 후 사용량 정보를 사용률로 변환하여 Message Queue를 통해 CEP로 전달(Network I/O의 경우는 사용량 정보를 그대로 CEP로 전달)
- CEP는 Monitoring으로부터 전달 받은 CPU/메모리 사용률 정보와 CPU/메모리별 임계치 정보 비교하고, 이벤트 발생 조건에 부합하는지 여부를 판단하여 이벤트를 발생시켜서 자동 스케줄링을 담당하는 Policy Manager로 쪽으로 전달. (Network I/O의 경우는 사용량 기반으로 처리)

아래 코드는 수집된 CPU 사용량 정보를 사용률 정보로 변환하는 Monitoring 코드임

```
@Override
public boolean doAction(PodDataInfo newPod) {
    try {
        if (!repository.existsById(newPod.getPodId())) return true;

        PodDataInfo oldPod = repository.findById(newPod.getPodId()).get();
        int cpu = calculateCPU(newPod, oldPod);
        newPod.getMetricMap().put("cpu_utilization", new BigDecimal(cpu));

        int memory = cacluateMemory(newPod, oldPod);
        newPod.getMetricMap().put("memory_utilization", new BigDecimal(memory));

        return true;
    }catch(Exception e) {
        log.error("error", e);
        return false;
    }
} private int calculateCPU(PodDataInfo n, PodDataInfo o) throws Exception {

    BigDecimal quota = n.getMetricMap().get("container_spec_cpu_quota");
    BigDecimal period = n.getMetricMap().get("container_spec_cpu_period");
    if(n.getScrapTime().equals(o.getScrapTime()) || quota == null) {
        return o.getMetricMap().get("cpu_utilization").intValue();
        //return 0;
```

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

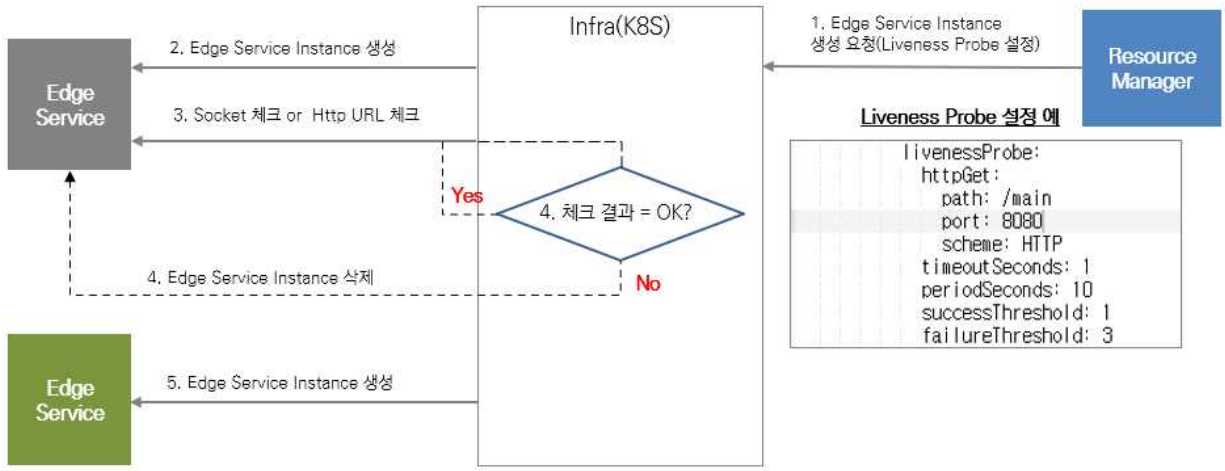
```
        }
        BigDecimal diffTime = n.getScrapTime().subtract(o.getScrapTime());
        BigDecimal diffValue =
n.getMetricMap().get("container_cpu_usage_seconds_total").subtract(o.getMetricMap().get("container_cpu
_usage_seconds_total"));
        BigDecimal div1 = diffValue.divide(diffTime, BigDecimal.ROUND_DOWN);
        BigDecimal div2 = quota.divide(period, BigDecimal.ROUND_DOWN);
        BigDecimal div3 = div1.divide(div2, BigDecimal.ROUND_DOWN);

        return (int) (div3.doubleValue() * 100);
}
```

○ 엣지 서비스의 부하 기반 자원 용량 프로비저닝 및 자동 복구 기술 개발

- 자동 복구 처리 로직 개발

  K8S Liveness Probe를 활용하여 인스턴스 자동 복구 기능 개발함. 인스턴스 자동 복구 기능 처리 흐름은 아래와 같음.

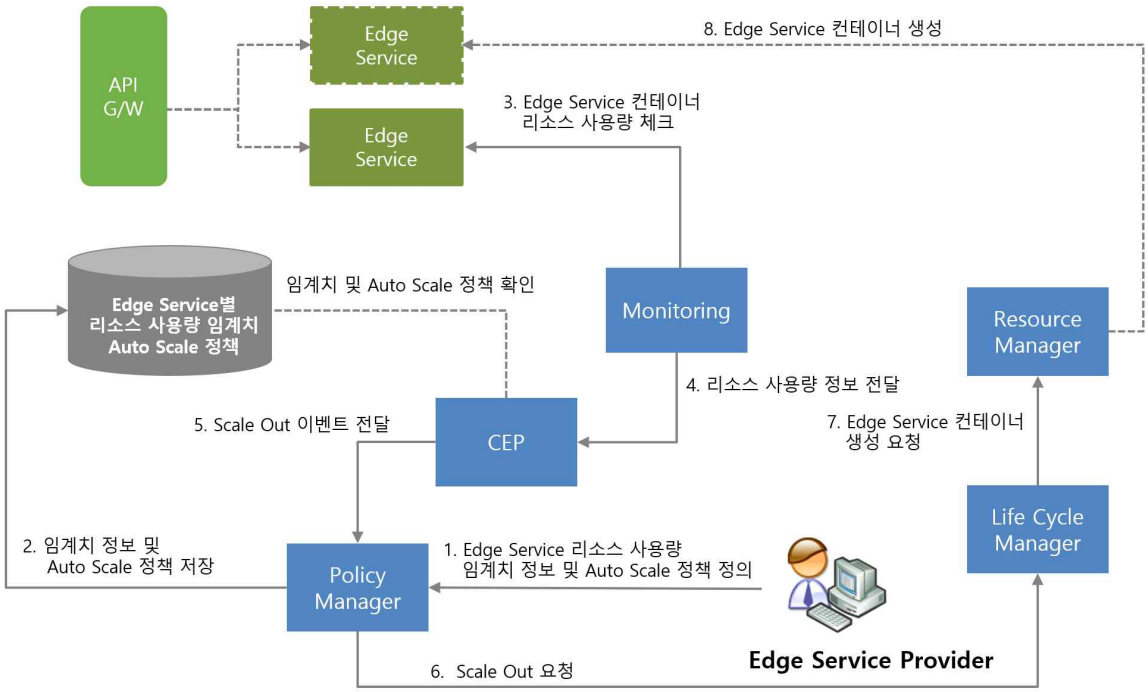


[Self Healing 처리 흐름]

  - 엣지 앱/서비스 인스턴스 생성 요청 시 Liveness Probe 관련 정보 설정 포함
  - 인스턴스 생성 후 주기적으로 Kubernetes에서 엣지 앱/서비스 인스턴스로 Liveness Probe 설정 정보에 따라 테스트 URL 호출
  - 테스트 URL 호출시 응답값이 에러값(2xx, 3xx가 아닌 모든 HTTP 상태 코드)이거나 응답하지 않는 경우 인스턴스 비정상 상태로 간주하고 해당 인스턴스를 삭제 후 새로운 인스턴스 생성

- 엣지 서비스 인스턴스 리소스 사용량 이벤트에 따른 Scale In/Out 처리 기능 개발

엣지 서비스 리소스 사용량에 따른 자원 자동 할당 관리 서비스는 아래와 같이 진행됨



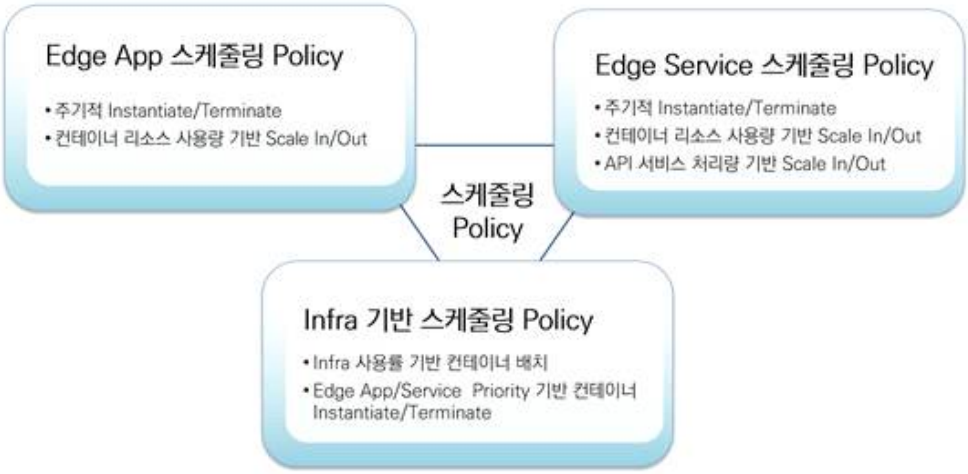
[Edge 서비스 컨테이너 부하 기반의 자동 확장 흐름]

- 엣지 서비스 Provider는 엣지 서비스 등록 시 해당 서비스의 리소스 사용량 임계치 정보(CPU/Memory 사용율, Network In/Out)와 리소스 사용량 임계치 관련 Auto Scale정책(최대량/최소량 연속 몇 회 반복 시 Auto Scale 진행, Scale In/Out Instance 수)을 포함하여 등록.
- Policy Manager는 엣지 서비스 Provider가 등록한 임계치 정보와 Auto Scale 정책 정보를 DB에 저장.
- Monitoring은 엣지 서비스 컨테이너의 리소스 사용량 정보를 주기적으로 수집하여 CEP에 전달
- CEP는 리소스 사용량 정보와 임계치 및 Auto Scale 정책을 확인하여 Scale Out 이벤트를 생성한 후 Policy Manager에 전달
- Policy Manager는 해당 엣지 서비스에 대한 Scale Out 진행 여부 확인 후 LifeCycle Manager에 Scale Out을 요청
- Life Cycle Manager는 Resource Manager에 엣지 서비스 컨테이너 생성을 요청하고 Resource Manager는 엣지 서비스 컨테이너를 생성

○ 다차원 메트릭 복합분석을 통한 자원 자동 스케줄링 기술 개발

• 자원 스케줄링 정책 관리 기술 개발

엣지 플랫폼의 자원 스케줄링 정책은 크게 아래와 같이 3가지로 구분함



[자원 스케줄링 정책]

- 엣지 앱 스케줄링 정책
  - 엣지 앱 개발자에 의해 등록 관리
  - 엣지 앱을 일정 주기 또는 일정 시간에 생성, 삭제, Scale In/Out
  - 엣지 앱이 실행되는 컨테이너의 리소스 모니터링 정보(CPU 사용률, Memory 사용률, Network 사용량) 조합 조건에 따라 컨테이너를 Scale In/Out
- 엣지 서비스 스케줄링 정책
  - 엣지 서비스 개발자에 의해 등록 관리.
  - 엣지 서비스를 일정 주기 또는 일정 시간에 생성, 삭제, Scale In/Out
  - 엣지 서비스가 실행되는 컨테이너의 리소스 모니터링 정보(CPU 사용률, Memory 사용률, Network 사용량)과 엣지 서비스의 기준 시간당 처리한 API 처리량 정보의 조합 조건에 따라 컨테이너를 Scale In/Out
- Infra 기반 스케줄링 Policy 정의
  - Node별 컨테이너 할당량을 고려하여 엣지 서비스와 엣지 앱의 컨테이너의 생성 위치 결정.
  - 리소스 부족 시 엣지 서비스와 엣지 앱별로 우선 순위에 따른 Scale In/Out

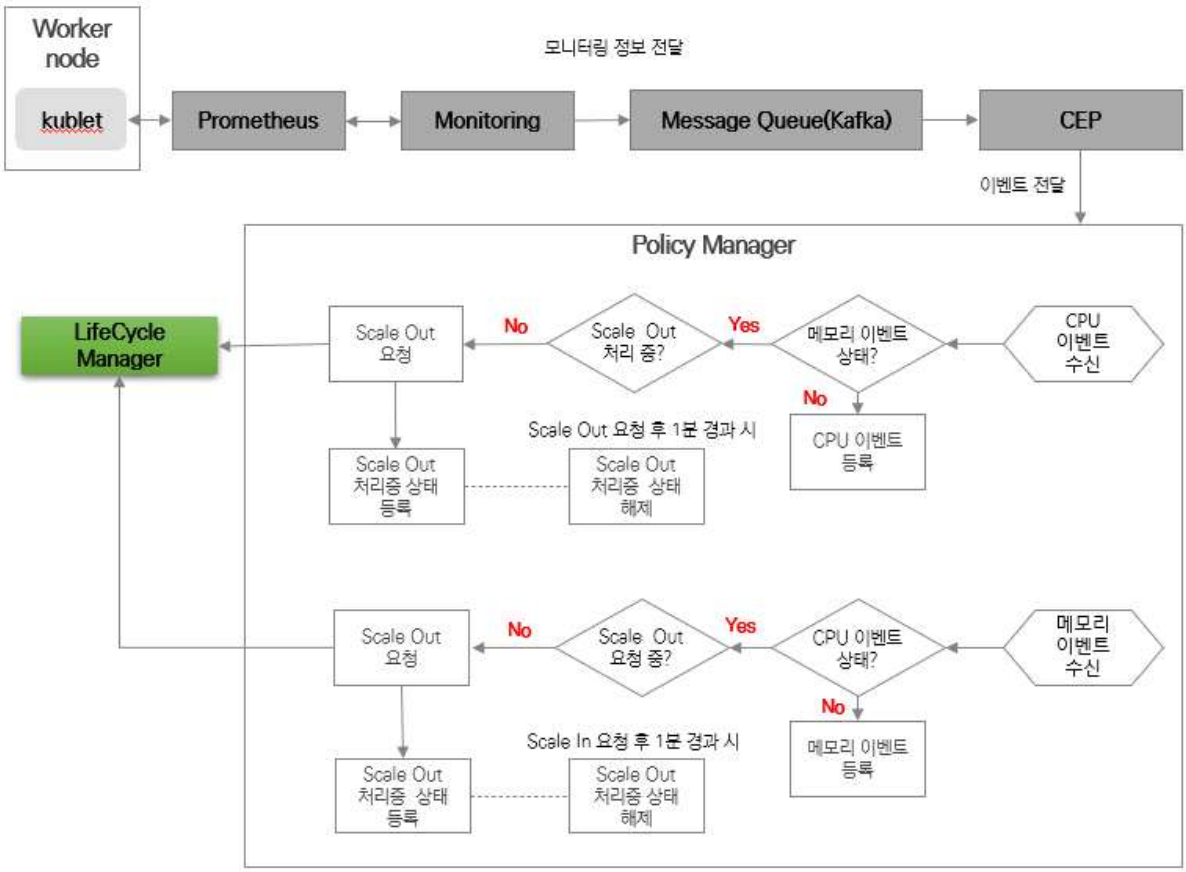
• 요청 및 정책에 따른 컨테이너 및 인프라 오케스트레이션 기술 개발
- 스케줄링 정책에 따른 자동 스케줄링 기능 개발
- 엣지 앱/서비스 스케줄링과 Infra 기반 스케줄링간 상호 연관성을 고려한 스케줄링 기능 개발

• 다차원 메트릭 복합 분석을 통한 자원 자동 스케줄링 기술 개발
- 다차원 메트릭 요소 정의 : CPU 사용률, 메모리 사용률, Network I/O, 일정 기간 동안 임계치 초과/미달 횟수, 임계치 초과/미달 연속 발생 여부 적용
- Scale In/Out 결정 요소 정의 : CPU 이벤트 발생, 메모리 이벤트 발생, CPU 이벤트 및 메모리 이벤트 동시 발생, Network I/O 이벤트 발생, CPU 와 Network I/O 이벤트 동시 발생

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 다차원 메트릭 요소와 Scale In/Out 결정 요소에 따른 엣지 서비스 자동 스케줄링 처리 기능 개발

CPU/메모리 사용률이 동시에 임계치 초과 이벤트 조건에 도달하여 자동 Scale Out이 처리되는 흐름은 아래와 같음



[다차원 메트릭 분석을 통한 엣지 서비스 자동 Scale Out 흐름]

- Monitoring을 엣지 서비스에 대한 CPU/메모리 사용률 정보를 수집하여 주기적으로 Message Queue를 통해 CEP에 전달
- CEP는 Monitoring으로부터 전달 받은 CPU/메모리 사용률 정보와 CPU/메모리 사용률 최대 임계치 정보와 비교하고 일정 기간 동안 임계치 초과 횟수, 임계치 초과 연속 발생 여부들을 고려하여 CPU와 메모리 사용률 관련 Scale Out 이벤트를 생성하여 Policy Manager로 전달
- Policy Manager는 CPU 관련 Scale Out 이벤트 수신 시 메모리 이벤트 등록 상태인지 확인하고 등록된 상태가 아닌 경우 CPU 이벤트 등록만 처리하고, 메모리 이벤트 등록 상태인 경우 Scale Out 처리가 진행 중인지 여부를 확인하고 Scale Out 처리 중이 아닌 경우 LifeCycle Manager로 Scale Out을 요청하고 Scale Out 처리 중 상태 등록(Scale Out 처리중 상태는 1분 경과 후 자동 해제 처리)
- Policy Manager는 메모리 관련 Scale Out 이벤트 수신 시 CPU 이벤트 등록상태인지 확인하고 등록 상태가 아닌 경우 메모리 이벤트 등록만 처리하고, CPU 이벤트 등록 상태인 경우 Scale Out 처리가 진행 중인지 여부를 확인하고

Scale Out 처리 중이 아닌 경우 LifeCycle Manager로 Scale Out을 요청하고 Scale Out 처리 중 상태 등록(Scale Out 처리중 상태는 1분 경과 후 자동 해제 처리)

- CPU 이벤트 등록 상태에서 1분 안에 Memory 이벤트를 수신하지 못하는 경우 CPU 이벤트는 해제 처리

아래 코드는 CPU와 메모리의 임계치 초과 조건을 동시에 만족할 때 Scale Out이 처리되도록 자동 Scale Out 정책이 설정되어 있는 상태에서 CEP로부터 CPU 관련 이벤트를 전달 받아 Scale Out 처리 여부를 판단하는 Policy Manager 코드임

```
@Override
public boolean updateDeployment(DeploymentInfo deployment, AlertDto alertDto) {
  if(deployment.getScaleState() == DeploymentScaleState.RUNNING) throw
                                        new ScaleStateRunningException();
  int ruleId = alertDto.getRuleId();
  EventKeyStateFactory eventkeyStateFactory = BeanUtils.getBean("eventkeyStateFactory",
                                                  EventKeyStateFactory.class);
  EventKeyState eventKeyState = eventkeyStateFactory.getEventKeyInstance(ruleId);
  eventKeyState.updateDeployment(deployment, alertDto, maxInterval);

  return true;
}
@Override
public boolean isReadyToScale(DeploymentInfo deployment) {
  DeploymentScaleState state = deployment.getScaleState();
  boolean cpuRequested = deployment.isCpuRequested();
  boolean memoryRequested = deployment.isMemoryRequested();

  if(!(state == DeploymentScaleState.RUNNING) && (cpuRequested && memoryRequested))
    return true;

  return false;
}
@Override
public DeploymentInfo getNewDeployment(AlertDto alertDto, String deploymentName) {
  int ruleId = alertDto.getRuleId();
  EventKeyStateFactory eventkeyStateFactory = BeanUtils.getBean("eventkeyStateFactory",
            EventKeyStateFactory.class);
  EventKeyState eventKeyState = eventkeyStateFactory.getEventKeyInstance(ruleId);

  return eventKeyState.getNewDeployment(alertDto, deploymentName);
}
@Override
public void updateDeployment(DeploymentInfo deployment, AlertDto alertDto, int maxInterval) {
  boolean storedCpuRequested = deployment.isCpuRequested();
  long storedCpuRequestTime = deployment.getCpuRequestTime();
  boolean storedMemoryRequested = deployment.isMemoryRequested();
```

```
    long storedMemoryRequestTime = deployment.getMemoryRequestTime();
    DeploymentScaleType storedType = deployment.getScaleType();

    String eventKey = alertDto.getTriggeringEventKey();
    long alertTime = DateUtils.stringDateToTimeMills(alertDto.getAlertTime());

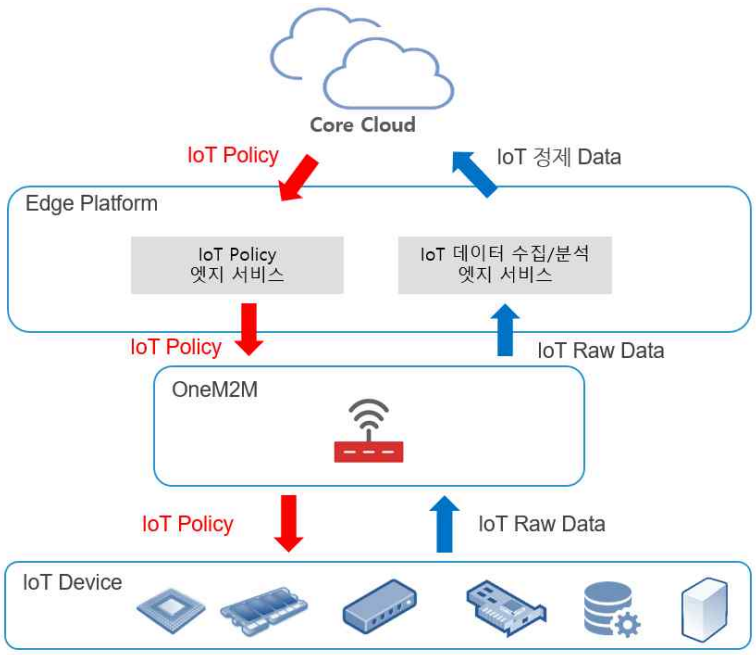
    if(storedMemoryRequested) {
      if(!storedType.isSameType(DeploymentScaleType.SCALE_OUT) || isOver(storedMemoryRequestTime,
            alertTime, maxInterval)) {
        deployment.setMemoryRequested(false);
      }
    }

    deployment.setScaleType(DeploymentScaleType.SCALE_OUT);
    deployment.setCpuRequested(true);
    deployment.setCpuRequestTime(alertTime);
}
```

▢ IoT 응용 서비스 지원을 위한 클라우드 엣지 플랫폼 통합

○ OneM2M(Mobius 플래폼 or WSO2) 연결을 위한 엣지 서비스 통합

- OneM2M IoT Gateway와의 연계를 통한 IoT 데이터 수집/분석 및 IoT Device 제어
  - 다양한 IoT Device의 데이터 수집/분석을 지원하는 엣지 서비스 개발
  - IoT Device별 정책 관리 및 정책 적용 엣지 서비스 개발
  - IoT Device와 Core Cloud의 IoT 서비스간 연계 지원 엣지 서비스 개발



[OneM2M – 엣지 플랫폼 연계]

▢ 클라우드 엣지 플랫폼 기능명세 개발

○ 3rd 엣지 앱 개발사를 위한 클라우드 엣지 플랫폼 표준 기능명세 규격 개발

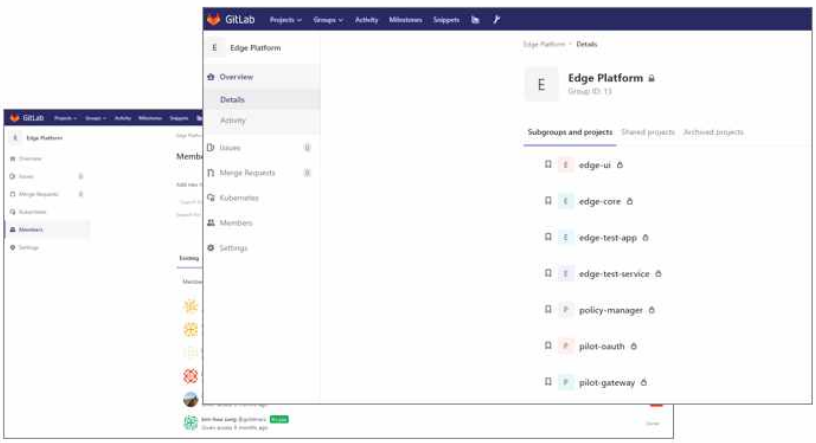
- 엣지 앱/서비스 이미지 등록
  - 엣지 앱/서비스 이미지 등록 방법 및 규칙 정의
- 엣지 앱 인증 정보 생성 및 인증 요청 규격
  - 엣지 앱 인증 정보 생성 및 등록 가이드, 엣지 앱 인증 요청 및 응답 규격 정의
- 엣지 앱 Descriptor 구조
  - 엣지 앱 등록을 위한 Descriptor 구조 정의
- 엣지 앱 패키지 구성
  - 엣지 앱 등록을 위한 패키지 구성 요소 및 요소별 구성 정보 정의
- 엣지 플랫폼 내 엣지 앱 등록 가이드
  - 엣지 플랫폼 내 포털을 이용한 엣지 앱 등록 가이드 정의
- 엣지 서비스 제공 API 이용 가이드
  - 엣지 서비스에서 제공하는 API 목록 제공 및 API 규격/샘플 코드 제공
- 엣지 앱 로그 생성 및 모니터링 규격
  - 외부에서 조회 가능한 앱 로그 생성 방법 및 로그 및 모니터링 정보 조회 규격 정의

▢ 최종 개발 결과물의 통합 및 시험

○ 각 기관 결과물 통합, 연동 시험

- 주관사 내 협업 소스 관리 솔루션(Gitlab)을 구축하여 주관/공동 기관 소스 통합

주관사에 설치된 GitLab 서버에 Edge Platform 그룹 생성/과제 관련 모듈 프로젝트 생성 및 소스 통합, 그룹 사용자 관리를 통해 Edge Platform 모듈 프로젝트에 대한 사용자 접근 통제

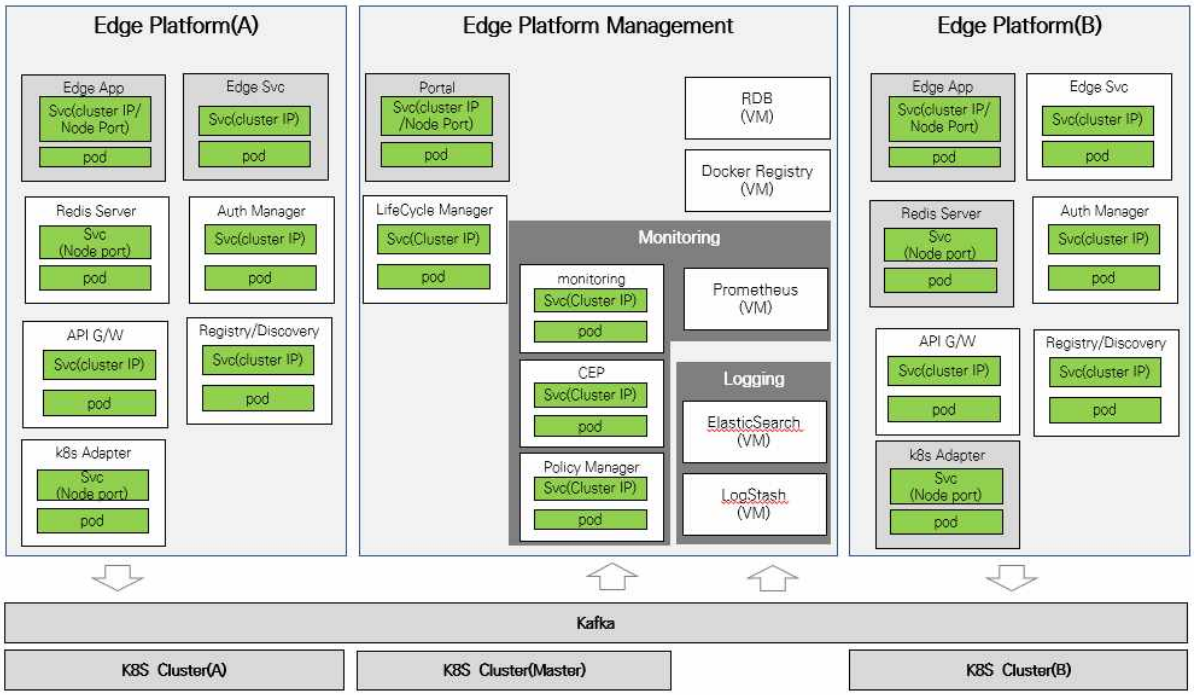


[Gitlab내 Edge Platform 그룹 내 프로젝트 및 그룹 사용자 관리 화면]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- 주관사에 K8S 기반의 클라우드 엣지 플랫폼 연동 테스트 환경 구축

3개의 K8S Cluster를 구성하고 2개의 K8S Cluster에는 각각 Edge Platform을, 1개의 K8S Cluster에는 Edge Platform Management를 설치, RDB, Docker Registry, 모니터링을 위한 Promethueus, Logging을 위한 Elastic Search, Logstash 등은 K8S Cluster가 아닌 개별 VM에 설치



[테스트 베드 구성도]

- 전체 기관 결과물에 대한 기능/통합 연동/정량적 목표 항목 등에 대한 자체 테스트 수행
- 테스트 오류 항목 보완 및 개선 진행

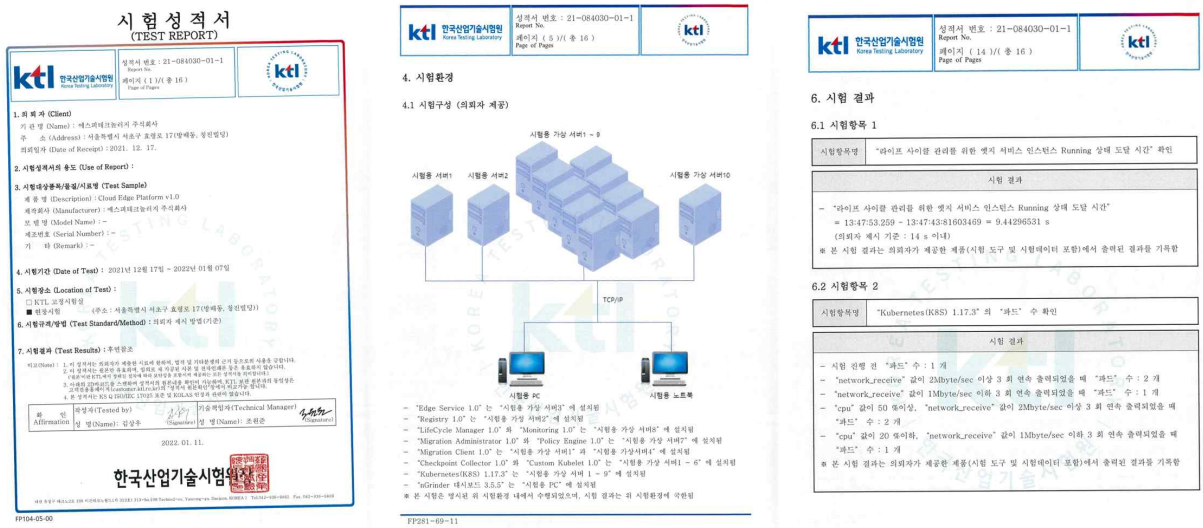
○ 기 구축된 클라우드 엣지 시스템 상에서의 최종 개발 결과물 시험 및 분석

- 정량적 목표 항목에 대한 공인인증 시험 수행

주관사에 기 구축된 클라우드 엣지 시스템 상에서 정량적 목표 항목 6가지에 대해 테스트를 진행

- 테스트 일자 : 2021년 12월 22일
- 테스트 기관 : KTL(한국산업기술시험원)
- 테스트 장소 : 주관사 본사 7층(스트라토)
- 테스트 참여자 : 김상후(KTL), 김화성(스트라토), 안재훈(KETI)
- 테스트 항목 : 라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달시간 측정 외에 5건(총 6건)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



시 험 성 적 서
(TEST REPORT)

ktl 한국산업기술시험원 Korea Testing Laboratory

성적서 번호 : 21-084030-01-1 Report No.
페이지 (1)/(총 16) Page of Pages

1. 의 뢰 자 (Client)
기 관 명 (Name) : 에스케이텔레콤㈜
주 소 (Address) : 서울특별시 서초구 효령로 17(방배동, 청진빌딩)
의뢰일자 (Date of Receipt) : 2021. 12. 17.

2. 시험성적서의 용도 (Use of Report) :

3. 시험대상품목/물질/시료명 (Test Sample)
제 품 명 (Description) : Cloud Edge Platform v1.0
제작회사 (Manufacturer) : 에스케이텔레콤㈜
모 델 명 (Model Name) : -
제조번호 (Serial Number) : -
기 타 (Remark) : -

4. 시험기간 (Date of Test) : 2021년 12월 17일 ~ 2022년 01월 07일

5. 시험장소 (Location of Test) :
□ KTL 고정시험실
■ 현장시험 (주소 : 서울특별시 서초구 효령로 17(방배동, 청진빌딩))

6. 시험규격/방법 (Test Standard/Method) : 의뢰자 제시 방법(기준)

7. 시험결과 (Test Results) : 추정불준

확 인 Affirmation	작성자(Tested by) 성 명(Name): 김장우 (Signature)	기술책임자(Technical Manager) 성 명(Name): 조한준 (Signature)

2022. 01. 11.

한국산업기술시험원장

FP104-05-00

ktl 한국산업기술시험원 Korea Testing Laboratory

성적서 번호 : 21-084030-01-1 Report No.
페이지 (5)/(총 16) Page of Pages

4. 시험환경

4.1 시험구성 (의뢰자 제공)

- "Edge Service 1.0" 는 "시험용 가상 서버3" 에 설치됨
- "Registry 1.0" 는 "시험용 가상 서버2" 에 설치됨
- "LifeCycle Manager 1.0" 와 "Monitoring 1.0" 는 "시험용 가상 서버8" 에 설치됨
- "Migration Administrator 1.0" 와 "Policy Engine 1.0" 는 "시험용 가상 서버7" 에 설치됨
- "Migration Client 1.0" 는 "시험용 가상 서버1" 와 "시험용 가상서버2" 에 설치됨
- "Checkpoint Collector 1.0" 와 "Custom Kubelet 1.0" 는 "시험용 가상 서버1 ~ 6" 에 설치됨
- "Kubernetes(K8S) 1.17.3" 는 "시험용 가상 서버 1 ~ 9" 에 설치됨
- "nGrinder 테시보즈 3.5.5" 는 "시험용 PC" 에 설치됨
※ 본 시험은 명시된 위 시험환경 내에서 수행되었으며, 시험 결과는 위 시험환경에 국한됨

FP281-09-11

ktl 한국산업기술시험원 Korea Testing Laboratory

성적서 번호 : 21-084030-01-1 Report No.
페이지 (14)/(총 16) Page of Pages

6. 시험 결과

6.1 시험항목 1

시험항목명	"라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간" 확인

시험 결과

- "라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간"
= 13:47:53.259 - 13:47:43:81603469 = 9.44296531 s
(의뢰자 제시 기준 : 14 s 이내)

※ 본 시험 결과는 의뢰자가 제공한 제품(시험 도구 및 시험데이터 포함)에서 출력된 결과를 기록함

6.2 시험항목 2

시험항목명	"Kubernetes(K8S) 1.17.3" 의 "파드" 수 확인

시험 결과

- 시험 진행 전 "파드" 수 : 1 개
- "network_receive" 값이 2Mbyte/sec 이상 3 회 연속 출력되었을 때 "파드" 수 : 2 개
- "network_receive" 값이 1Mbyte/sec 이하 3 회 연속 출력되었을 때 "파드" 수 : 1 개
- "cpu" 값이 50 %이상, "network_receive" 값이 2Mbyte/sec 이상 3 회 연속 출력되었을 때 "파드" 수 : 2 개
- "cpu" 값이 20 %이하, "network_receive" 값이 1Mbyte/sec 이하 3 회 연속 출력되었을 때 "파드" 수 : 1 개

※ 본 시험 결과는 의뢰자가 제공한 제품(시험 도구 및 시험데이터 포함)에서 출력된 결과를 기록함

[공인 인증 시험 성적서 - KTL]

- 성능 분석에 따른 최적화 및 고도화

  자체 테스트 결과 분석을 통해 개선 사항 도출 후 최적화 진행

  - 인증 API 권한 체크 및 API Access Control 처리 시간 단축
  - 엣지 서비스 인스턴스 기동 및 Registry 등록 시간 단축
  - 서비스 인스턴스 성능 정보 수집/분석 시간 단축
  - Self Healing 처리를 위한 엣지 서비스 인스턴스 상태 체크 최적화
  - Live Migration 처리 안정화
  - Logging을 통해 수집된 로그 분석 정확도 개선
  - 엣지 앱/서비스 등록을 위한 Descriptor 정보 보완 및 등록 시 사용자 편의성 보완

□ 상용 서비스에 대한 파일럿 검증

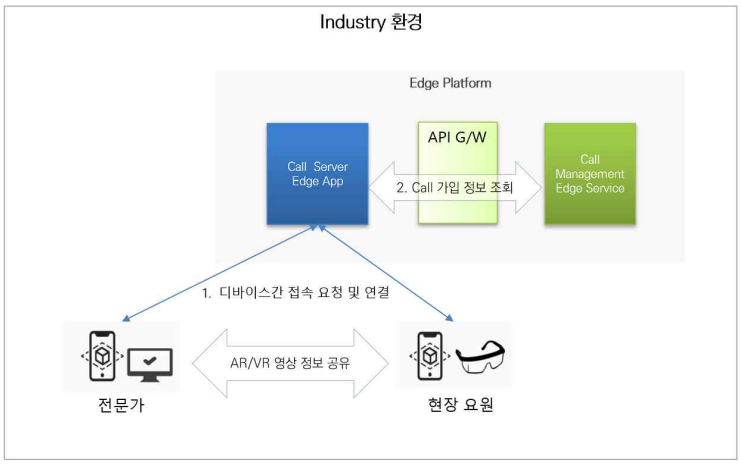
○ 주관 기관의 상용 서비스인 Smart Helper(AR/VR 기반의 원격 작업 지시 서비스)에 대한 파일럿 검증

- Smart Helper를 대상으로 파일럿 검증 진행

  - Smart Helper 개요

  Smart Helpler 서비스는 전문가와 실무자가 원격 AR/VR 화상 통화를 이용하여 직관적으로 소통함으로써 실시간으로 문제를 해결할 수 있도록 지원하는 협업 서비스

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[Smart Helper 개념도]

- Smart Helper 서버 App를 MSA 구조로 변경 작업 진행
- 클라우드 엣지 플랫폼 환경하에 Smart Helper 서비스 구축
- Smart Helper의 기본 서비스에 대한 검증
- Smart Helper의 전체 서비스에 대한 통합 검증

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

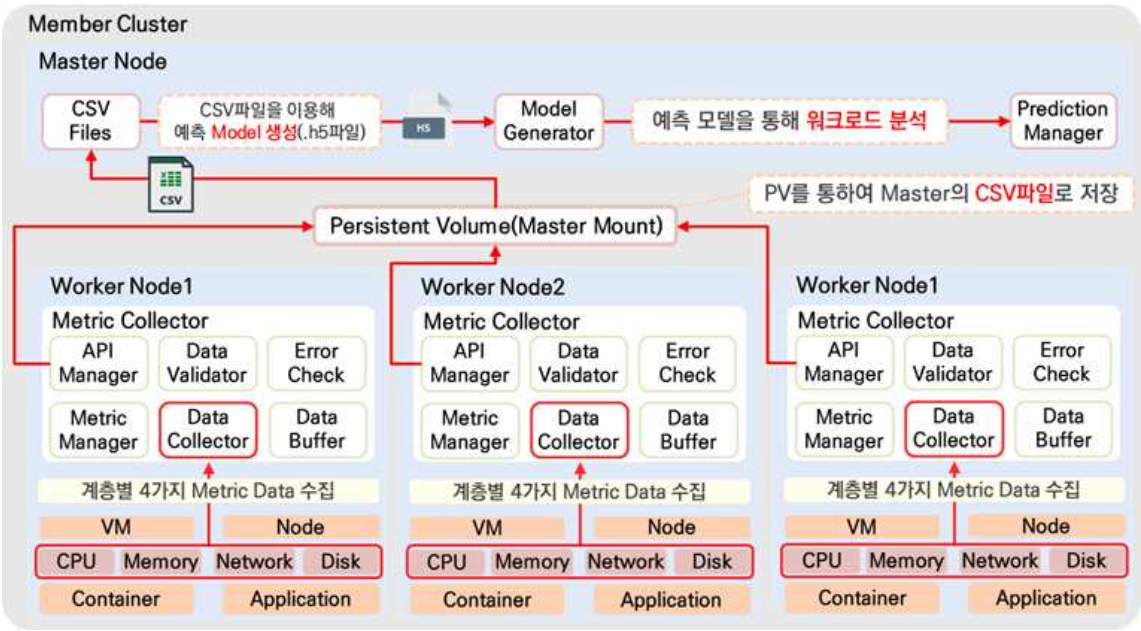
▣ 참여기관 1(KETI) :

- Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 최적화 및 고도화
- 클라우드 엣지 컨테이너기반 IoT 응용 서비스 연동 최적화

▢ Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 최적화 및 고도화

○ 마이크로서비스 기반 Zone In/Out 여부 판단 기술 최적화 및 고도화

- 클라우드 엣지 자원 계층별(Node, VM, Container, Application) 다차원 메트릭 기반 워크로드 분석 기술 최적화 및 안정화



[계층 별 메트릭 수집 및 워크로드 분석]

- 각 노드의 Metric 수집 및 저장을 담당하는 Metric Collector 모듈을 이용한 클라우드 엣지 기반 다차원 메트릭(CPU Usage, Memory Usage, Network I/O, Disk I/O) 수집

```
metricSave := ns.Writer.Write([]string{
    "Time",
    "NodeIP",    // Node IP
    "NodeName", // Node Name
    "CPU_utilization",
    "MEM_utilization",
    "NetworkTotalBytes",
    "DiskIOMerge",
    "DiskIOWaitTime",
    "DiskUsage",
})
ns.Writer.Flush()
```

[다차원 메트릭 수집]

- 워크로드 분석 최적화를 위해 Node, VM, Container, Application 단위로 세분화

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

하여 다차원 메트릭 수집완료

```
type NodeResponse struct {
    Time              string  `json:"Time"`
    Type              string  `json:"Type"`
    ClusterName       string  `json:"clusterName"`
    Name              string  `json:"Name"`
    CPU_utilization   float32 `json:"CPU_utilization"`
    MEM_utilization   float32 `json:"MEM_utilization"`
    NetworkTotalBytes int     `json:"NetworkTotalBytes"`
    DiskUsage         float32 `json:"DiskUsage"`
}
```

[Node 계층 메트릭]

```
type VMResponse struct {
    Time              string  `json:"Time"`
    Type              string  `json:"Type"`
    ClusterName       string  `json:"clusterName"`
    Name              string  `json:"Name"`
    CPU_utilization   float32 `json:"CPU_utilization"`
    MEM_utilization   float32 `json:"MEM_utilization"`
    NetworkTotalBytes int     `json:"NetworkTotalBytes"`
    DiskUsage         float32 `json:"DiskUsage"`
}
```

[VM 계층 메트릭]

```
type ContainerResponse struct {
    Time              string  `json:"Time"`
    Type              string  `json:"Type"`
    ClusterName       string  `json:"clusterName"`
    NodeName          string  `json:"NodeName"`
    Name              string  `json:"Name"`
    CPU_utilization   float32 `json:"CPU_utilization"`
    MEM_utilization   float32 `json:"MEM_utilization"`
    NetworkTotalBytes int     `json:"NetworkTotalBytes"`
    DiskUsage         float32 `json:"DiskUsage"`
    Is_activated      bool    `json:"is_activated"`
}
```

[Container 계층 메트릭]

```
type ApplicationResponse struct {
    Time              string  `json:"Time"`
    Type              string  `json:"Type"`
    PodName           string  `json:"PodName"`
    Name              string  `json:"Name"`
    CPU_utilization   float32 `json:"CPU_utilization"`
    MEM_utilization   float32 `json:"MEM_utilization"`
    NetworkTotalBytes int     `json:"NetworkTotalBytes"`
    DiskUsage         float32 `json:"DiskUsage"`
    Cpu_transition    string  `json:"cpu_transition"`
    Memory_transition string  `json:"memory_transition"`
    Network_transition string `json:"network_transition"`
    Disk_transition   string  `json:"disk_transition"`
}
```

[Application 계층 메트릭]

- 수집된 계층별 메트릭 데이터의 신뢰성 향상을 위해 수집 주기를 3s로 조정완료
- 계층별 메트릭 수집 시 병렬 스레드 처리하여 과부하 개선 완료

```
root@k8s-cluster3-master:/keti/csv# tree
.
├── k8s-cluster1-worker1
│   ├── checkpoint-collector-44jjp.csv
│   ├── deployment-pilot-registry-cfb6946f9-fjllk.csv
│   ├── keti-daemonset-pod-nttms.csv
│   ├── kube-flannel-ds-p6sk7.csv
│   ├── kube-proxy-9wp8x.csv
│   ├── kubernetes-client-deploy-77fbf865bd-vdmd6.csv
│   ├── kubernetes-dashboard-84965dbbdd-h4t9w.csv
│   └── metric-collector-mjtk6.csv
├── k8s-cluster1-worker1.csv
├── k8s-cluster1-worker2
│   ├── checkpoint-collector-lftk5.csv
│   ├── dashboard-metrics-scraper-c79c65bb7-qch76.csv
│   ├── deployment-ea16dc466c4a1-799d874c56-mx22b.csv
│   ├── keti-daemonset-pod-fnm9p.csv
│   ├── kube-flannel-ds-m7l7t.csv
│   ├── kube-proxy-w56n5.csv
│   └── metric-collector-ttxnc.csv
├── k8s-cluster1-worker2.csv
├── k8s-cluster2-worker1
│   ├── checkpoint-collector-6n9gp.csv
│   ├── dashboard-metrics-scraper-c79c65bb7-mm6xc.csv
│   ├── keti-daemonset-pod-qxltt.csv
│   ├── kube-flannel-ds-2wlcc.csv
│   ├── kube-proxy-l7n6v.csv
│   └── metric-collector-l7bz9.csv
├── k8s-cluster2-worker1.csv
├── k8s-cluster2-worker2
│   ├── checkpoint-collector-lr6bt.csv
│   ├── keti-daemonset-pod-nkvhp.csv
│   ├── kube-flannel-ds-z8tzw.csv
│   ├── kube-proxy-4hhs5.csv
│   ├── kubernetes-dashboard-84965dbbdd-vcrlv.csv
│   └── metric-collector-8tg4d.csv
```

[CSV파일로 저장된 메트릭 데이터]

- 수집한 메트릭 데이터를 Persistant Volume을 통해 CSV 파일로 저장

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

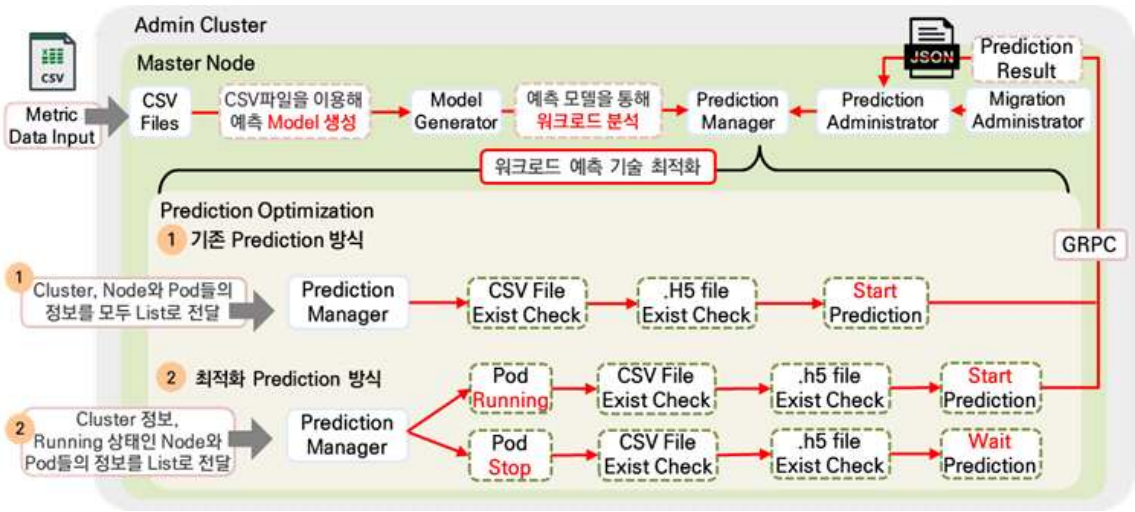
- CSV 파일 기반의 메트릭 데이터를 통해 예측 Model 생성
- 생성된 예측 Model(.h5)을 통해 Prediction Manager에서 Node와 Pod의 과부하를 예측

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: metric-collector
  namespace: edge-service
spec:
  selector:
    matchLabels:
      name: metric-collector
  updateStrategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        name: metric-collector
        namespace: edge-service
    spec:
      serviceAccountName: metric-collector
      containers:
        - name: metric-collector
```

[Metric Collector의 yaml]

- 다중 클러스터 내부 모든 Node 내부 메트릭 수집을 위해 Metric Collector를 DaemonSet 형식으로 배포

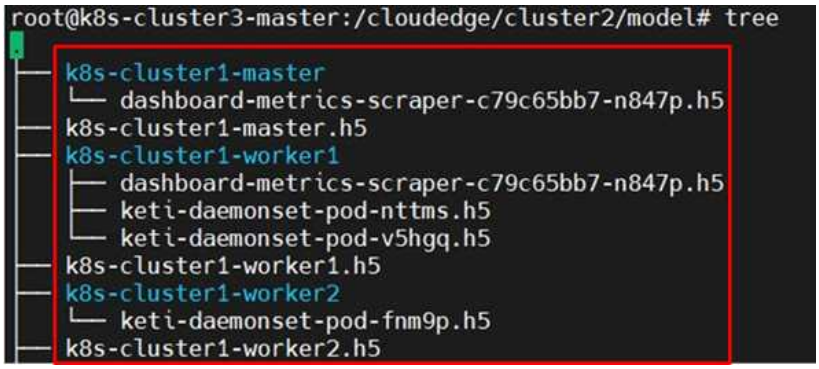
• 클라우드 엣지 자원 계층별 워크로드 예측 기술 최적화 및 안정화



[계층별 워크로드 예측 기술 최적화 과정]

- 기존 예측 방식은 Node와 Pod의 상태를 고려하지 않고 예측을 진행 하였으나 현재 예측 방식은 Node와 Pod의 Running 상태를 확인 후 예측 진행하여 최적화 완료
- Prediction Manager의 계층별 예측 결과를 JSON 형식으로 변환하여 gRPC통신을 통해 Prediction Administrator로 전송
- Prediction Manager로부터 받은 해당 Pod 및 Node의 예측결과를 토대로 Prediction Administrator모듈 내 Analyzer에서 분석 후 Migration List를 생성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[생성된 Node 및 Pod의 h5 파일]

- Node 및 Pod 상태 모니터링 기반으로 생성된 모델 파일인 h5 파일로 예측 진행
- Admin Cluster의 Persistant Volume을 통해 h5 파일 형식의 모델 통합관리



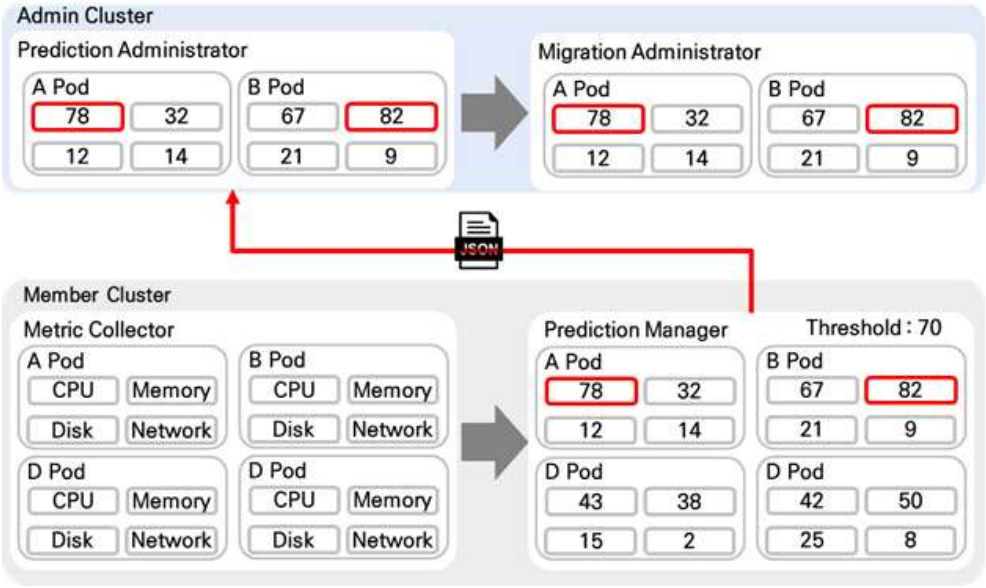
[워크로드 예측 최적화를 위한 모듈]

- 워크로드 예측 최적화를 위해 Prediction Administrator와 Prediction Manager를 Pod화하여 발생되는 데이터를 gRPC 통신을 통해 전송

• 엣지 서비스 워크로드에 따른 Live Migration Decision 기술 최적화 및 안정화

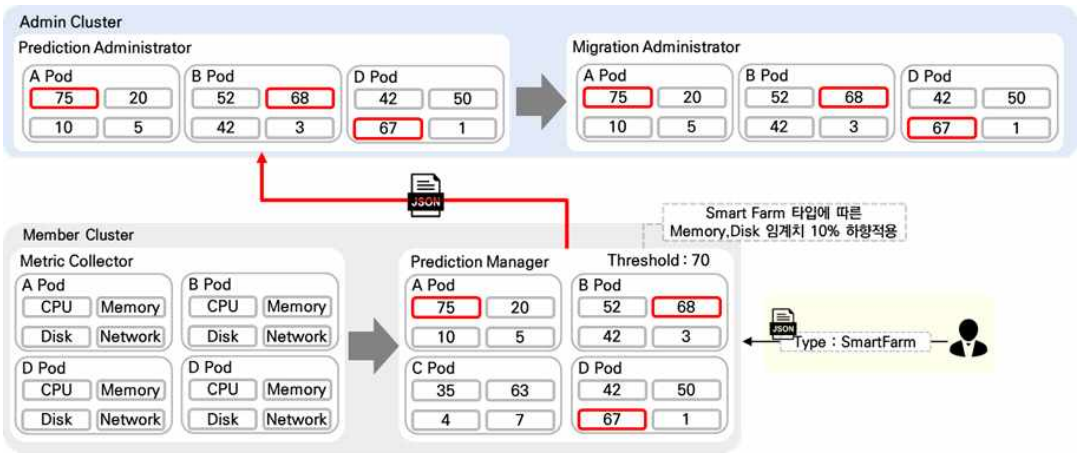
- Prediction 관련 모듈(Prediction Administrator, Prediction Manager)간 내부 gRPC 통신으로 Prediction 모듈 간 발생한 데이터의 전송 지연 시간 최소화 및 안정화
- Prediction Administrator의 모듈 내부에서의 예측 및 분석을 통해 리소스 과부하 대상 결정 및 목적지 선정
- 리소스 과부하 및 위험 대상으로 선정된 Pod의 리스트 화를 통한 리소스 과부하 대상 통합 관리

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



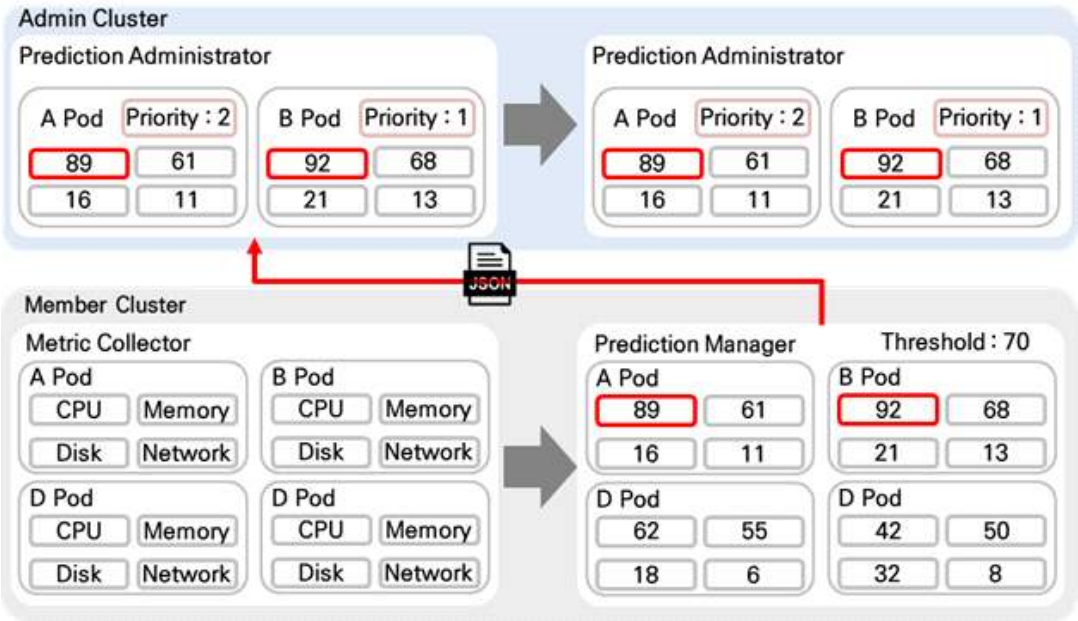
[자원 사용률 기반 마이그레이션 대상 선정]

- 배포된 서비스 중 자원 사용률이 높은 Migration 대상 및 목적지 선정 로직 적용



[환경 및 서비스 타입별 자원 사용률이 높은 마이그레이션 대상 선정]

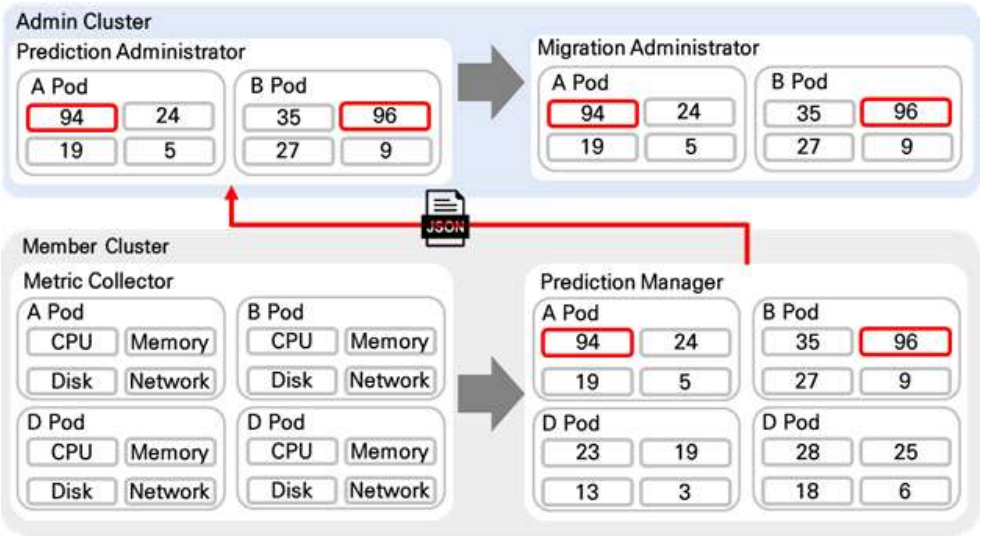
- 환경 및 서비스 특성을 고려한 Migration 대상 및 목적지 선정 로직 적용



[마이그레이션 대상 선정의 신뢰성 향상을 위한 우선순위 모델 적용]

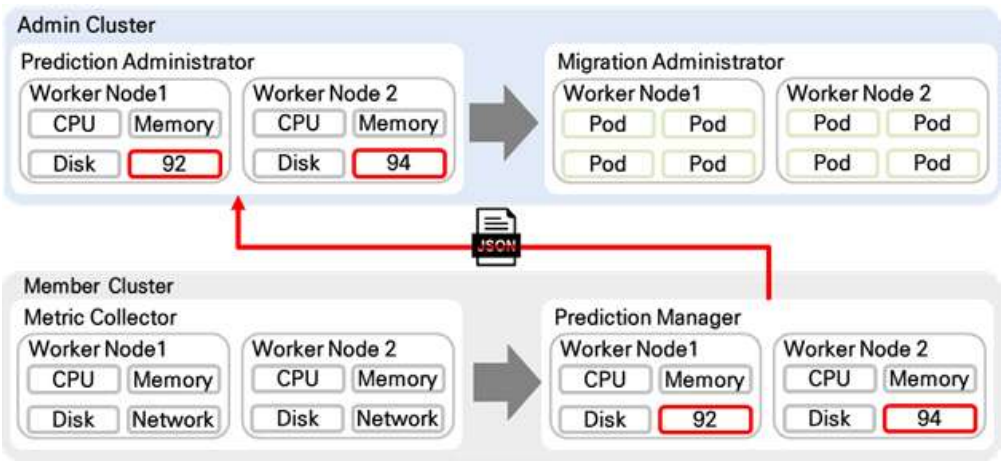
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- Migration 대상 선정 로직의 신뢰성 및 정확도 향상을 위한 우선순위 모델 적용



[특정 자원 사용률 기반 마이그레이션 대상 선정]

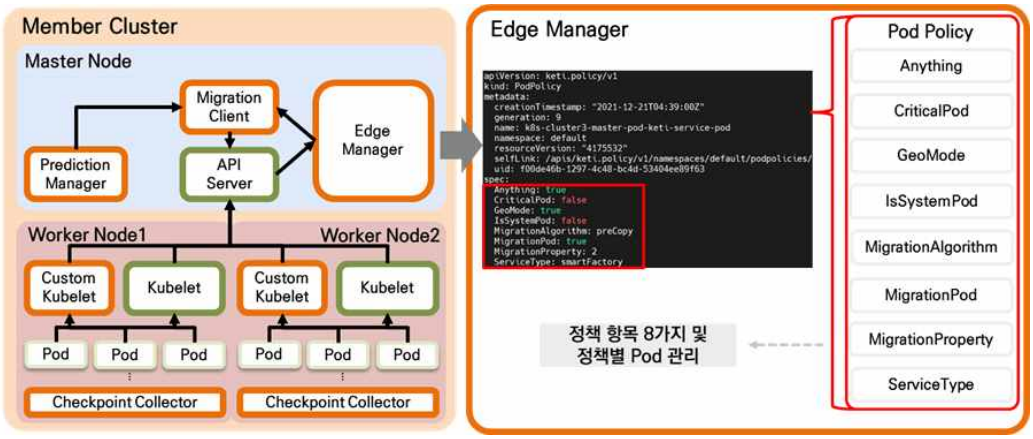
- 특정 자원 사용률이 높은 대상 선정 로직 적용



[네트워크 응답 속도 기반 마이그레이션 대상 선정]

- 실시간으로 네트워크 응답 속도를 체크하여 Migration 대상 및 목적지 선정 로직 적용

• 클라우드 엣지 간 라이브 마이그레이션 정책 코어 최적화 및 안정화



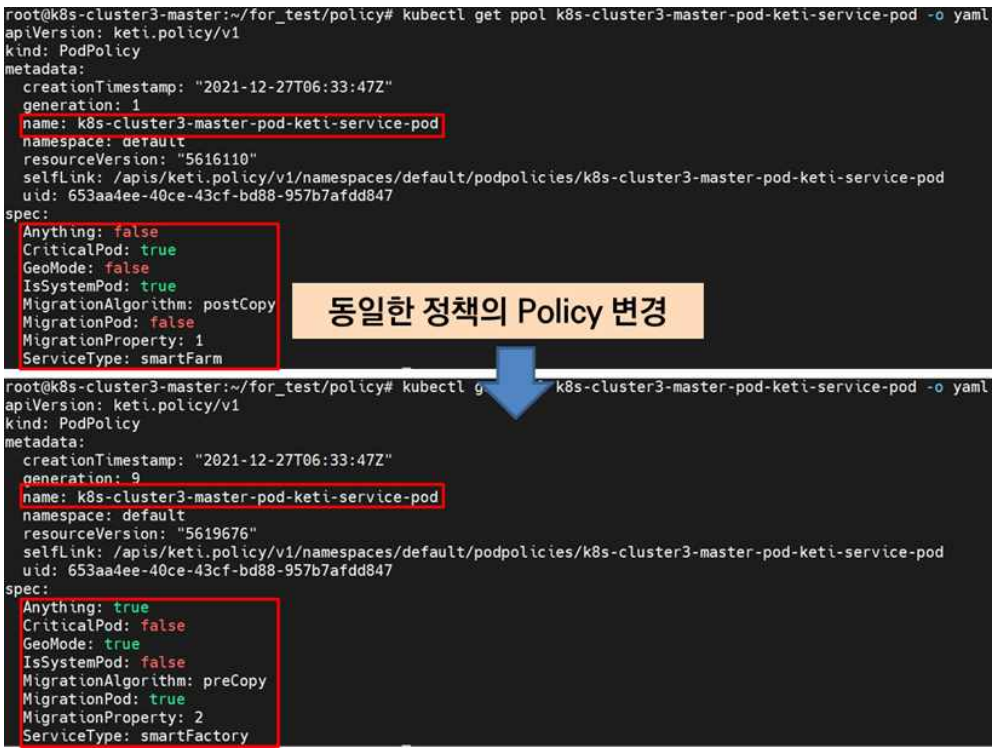
[라이브 마이그레이션 정책 코어]

- 통합 연동 및 사용자 편의를 위한 외부에서의 정책 설정용 REST API 개발



[REST API 중 GET 요청]

- 관리자의 편의성을 위해 개발된 REST API 중 GET 요청을 통해 클러스터 내부 운용중인 Pod 정책 조회가능



[REST API 중 POST 요청]

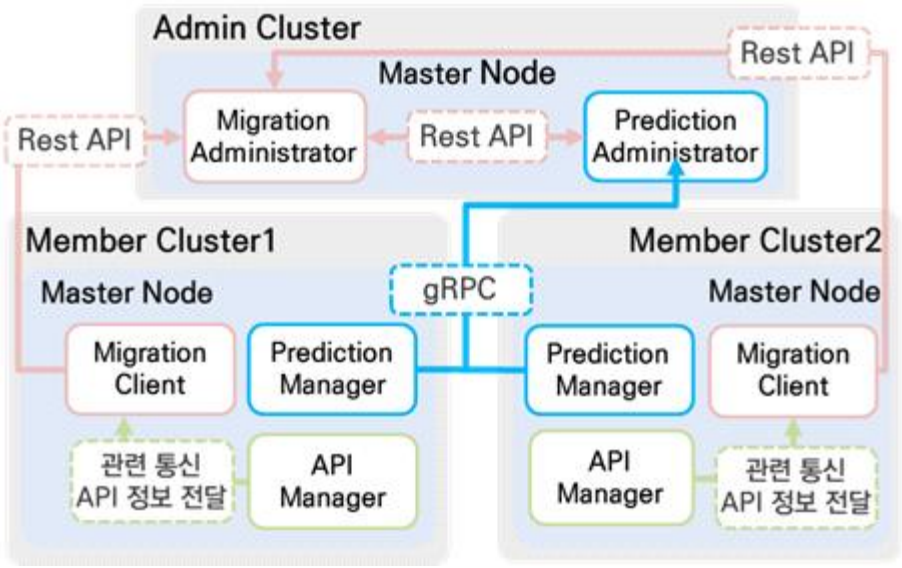
- 관리자의 편의성을 위해 개발된 REST API 중 POST 요청을 통해 운용중인 Pod의 정책 설정 및 변경 가능

- 클라우드 엣지 기반 플랫폼 운용 시 라이브 마이그레이션 정책 항목 수 8개(Anything, CriticalPod, GeoMode, IsSystemPod, MigrationAlgorithm, MigrationPod, MigrationProperty, ServiceType)로 확장 완료

○ 마이크로서비스 기반 Zone 이탈 여부 정보 전달 기술 최적화 및 고도화

• 라이브 서비스 마이그레이션을 위한 이전 목적지 자원 요청(프로비저닝) 지연 최적화

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[다중 클러스터 기반 클러스터 간 통신 구조]

- 관리자 클러스터의 내부 모듈인 Migration Administrator와 Prediction Administrator의 통신 지연 최적화를 위한 REST API 적용 완료

```go
func NewServerStart() {
    klog.Infoln("Call GRPCServerStart Func")
    l, err := net.Listen("tcp", ":30561")
    if err != nil {
        klog.Errorln(err)
    }
    s := grpc.NewServer()
    mtom.RegisterMtoMClientServer(s, &Server{})
    if err := s.Serve(l); err != nil {
        klog.Errorln(err)
    }
}
```

[Migration 모듈 간 gRPC 통신]

- Migration Administrator와 Migration Client의 내부 통신 지연 최적화를 위해 gRPC 적용 완료

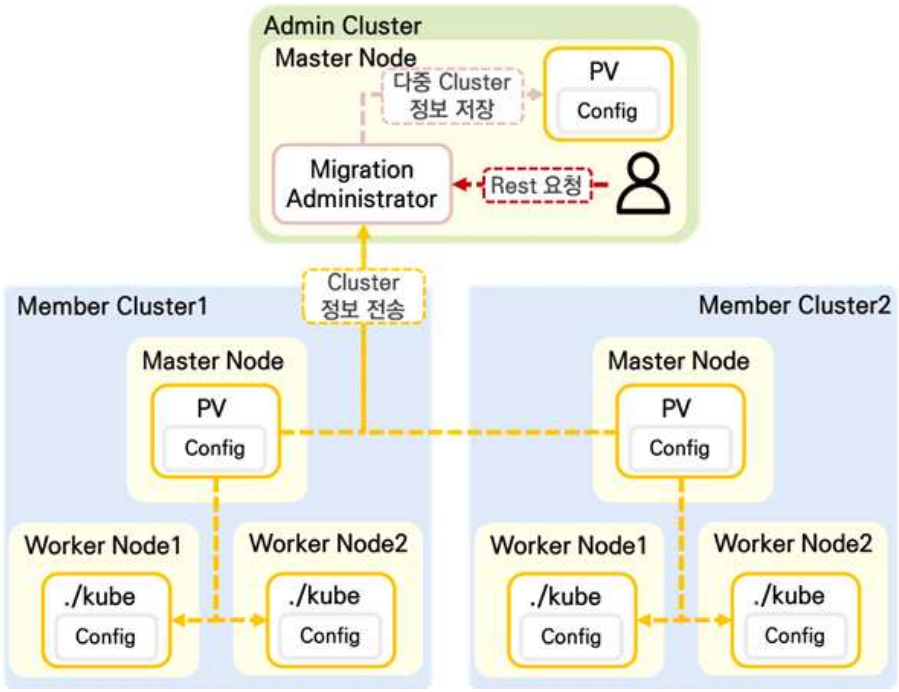
```python
def serve(ipAddr):
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=config.info['max_request']))
    generate_pb2_grpc.add_GreeterServicer_to_server(Greeter(), server)
    server.add_insecure_port(ipAddr)
    server.start()

    return server
```

[Prediction 모듈 간 gRPC 통신]

- Prediction 관련 모듈인 Prediction Administrator와 Prediction Manager의 통신 지연 최적화를 위한 gRPC 적용

• 실시간 마이그레이션 인증/인가/접근제어 지연 최적화



[다중 Cluster의 Config 통합 관리]

```
func (JoinResource) Post(rw http.ResponseWriter, r *http.Request, ps httprouter.Params) Response {
    body, err := ioutil.ReadAll(r.Body)
    if err != nil {
        klog.Errorln(err)
    }
    req := scp.SetJoinRequst(body)
    path := req.GetConfig()
    ccs := client.GetClusterClientSet()
    klog.Infoln(path)
    ccs.SetConfig(req.ClusterName, path)
    ccs.MakeClient(req.ClusterName)
    return Response{200, "", nil}
}
```

[REST API를 사용한 Member Cluster들의 Config 전송]

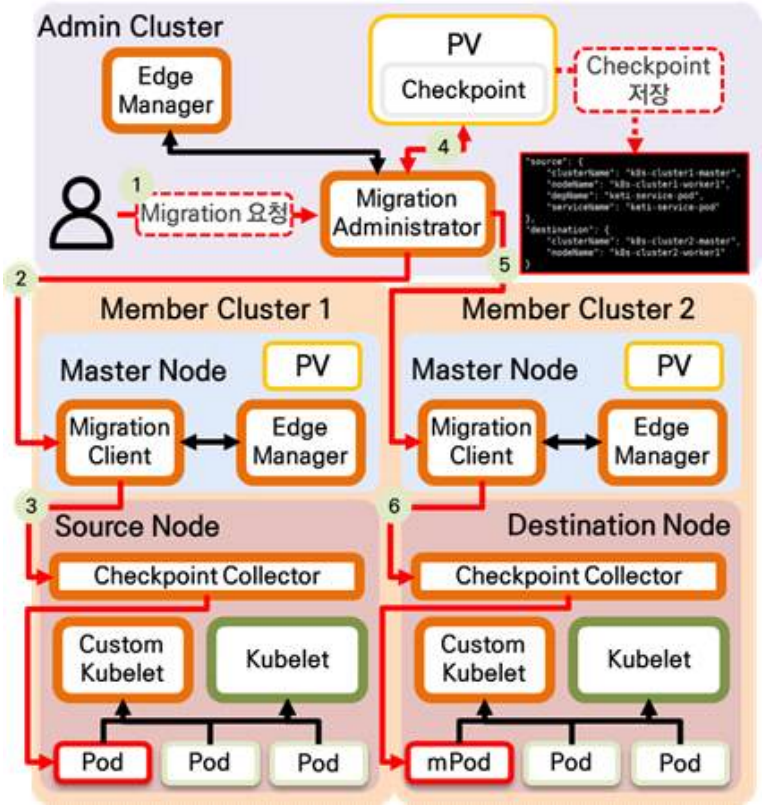
- 통합연동 및 사용자의 인증/인가 편의성을 위한 REST API로의 확장
- REST API를 통해 관리자가 추가로 운용중인 Cluster의 인증 및 인가 가능



[Config 전송을 통한 인증과정]

- 클러스터 간 마이그레이션 요청을 보내기 위한 클러스터 인증
- 해당 사용자의 접근 제어에 관한 지연 감소
- Migration Administrator의 Config 통합관리를 통한 Cluster 간 마이그레이션 최적화 완료

• 클라우드 엣지 Controller 기반 엣지 서비스 마이그레이션 Push(Alert) 기술 연동 최적화



[Migration Push 기술 연동]

```go
func (MigrationResource) Post(rw http.ResponseWriter, r *http.Request, ps httprouter.Params) Response {
    st := time.Now().Format(time.ANSIC)
    mres := &MigrationResponse{}
    mres.MigrationStartTime = st
    mres.MigrationSource = make([]string, 0)
    body, err := ioutil.ReadAll(r.Body)
    if err != nil {
        klog.Errorln(err)
    }
    req := SetRequest(body)
    scn := req.Source.ClusterName + "/" + req.Source.NodeName
    fcn := req.Target.ClusterName + "/" + req.Target.NodeName

    mres.MigrationSource = setSource(req.Source.ClusterName, req.Source.DepName)

    migrationRequest(*req, mres.MigrationSource, req.Source.DepName)

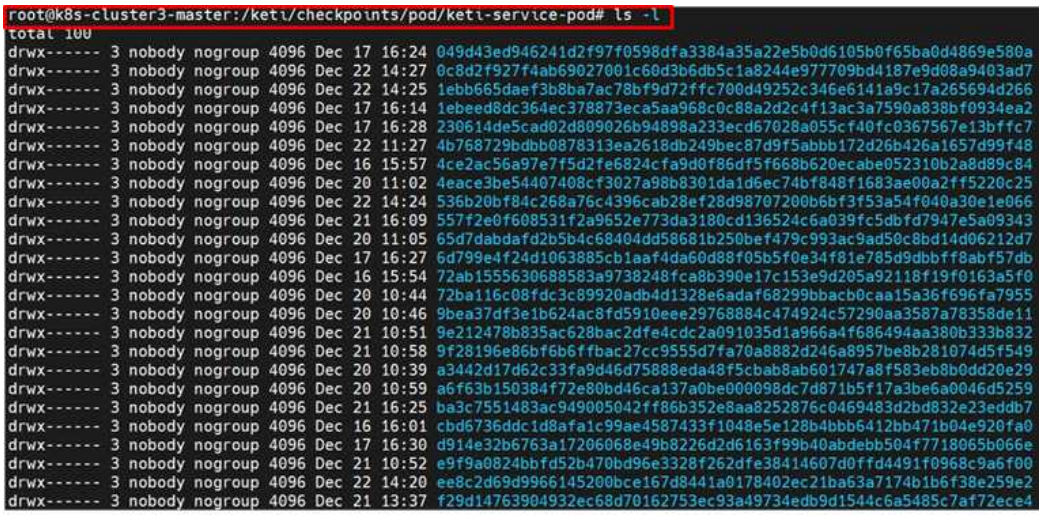
    time.Sleep(time.Second * 5)

    ft := time.Now().Format(time.ANSIC)
    mres.MigrationStartCN = scn
    mres.MigrationFinishCN = fcn
    mres.MigrationFinishTime = ft

    return Response{200, "", mres}
}
```

[REST API를 통한 Migration]

- 통합 연동 및 인증 편의를 위한 REST API 지원
- 사용자의 Migration 요청이 오면 Migration Administrator는 데이터들을 분석 후 Migration Client에 Migration Target Pod와 Migration Destination Point를 리스트화 하여 전송 후 Migration Client는 CheckPoint를 기반으로 마이그레이션 수행
- 마이그레이션 수행 시 목적지 Node의 리소스 분석 후 노드 및 클러스터 마이그레이션 또는 노드 마이그레이션 선정

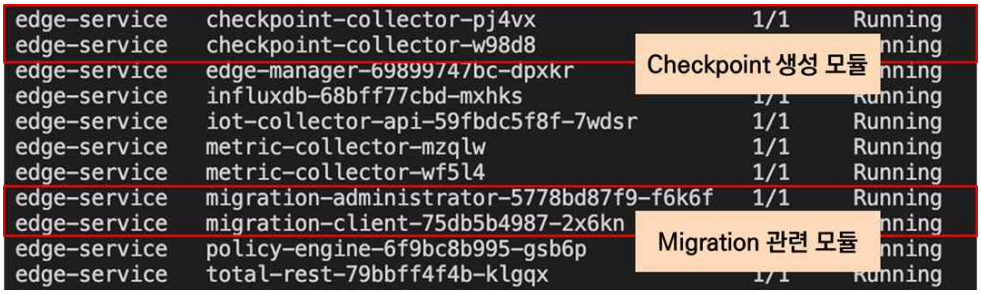


[Migration이 완료된 Pod의 CheckPoint 통합 관리]

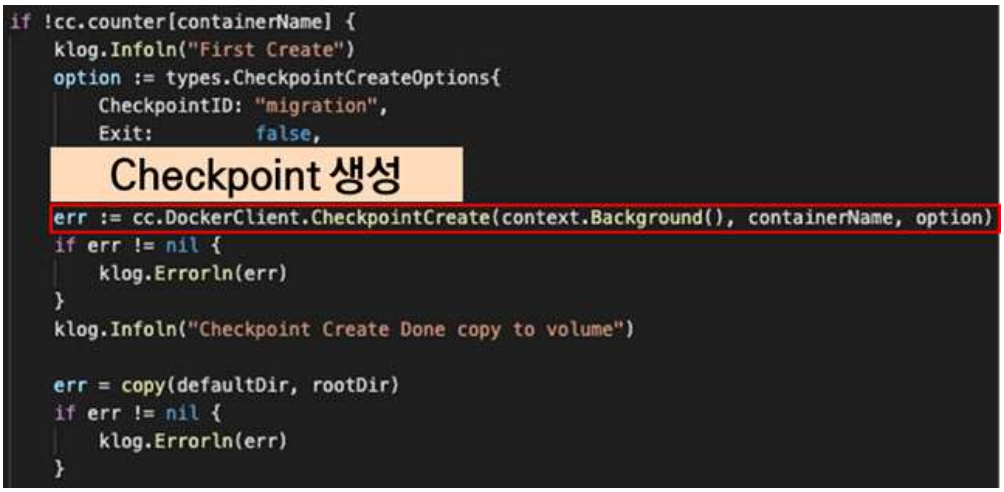
- Admin Cluster의 Persistant Volume을 통한 Migration이 완료된 Checkpoint 통합 관리
- Checkpoint 생성 및 관리를 위한 Checkpoint Collector 모듈 최적화 완료
- Migration 요청 시 해당 클러스터의 실질적 Migration 수행을 위한 Migration Client 모듈 최적화 완료
- Migration이 완료된 Pod의 Image를 Docker Hub로 관리

○ Zone 이동 시, 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발 및 안정화

• 엣지 서비스 자원 사용 특성(Memory 중심) 분석 기술 개발 및 배포 모듈 개발



[서비스 자원 사용 특성 적용 배포 모듈 최적화]

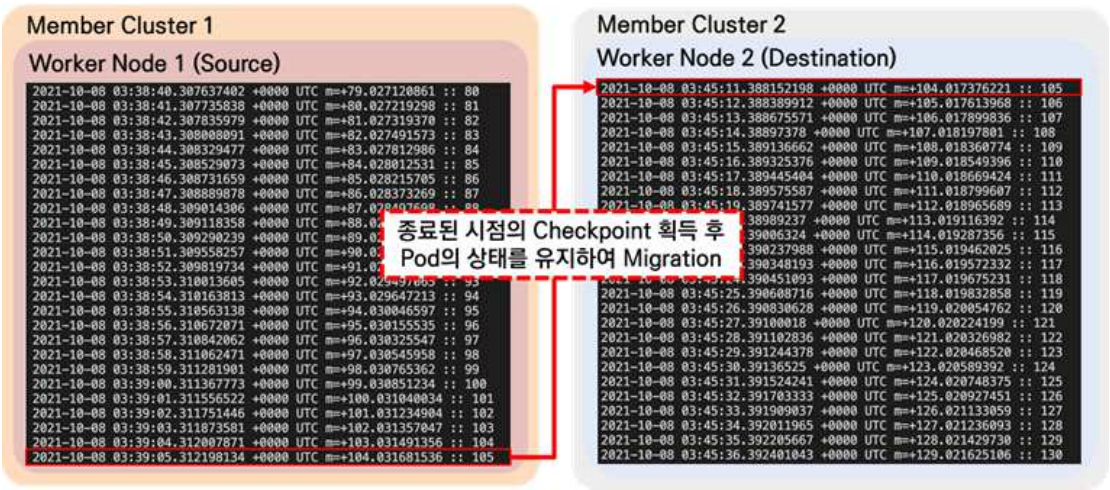


[Memory 중심 마이그레이션 기능 중 Checkpoint 생성 최적화]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- Checkpoint Collector 모듈을 DaemonSet 타입으로 배포하여 Admin Cluster에 소속되어 있는 모든 Member Cluster들의 Checkpoint 생성 및 관리 최적화 완료
- Migration 관련 모듈(Migration Administrator, Migration Client)을 Deployment로 배포하여 Migration 관련 기능 수행 최적화 완료
- Checkpoint Collector와 Migration 관련 모듈의 연동으로 서비스 자원 사용 특성을 기반으로 한 마이그레이션 수행 관련 항목 최적화 완료
- 서비스 자원 사용 특성 중심 마이그레이션 항목 최적화 완료

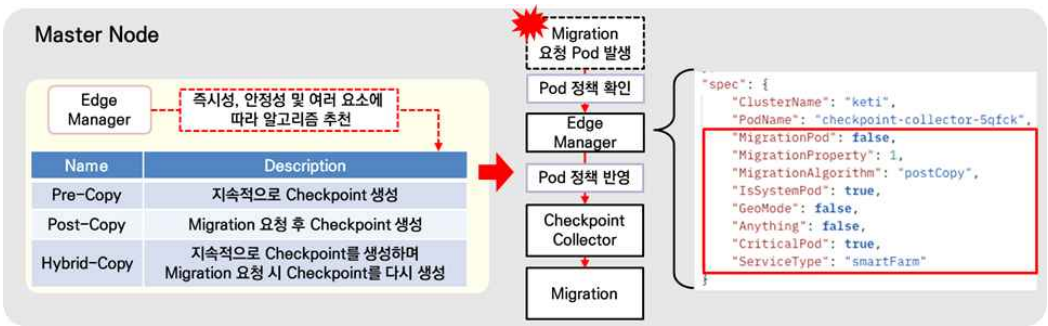
• 플랫폼 간 State-Full & Stateless 서비스 라이브 마이그레이션 기술 개발 및 배포 모듈 개발



[Checkpoint 수집 후 Migration 수행]

- 클라우드 엣지 기반 마이그레이션 기술 최적화 완료
- State-Full 및 Stateless 기반 마이그레이션 Freeze Time 최소화 완료
- Edge Manager 모듈 내부 마이그레이션 정책 조회를 통한 마이그레이션 분류 기능 최적화 완료
- 마이그레이션 Decision API 개수 증가를 통한 확장성 고려
- 자체 Custom Kubelet의 CRU를 통한 Migration Pod 생성 및 관리

• 엣지 서비스 자원 특성에 따른 라이브 마이그레이션 알고리즘 추천 기술 개발 및 배포 모듈 개발



[마이그레이션 알고리즘 추천 기술 반영]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- Edge Manager를 통해 관리하는 마이그레이션 정책 기반 마이그레이션 알고리즘 선정
- 즉시성, 안정성을 위한 마이그레이션 알고리즘 추천기술 최적화 완료
- Checkpoint를 사용한 마이그레이션 알고리즘 기술 개발
- 알고리즘 사용을 위한 배포 모듈 개발

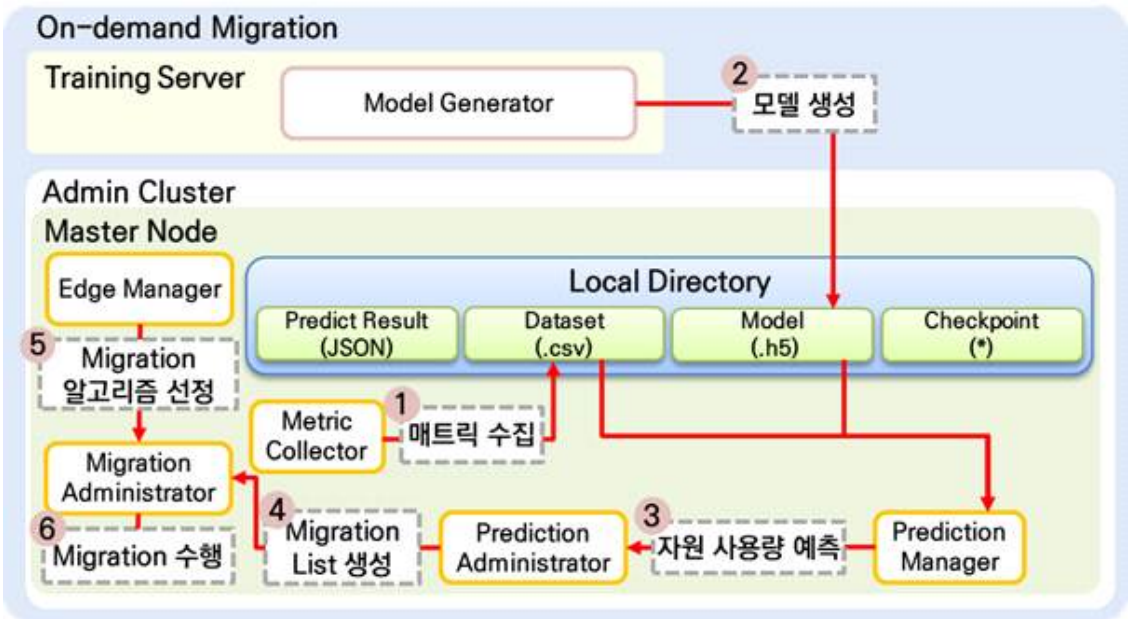
• 엣지 서비스 자원 특성에 따른 플랫폼 간 라이브 마이그레이션 기술 최적화 및 배포 모듈 개발

```
root@k8s-cluster3-master:~/for_test/policy# kubectl get ppol k8s-cluster3-master-pod-keti-service-pod -o yaml
apiVersion: keti.policy/v1
kind: PodPolicy
metadata:
  creationTimestamp: "2021-12-27T06:33:47Z"
  generation: 1
  name: k8s-cluster3-master-pod-keti-service-pod
  namespace: default
  resourceVersion: "5616110"
정책 내부 알고리즘 적용   paces/default/podpolicies/k8s-cluster3-master-pod-keti-service-pod
                          dd847
spec:
  Anything: false
  CriticalPod: true
  GeoMode: false
  IsSystemPod: true
  MigrationAlgorithm: postCopy
  MigrationPod: false
  MigrationProperty: 1
  ServiceType: smartFarm
```

[Pod의 마이그레이션 알고리즘 정책 적용]

- Copy 알고리즘의 비동기적 처리를 통한 서비스 지연 시간 최소화
- Pod의 마이그레이션 정책 적용을 통한 Copy 알고리즘 선정
- 엣지 서비스 자원 특성에 따른 정책 반영 및 마이그레이션 적용으로 라이브 마이그레이션 기술 최적화
- 관리자가 운용중인 환경에 따라 Migration 알고리즘을 변경 가능

• On-demand 마이그레이션 기술 최적화 및 플랫폼 연동 안정화



[On-demand 마이그레이션 과정]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

- Metric Collector에서 수집한 메트릭 데이터 기반 On-demand 마이그레이션 최적화 완료
- Admin Cluster의 Master Node의 Persistant Volume을 통해 관리 대상인 각 클러스터의 Metric 데이터 및 Model 데이터와 Migration Pod의 Check-Point 데이터 통합관리

Pod 과부하 예측 리스트


```
"type": "Smart Factory",
"migration_Order": 2,
"nodeName": "k8s-cluster2-worker1",
"CPU_utilization": 40.029,
"MEM_utilization": 0.52,
"Overhead_Metric": "CPU Overhead",
"podName": "keti-daemonset-pod-qxltt",
"destination": [
    {
        "clusterName": "k8s-cluster2-master",
        "nodeName": "k8s-cluster2-worker2"
    }
]
```


[Prediction Administrator에서 전송한 과부하 예측 리스트]

- 수집된 데이터를 기반으로 Prediction 관련 모듈에서 예측 진행 후 결과를 Migration Administrator 모듈로 전송
- 해당 리스트를 기반으로 Migration관련 모듈에서 Migration 수행

• 주관 기관의 상용 서비스인 Smart Helper 서비스 시범 환경에서의 통합 연동

- 주관 기관 협력 하에 실증 및 연동(Smart Helper) 모듈화 지원 완료

통합 연동을 위한 API


```
edge-service   coaptest-server-6b78c89b8d-gl8bw          1/1   Running
edge-service   iot-collector-58d5699d9d-hzd9j            1/1   Running
edge-service   iot-collector-api-6dbb5ff9b8-xnc86        1/1   Running
                                                        1/1   Running
                                                        1/1   Running
                                        r-66b595d99d-plrhp  1/1   Running
edge-service   total-rest-6d9577d99d-v987w               1/1   Running
```


[통합 연동을 위한 API]

- 관련 API 통신을 위한 모듈 개발 완료

▢ 클라우드 엣지 컨테이너 기반 IoT 응용 서비스 연동 최적화

○ 대규모 IoT 디바이스 데이터 수집 서비스 응용 클러스터 개발 및 플랫폼 연동 최적화

• 클라우드 엣지 플랫폼 기반 MQTT 프로토콜 지원 IoT 데이터 Collector 최적화

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

```
func HandleSubscribe(id string, topic string) {
    var broker = "10.0.6.37"
    var port = 31883
    MQTT mqtt.NewClientOptions()
    opts.AddBroker(fmt.Sprintf("tcp://%s:%d", broker, port))
    opts.SetClientID(id)
    opts.SetUsername(id)
    opts.SetPassword("public")
    opts.SetAutoReconnect(true)
```

[MQTT 프로토콜 적용]

```
edge-manager-7f77964688-4682d          1/1     Running   0          22d
influxdb-68bff77cbd-7zf7v              1/1     Running   4          106d
keti-service-pod                       1/1     Running   0          24h
mosquitto-broker-6fd48d4b6f-9vgjg      1/1     Running   3          106d
web-service-7f8794bf86-vm7xq           1/1     Running   4          118d
checkpoint-collector-2c6vs             1/1     Running   0          9d
checkpoint-collector-kv7sc             1/1     Running   0          9d
checkpoint-collector-qt2tz             1/1     Running   0          9d
```

[Mosquitto broker 사용]

- IoT Collector 모듈 내부에서 MQTT 프로토콜을 사용한 데이터 수집
- Mosquitto broker를 통한 MQTT 운영에서의 안정성 향상
- IoT Collector 모듈 내부 비동기적 처리를 통한 MQTT 프로토콜 적용 완료

• 클라우드 엣지 플랫폼 기반 CoAP 프로토콜 지원 IoT 데이터 Collector 최적화

```
func CoapServer() {
    r := mux.NewRouter()
    r.Use(loggingMiddleware)
    r.Handle("/device", mux.HandlerFunc(handleDevice))
    r.Handle("/sensor", mux.HandlerFunc(handleSensor))
    CoAP
    coap.ListenAndServe("udp", ":5686", r)
}
```

[CoAP 프로토콜 적용]

- IoT Collector 모듈 내부에서 CoAP 프로토콜을 사용하여 데이터 수집
- IoT Collector 모듈 내부 비동기적 처리를 통한 CoAP 프로토콜 적용 완료

• 클라우드 엣지 플랫폼 기반 HTTP 프로토콜 지원 IoT 데이터 Collector 최적화

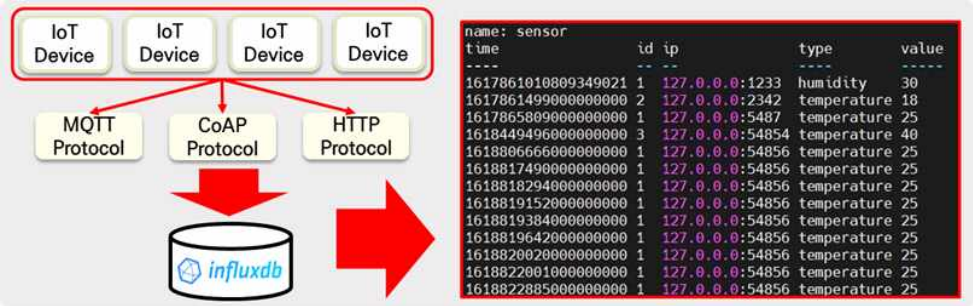
```
func HandleRequests() {
    fmt.Println("http Server Start")
    router := mux.NewRouter().StrictSlash(true)

    router.HandleFunc("/", homePage).Methods("GET")
    router.HandleFunc("/create-device", createDevice).Methods("POST")
    router.HandleFunc("/create-sensor", createSensor).Methods("POST")
    router.HandleFunc("/read", getArticles).Methods("GET")
    router.HandleFunc("/read/{id}", getArticle).Methods("GET")
    router.HandleFunc("/update", updateArticle).Methods("PUT")
    router.HandleFunc("/delete/{id}", deleteArticle).Methods("DELETE")
    HTTP
    log.Fatal(http.ListenAndServe(":10000", router))
}
```

[HTTP 프로토콜 적용]

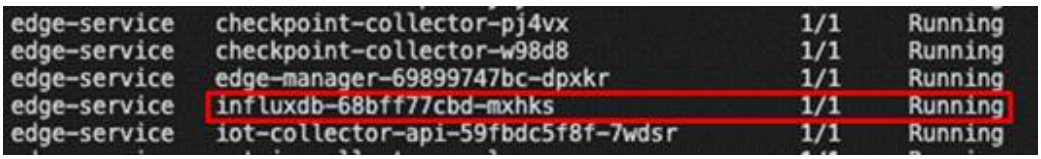
- IoT Collector 모듈 내부에서 HTTP 프로토콜을 사용하여 IoT Device의 데이터 수집 가능
- IoT Collector 모듈 내부 비동기적 처리를 통한 HTTP 프로토콜 적용 완료

• 다중 IoT 데이터 통합을 위한 Data Hub 기술 최적화 및 플랫폼 연동 안정화



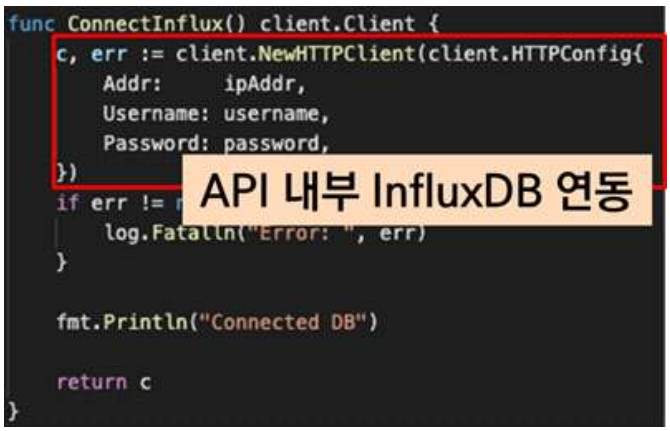
[통신 프로토콜을 사용한 IoT 디바이스의 데이터 저장]

- MQTT, CoAP, HTTP 통신 프로토콜을 사용한 IoT 디바이스들의 데이터 수집으로 통합 최적화 완료



[IoT 디바이스 데이터를 저장하는 InfluxDB 의 Pod]

- 운용중인 IoT 디바이스의 데이터 수집 및 저장 최적화를 위해 InfluxDB의 Pod화 완료



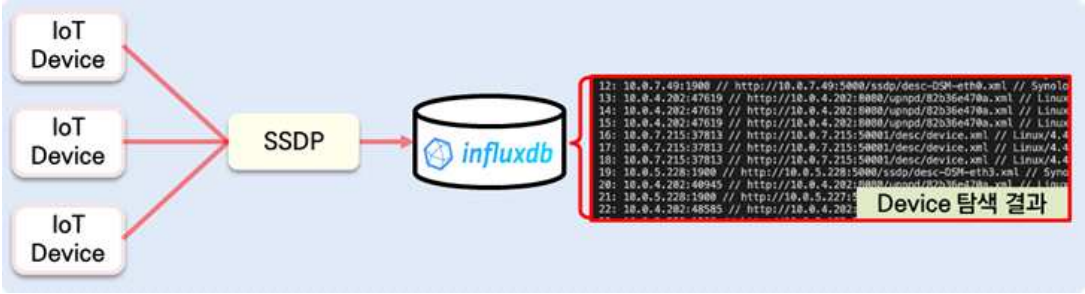
[IoT Collector API 내부 InfluxDB 연동]

- IoT Collector와 InfluxDB 연동 관련 API 개발을 통한 플랫폼 연동 안정화
- MQTT 통신과 관련된 Broker의 Pod화를 통해 통신 최적화 완료
- Worker 스레드를 통해 MQTT,CoAP,HTTP 통신으로 데이터 수집 시 지연 최소화

○ 컨테이너 기반 OneM2M(Mobius 플랫폼 or WSO2) 기반 IoT 플랫폼 연동 기술 최적화 및 안정화

• IoT 디바이스 Discovery 기술 최적화 및 플랫폼 연동 안정화

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



[IoT 디바이스 Discovery 기술 최적화]

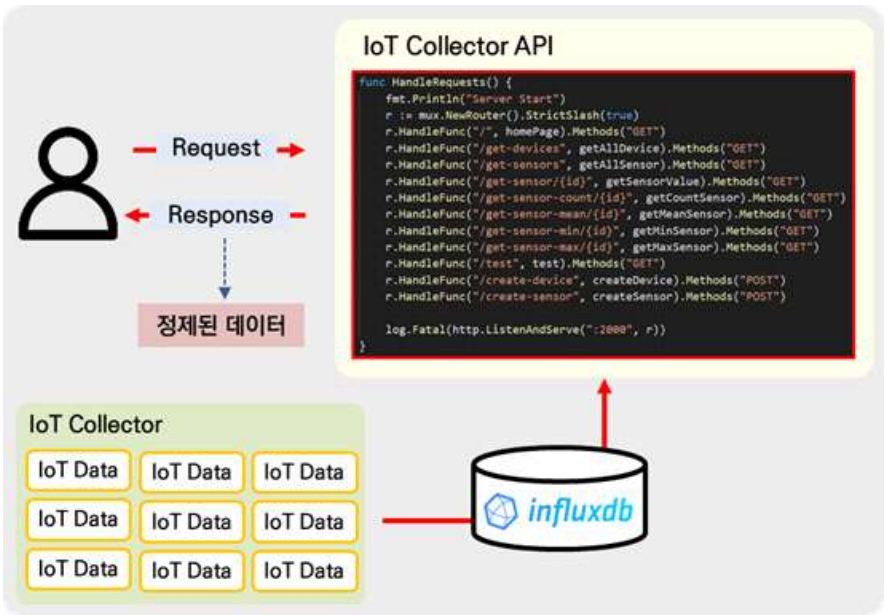
- 관리자의 편의성을 위해 SSDP를 활용하여 주변 디바이스 자동 탐색을 위한 Discovery 기술 최적화
- 디바이스 탐색을 위한 오픈소스 중 SSDP 사용을 통해 플랫폼과 디바이스 간 연동 안정화



[SSDP를 사용한 Device 탐색]

- 디바이스 탐색을 위한 Discovery 기술의 비동기적 처리

• 대규모 IoT 디바이스 데이터 저장을 위한 Repository 기술 안정화



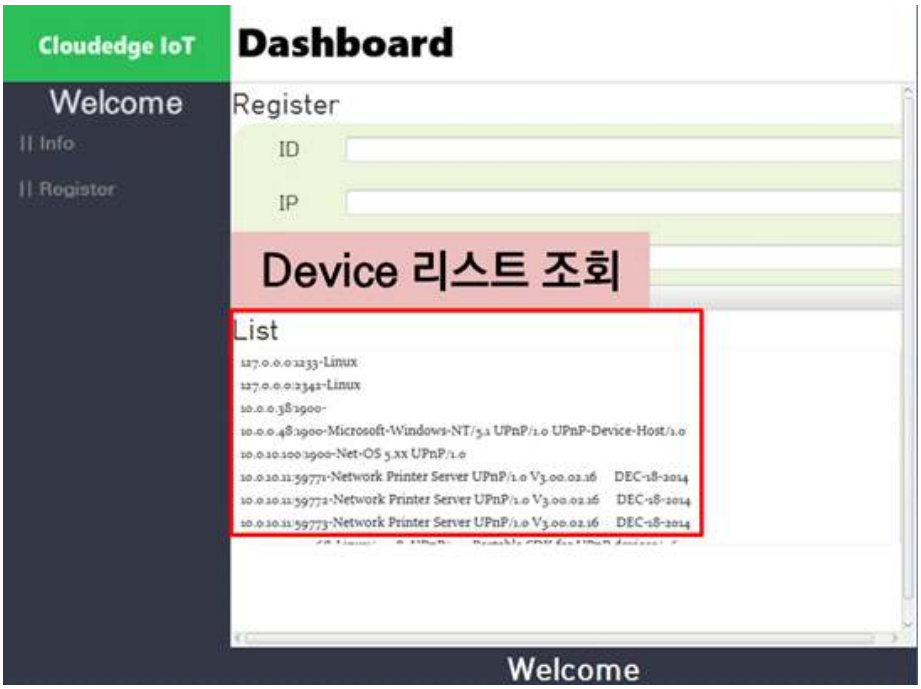
[데이터 정제를 위한 API]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

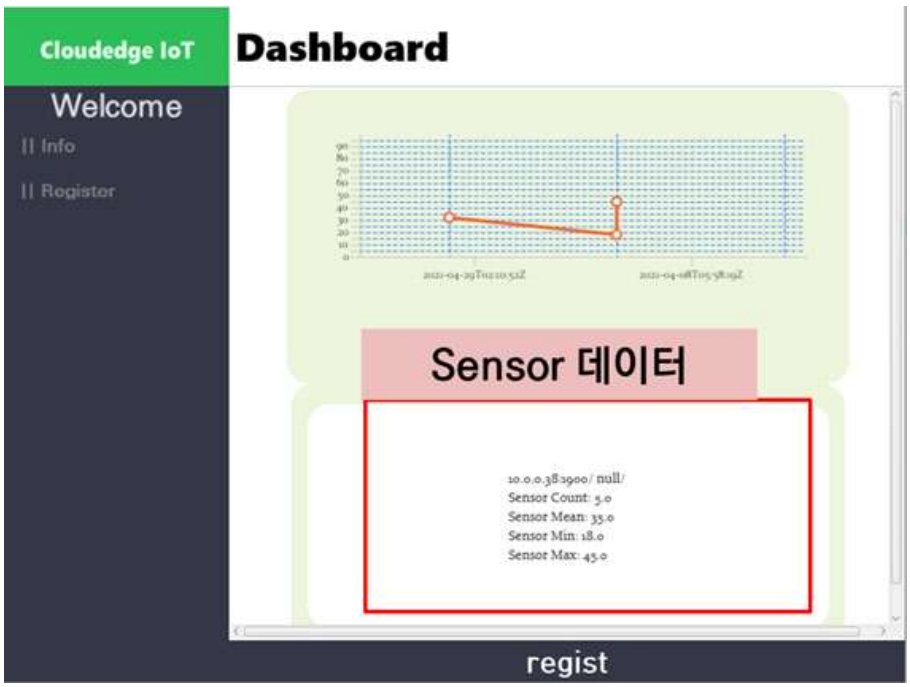
- IoT Collector에 수집된 IoT Data들의 데이터 정제를 위한 독립적인 API 개발
- 시각화 모듈과의 연동을 위한 REST API 사용 및 확장
- InfluxDB Measurement 중 Device, Sensor 관련 데이터 정제를 목적으로 하는 API 개발을 통해 Repository 기술 안정화

○ 클라우드 엣지 컨테이너 기반 IoT 응용 서비스 시각화 기술 안정화

• 개발 컨테이너 기반 OneM2M(Mobius 플랫폼 or WSO2) 연결 지원을 위한 시각화 안정화



[시각화 모듈 중 Device 리스트 조회]



[시각화 모듈 중 Sensor 데이터 조회]

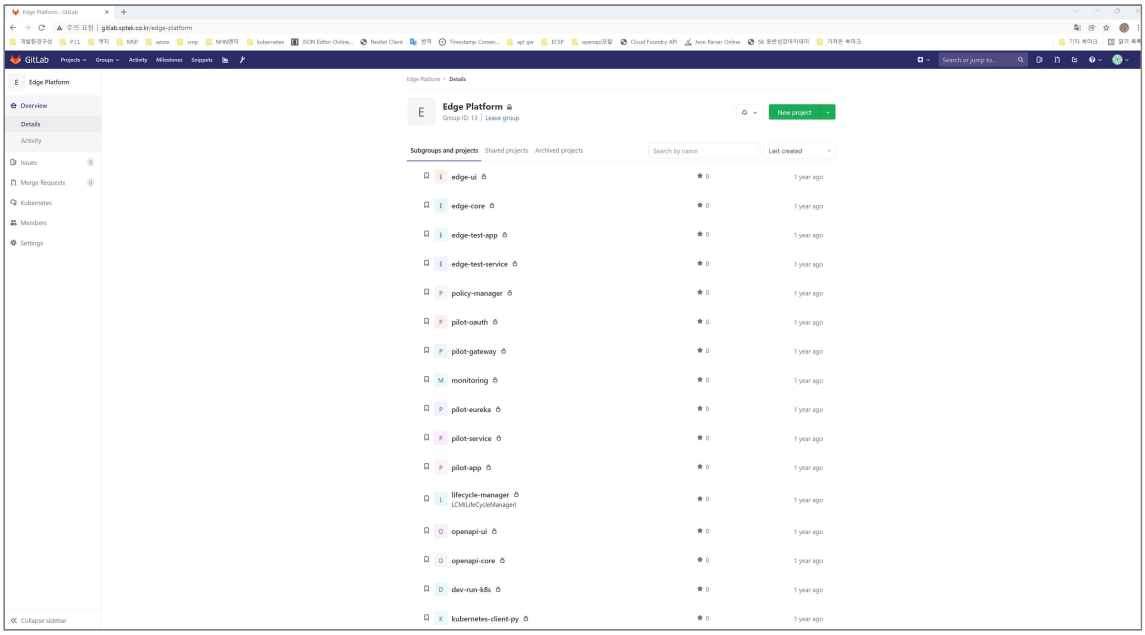
- InfluxDB 내부 데이터 조회를 위한 시각화 모듈 개발
- IoT 관련 데이터 정제 API를 통해 Device, Sensor 데이터 조회

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

자체 SW 품질관리

개발결과물의 품질 확보 위해 내부적으로 SW품질관리를 수행함

▢ 주관사 내에 “Gitlab”을 구축하여 개발 라이프사이클 관리 및 각 참여기관과의 의사소통, 산출물 관리에 활용함.



[주관사 내 엣지 플랫폼 개발을 위한 Gitlab 사이트]

○ 주관사 내에 구축된 Gitlab에 각 참여기관 담당자들을 등록하여 의사소통, 일정, 산출물 등을 관리하며 통합 기능시험 및 검증을 위한 업무를 수행 중

기관	Gitlab 멤버
스트라토 (구, 에스피테크놀러지)	김형두(상무), 김화성(부장), 백승현(부장), 김민수(차장), 이명호(차장), 김재훈(차장), 유승백(대리), 조대진(대리)
KETI	안재훈(책임), 김영환(수석), 전기만(책임), 허태혁(원)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

표준화 활동

▢ 국내 표준화 활동내역

○ 표준 제정을 위한 기반 활동 추진

- TTA PG1003(클라우드) 회의 참여

·‘19.3.19(한국MS), ‘19.5.22(한국MS), ’19.7.17(한국MS)

·‘20.4.29(화상회의), ‘20.6.18(서울비즈센터), ‘20.8.19(NIA 서울사무소)

·’21.1.21(화상회의), ‘21.3.17(화상회의), ’21.7.30(화상회의), ‘21.9.3(화상회의)

- 지능형컴퓨팅기술포럼 회의 참여

·‘19.4.10(역삼), ’19.6.28(제주)

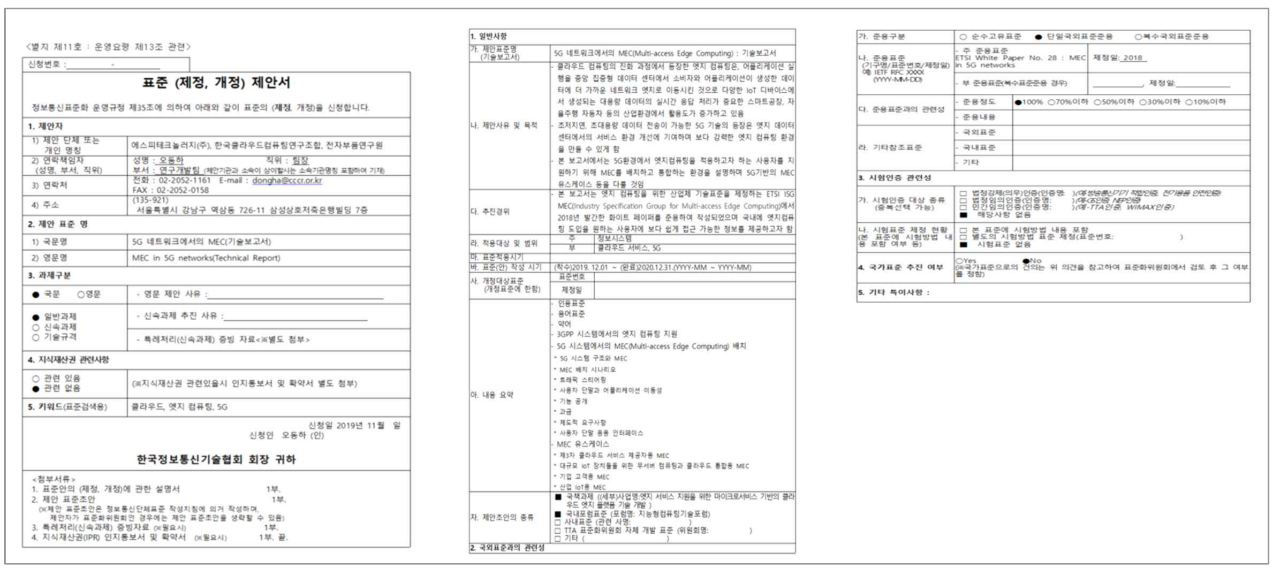
·‘20.4.28(Zoom), ‘20.10.23(서면)

·‘21.4.27(화상회의), ‘21.5.18(화상회의), ’21.6.21(화상회의), ‘21.6.25(제주)

○ 표준화 항목 개발 및 표준화 제안서 작성

- “5G 네트워크에서의 MEC(기술보고서)” 내용으로 표준화 항목 개발 및 표준화 제안서 작성

· 엣지 컴퓨팅을 위한 산업체 기술표준을 제정하는 ETSI ISG MEC에서 2018년 발간한 화이트 페이퍼를 준용하여 표준을 개발



〈별지 제11호 : 운영요령 제13조 관련〉

신청번호 :

표준 (제정, 개정) 제안서

정보통신표준화 운영규정 제35조에 의하여 아래와 같이 표준의 (제정, 개정)을 신청합니다.

1. 제안자		
1) 제안 단체 또는 개인 명칭	에스피테크놀러지(주), 한국클라우드컴퓨팅연구조합, 전자부품연구원	
2) 연락책임자 (성명, 부서, 직위)	성명 : 오동하 직위 : 팀장 부서 : 연구개발팀 (제안기관과 소속이 상이할시는 소속기관명칭 포함하여 기재)	
3) 연락처	전화 : 02-2052-1161 E-mail : donghai@cccr.or.kr FAX : 02-2052-0158	
4) 주소	(135-921) 서울특별시 강남구 역삼동 726-11 삼성상호저축은행빌딩 7층	
2. 제안 표준 명		
1) 국문명	5G 네트워크에서의 MEC(기술보고서)	
2) 영문명	MEC in 5G networks(Technical Report)	
3. 과제구분		
● 국문 ○영문	· 영문 제안 사유 :	
● 일반과제 ○ 신속과제 ○ 기술규격	· 신속과제 추진 사유 : · 특례처리(신속과제) 증빙 자료<별첨 참부>	
4. 지식재산권 관련사항		
○ 관련 있음 ● 관련 없음	(※지식재산권 관련있을시 인지통보서 및 확약서 별도 첨부)	
5. 키워드(표준검색용)	클라우드, 엣지 컴퓨팅, 5G	

신청일 2019년 11월 일

신청인 오동하 (인)

한국정보통신기술협회 회장 귀하

<첨부서류>
1. 표준안의 (제정, 개정)에 관한 설명서 1부.
2. 제안 표준초안 1부.
(※제안 표준초안은 정보통신단체표준 작성지침에 의거 작성하며, 제안자가 표준화위원인 경우에는 제안 표준초안을 생략할 수 있음)
3. 특례처리(신속과제) 증빙자료 (※필요시) 1부.
4. 지식재산권(IPR) 인지통보서 및 확약서 (※필요시) 1부. 끝.

1. 일반사항	
가. 제안표준명 (기술보고서)	5G 네트워크에서의 MEC(Multi-access Edge Computing) : 기술보고서
나. 제안사유 및 목적	- 클라우드 컴퓨팅의 진화 과정에서 등장한 엣지 컴퓨팅은, 어플리케이션 실행을 중앙 집중형 데이터 센터에서 소비자와 어플리케이션이 생성한 데이터에 더 가까운 네트워크 엣지로 이동시킨 것으로 다양한 IoT 디바이스에서 생성되는 대용량 데이터의 실시간 응답 처리가 중요한 스마트공장, 자율주행 자동차 등의 산업환경에서 활용도가 증가하고 있음 - 초지지연, 초대용량 데이터 전송이 가능한 5G 기술의 등장은 엣지 데이터 센터에서의 서비스 환경 개선에 기여하여 보다 강력한 엣지 컴퓨팅 환경을 만들 수 있게 함 - 본 보고서에서는 5G환경에서 엣지컴퓨팅을 적용하고자 하는 사용자를 지원하기 위해 MEC를 배치하고 통합하는 환경을 설명하여 5G기반의 MEC 유스케이스 등을 다룰 것임
다. 추진경위	- 본 보고서는 엣지 컴퓨팅을 위한 산업계 기술표준을 제정하는 ETSI ISG MEC(Industry Specification Group for Multi-access Edge Computing)에서 2018년 발간한 화이트 페이퍼를 준용하여 작성되었으며 국내에 엣지컴퓨팅 도입을 원하는 사용자에 보다 쉽게 접근 가능한 정보를 제공하고자 함
라. 적용대상 및 범위	주 정보시스템 정 클라우드 서비스, 5G
마. 표준적용지기	
바. 표준(안) 작성 시기	(착수)2019. 12.01 ~ (완료)2020.12.31.(YYYY-MM ~ YYYY-MM)
사. 개정대상표준 (개정표준에 한함)	표준번호 제정일
아. 내용 요약	인용표준 용어표준 약어 - 3GPP 시스템에서의 엣지 컴퓨팅 지원 - 5G 시스템에서의 MEC(Multi-access Edge Computing) 배치 * 5G 시스템 구조와 MEC * MEC 배치 시나리오 * 트래픽 스티어링 * 사용자 단말과 어플리케이션 이동성 * 기능 공개 * 과금 * 제도적 요구사항 * 사용자 단말 응용 인터페이스 - MEC 유스케이스 * 제3자 클라우드 서비스 제공자용 MEC * 대규모 IoT 장치들을 위한 루서버 컴퓨팅과 클라우드 통합용 MEC * 기업 고객용 MEC * 산업 IoT용 MEC
자. 제안초안의 종류	■ 국책과제 (세종시/산업엔엣지 서비스 지원을 위한 마이크로서비스 기반의 클라우드 엣지 플랫폼 기술 개발) ■ 국내표준초안 (포함명: 지능형컴퓨팅기술포럼) □ 사내표준 (관련 사항:) □ TTA 표준화위원회 자체 개발 표준 (위원회명:) □ 기타 ( )
2. 국외표준과의 관련성	

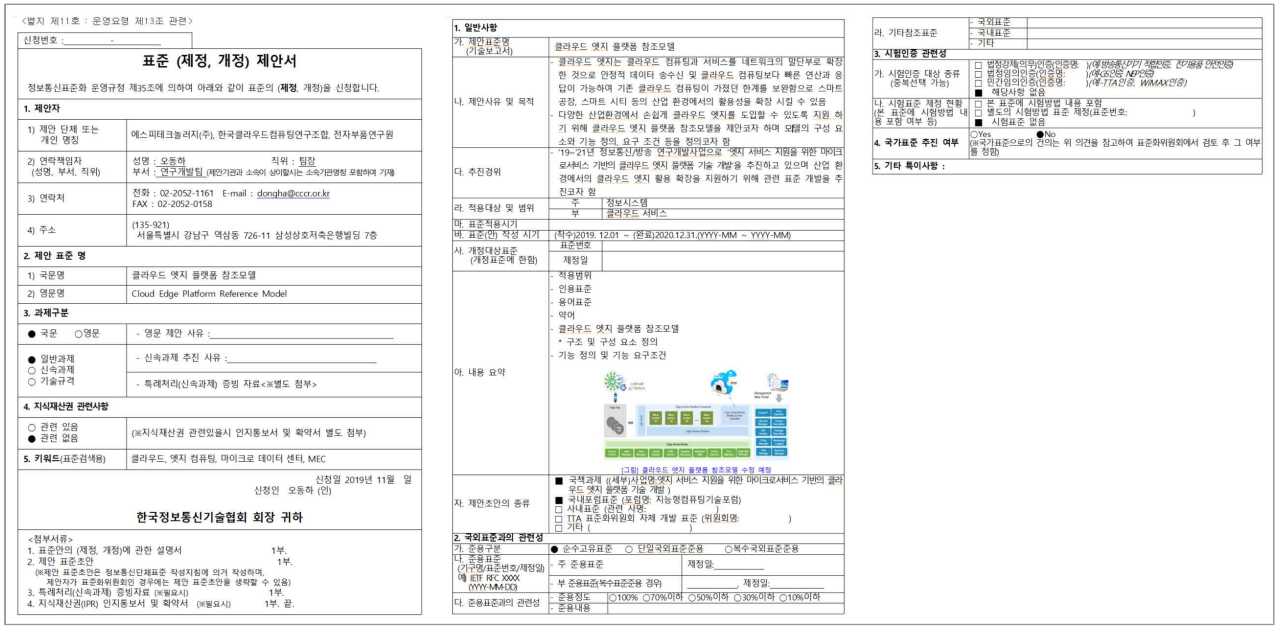
가. 준용구분	○ 순수고유표준 ● 단일국외표준준용 ○복수국외표준준용
나. 준용표준 (가)구분/표준번호/제정일) 예) IETF RFC XXXX (YYYY-MM-DD)	- 주 준용표준 ETSI White Paper No. 28 : MEC in 5G networks 제정일: 2018
	- 부 준용표준(복수표준준용 경우) 제정일:
다. 준용표준과의 관련성	- 준용정도 ●100% ○70%이하 ○50%이하 ○30%이하 ○10%이하
	- 준용내용
라. 기타참조표준	- 국내표준
	- 국내표준
	- 기타
3. 시험인증 관련성	
가. 시험인증 대상 종류 (중복선택 가능)	□ 법정강제(의무)인증(인증명:)(예:정보통신기기 적합인증, 전기용품 안전인증) □ 법정임의인증(인증명:)(예:KS인증, ABP인증) □ 민간임의인증(인증명:)(예:TTA인증, WiMAX인증) ■ 해당사항 없음
나. 시험표준 제정 현황 (본 표준에 시험방법 내용 포함 여부 등)	□ 본 표준에 시험방법 내용 포함 ■ 별도의 시험방법 표준 제정(표준번호:) ■ 시험표준 없음
4. 국가표준 추진 여부	○Yes ●No (국가표준으로의 전환는 위 의견을 참고하여 표준화위원회에서 검토 후 그 여부를 정함)
5. 기타 특이사항 :	

[표준화 과제제안서]

- 산업 환경에서의 클라우드 엣지 활용 확장을 지원하기 위해 클라우드 엣지 플랫

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

폼의 구조 및 구성 요소를 정의한 참조 모델과 기능 정의 및 기능 요구 조건을 내용으로 표준화 항목 개발 및 표준화 제안서 작성



〈별지 제11호 : 운영요령 제13조 관련〉

신청번호 :

표준 (제정, 개정) 제안서

정보통신표준화 운영규정 제35조에 의하여 아래와 같이 표준의 (**제정**, 개정)을 신청합니다.

1. 제안자	
1) 제안 단체 또는 개인 명칭	에스피테크놀러지(주), 한국클라우드컴퓨팅연구조합, 전자부품연구원
2) 연락책임자 (성명, 부서, 직위)	성명 : 오동하 직위 : 팀장 부서 : 연구개발팀 (제안기관과 소속이 상이할시는 소속기관명칭 포함하여 기재)
3) 연락처	전화 : 02-2052-1161 E-mail : dongha@cccr.or.kr FAX : 02-2052-0158
4) 주소	(135-921) 서울특별시 강남구 역삼동 726-11 삼성상호저축은행빌딩 7층
2. 제안 표준 명	
1) 국문명	클라우드 엣지 플랫폼 참조모델
2) 영문명	Cloud Edge Platform Reference Model
3. 과제구분	
● 국문 ○영문	- 영문 제안 사유 :
● 일반과제 ○ 신속과제 ○ 기술규격	- 신속과제 추진 사유 : - 특례처리(신속과제) 증빙 자료<※별도 첨부>
4. 지식재산권 관련사항	
○ 관련 있음 ● 관련 없음	(※지식재산권 관련있을시 인지통보서 및 확약서 별도 첨부)
5. 키워드(표준검색용)	클라우드, 엣지 컴퓨팅, 마이크로 데이터 센터, MEC

신청일 2019년 11월 일

신청인 오동하 (인)

한국정보통신기술협회 회장 귀하

<첨부서류>
1. 표준안의 (제정, 개정)에 관한 설명서 1부.
2. 제안 표준초안 1부.
(※제안 표준초안은 정보통신단체표준 작성지침에 의거 작성하며, 제안자가 표준화위원회인 경우에는 제안 표준초안을 생략할 수 있음)
3. 특례처리(신속과제) 증빙자료 (※필요시) 1부.
4. 지식재산권(IPR) 인지통보서 및 확약서 (※필요시) 1부. 끝.

1. 일반사항		
가. 제안표준명 (기술보고서)	클라우드 엣지 플랫폼 참조모델	
나. 제안사유 및 목적	- 클라우드 엣지는 클라우드 컴퓨팅과 서비스를 네트워크의 말단부로 확장한 것으로 안정적 데이터 송수신 및 클라우드 컴퓨팅보다 빠른 연산과 응답이 가능하여 기존 클라우드 컴퓨팅이 가졌던 한계를 보완함으로 스마트 공장, 스마트 시티 등의 산업 환경에서의 활용성을 확장 시킬 수 있음 - 다양한 산업환경에서 손쉽게 클라우드 엣지를 도입할 수 있도록 지원 하기 위해 클라우드 엣지 플랫폼 참조모델을 제안코자 하며 모델의 구성 요소와 기능 정의, 요구 조건 등을 정의코자 함	
다. 추진경위	- '19-21년 정보통신기술진흥센터 '연구개발사업으로 "엣지 서비스 지원을 위한 마이크로서비스 기반의 클라우드 엣지 플랫폼 기술 개발'을 추진하고 있으며 산업 환경에서의 클라우드 엣지 활용 확장을 지원하기 위해 관련 표준 개발을 추진코자 함	
라. 적용대상 및 범위	주	정보시스템
	부	클라우드 서버스
마. 표준적용시기		
바. 표준(안) 작성 시기	(착수)2019. 12.01 ~ (완료)2020.12.31.(YYYY-MM ~ YYYY-MM)	
사. 개정대상표준 (개정표준에 한함)	표준번호	
	제정일	
아. 내용 요약	- 적용범위 - 인용표준 - 용어표준 - 약어 - 클라우드 엣지 플랫폼 참조모델 * 구조 및 구성 요소 정의 - 기능 정의 및 기능 요구조건	
자. 제안조안의 종류	■ 국책과제 ((세부)사업명:엣지 서비스 지원을 위한 마이크로서비스 기반의 클라우드 엣지 플랫폼 기술 개발) ■ 국내보험표준 (포함명: 지능형컴퓨팅기술포럼) □ 사내표준 (관련 사명:) □ TTA 표준화위원회 자체 개발 표준 (위원회명:) □ 기타 ( )	
2. 국외표준과의 관련성		
가. 준용구분	● 순수고유표준 ○ 단일국외표준준용 ○복수국외표준준용	
다. 준용표준 (기구명/표준번호/제정일) 예 IETF RFC XXXX (YYYY-MM-DD)	- 주 준용표준	제정일:
	- 부 준용표준(복수표준준용 경우)	제정일:
다. 준용표준과의 관련성	- 준용정도 ○100% ○70%이하 ○50%이하 ○30%이하 ○10%이하	
	- 준용내용	

라. 기타참조표준	- 국외표준	
	- 국내표준	
	- 기타	
3. 시험인증 관련성		
가. 시험인증 대상 종류 (중복선택 가능)	□ 법정강제의무인증(인증명:)(예:방송통신기기 적합인증, 전기용품 안전인증) □ 법정임의인증(인증명:)(예:GCF인증, NEP인증) □ 민간임의인증(인증명:)(예:TTA인증, WiMAX인증) ■ 해당사항 없음	
나. 시험표준 제정 현황 (본 표준에 시험방법 내용 포함 여부 등)	□ 본 표준에 시험방법 내용 포함 □ 별도의 시험방법 표준 제정(표준번호:) ■ 시험표준 없음	
4. 국가표준 주진 여부	○Yes ●No (※국가표준으로의 전이는 위 의견을 참고하여 표준화위원회에서 검토 후 그 여부를 정함)	
5. 기타 특이사항 :		

[표준화 과제제안서]

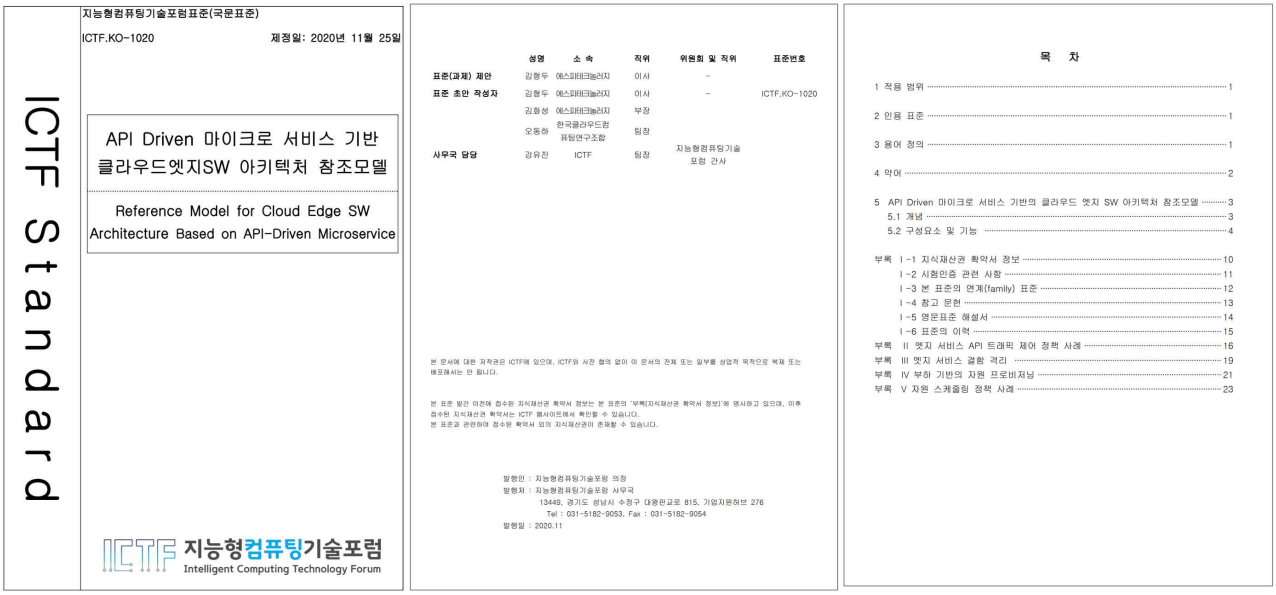
○ 포럼 표준 제정 추진

- TTA의 전략 포럼인 지능형 컴퓨팅 기술포럼 기술표준 제안 및 제정 (ICTF.KO-1020, API Driven 마이크로 서비스 기반 클라우드 엣지 SW 아키텍처 참조모델, '20.11.25)



[ICTF.KO-1020, 지능형 컴퓨팅 기술 포럼 홈페이지 게재]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



ICTF Standard

지능형컴퓨팅기술포럼표준(국문표준)

ICTF.KO-1020 제정일: 2020년 11월 25일

API Driven 마이크로 서비스 기반 클라우드엣지SW 아키텍처 참조모델

Reference Model for Cloud Edge SW Architecture Based on API-Driven Microservice

ICTF 지능형컴퓨팅기술포럼 Intelligent Computing Technology Forum

	성명	소 속	직위	위원회 및 직위	표준번호
표준(과제) 제안	김형두	에스피테크놀러지	이사	-	
표준 초안 작성자	김형두	에스피테크놀러지	이사	-	ICTF.KO-1020
	김철성	에스피테크놀러지	부장		
	오동헌	한국클라우드컴퓨팅연구조합	팀장		
사무국 담당	강유진	ICTF	팀장	지능형컴퓨팅기술포럼 간사	

본 문서에 대한 저작권은 ICTF에 있으며, ICTF의 사전 협의 없이 이 문서의 전체 또는 일부를 상업적 목적으로 복제 또는 배포해서는 안 됩니다.

본 표준 발간 이전에 접수된 지식재산권 확약서 정보는 본 표준의 '부록(지식재산권 확약서 정보)'에 명시하고 있으며, 이후 접수된 지식재산권 확약서는 ICTF 웹사이트에서 확인할 수 있습니다.
본 표준과 관련하여 접수된 확약서 외의 지식재산권이 존재할 수 있습니다.

발행인 : 지능형컴퓨팅기술포럼 의장
발행처 : 지능형컴퓨팅기술포럼 사무국
13449, 경기도 성남시 수정구 대왕판교로 815, 기업지원허브 276
Tel : 031-5182-9053, Fax : 031-5182-9054
발행일 : 2020.11

목 차

1 적용 범위 1
2 인용 표준 1
3 용어 정의 1
4 약어 2
5 API Driven 마이크로 서비스 기반의 클라우드 엣지 SW 아키텍처 참조모델 3
5.1 개념 3
5.2 구성요소 및 기능 4
부록 I-1 지식재산권 확약서 정보 10
I-2 시험인증 관련 사항 11
I-3 본 표준의 연계(family) 표준 12
I-4 참고 문헌 13
I-5 영문표준 해설서 14
I-6 표준의 이력 15
부록 II 엣지 서비스 API 트래픽 제어 정책 사례 16
부록 III 엣지 서비스 결함 격리 19
부록 IV 부하 기반의 자원 프로비저닝 21
부록 V 자원 스케줄링 정책 사례 23

[ICTF.KO-1020, API Driven 마이크로 서비스 기반 클라우드 엣지 SW 아키텍처 표준]

○ 포럼 표준(기술보고서) 제정 추진

- TTA의 전략 포럼인 지능형 컴퓨팅 기술포럼 표준 기술 보고서 제정 (ICTFR.KO-1024, 5G 네트워크에서의 MEC, '21.12.01)



한국컴퓨팅산업협회 Korea Computing Industry Association

협회소개 회원사안내 협회업무 공동A/S 알림 정보 교육 ICTF포럼 나의강의실 | 회원가입 | 로그인

Intelligent Computing Technology Forum

지능형컴퓨팅 기술 포럼

포럼

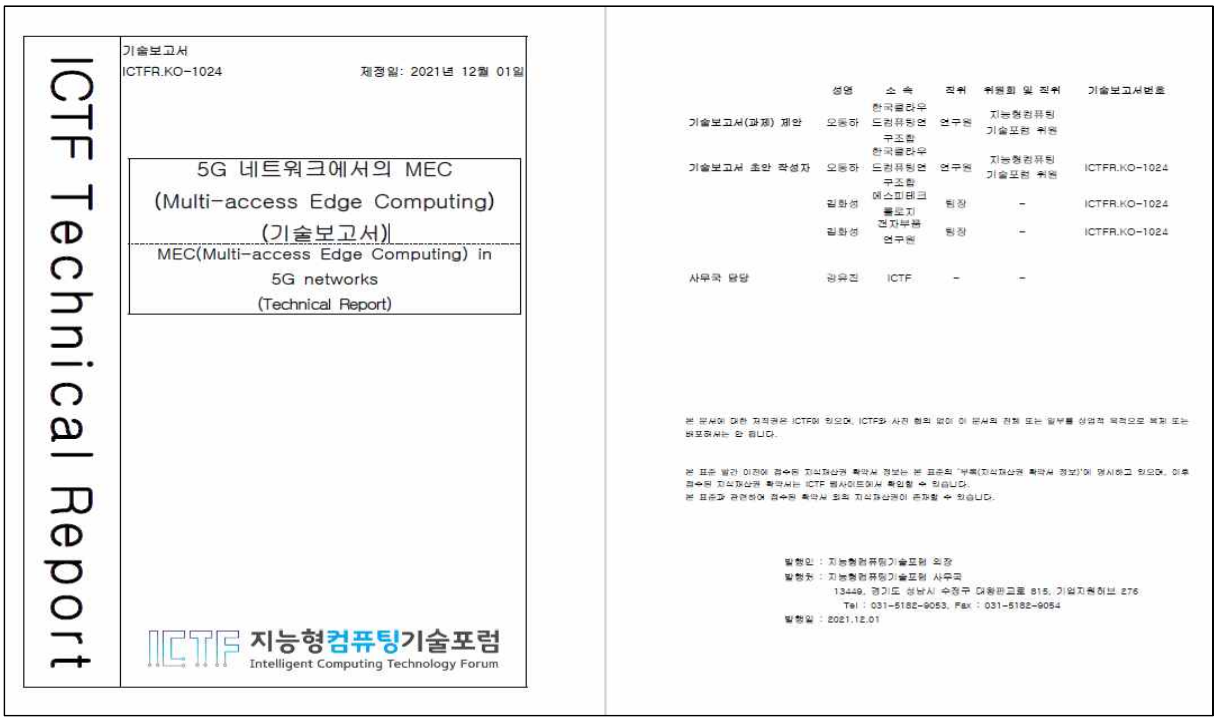
포럼소개
포럼일정
월별일정
연간일정
포럼활동
가입안내
포럼자료실

포럼자료실

번호	제목	글쓴이	날짜	조회
공지	지능형컴퓨팅기술포럼 표준 및 기술보고서 작성 양식	admin	2020-09-09	172
공지	포럼 가입 신청서	admin	2020-05-12	170
39	[2021 포럼표준]ICTFR.KO-1024 5G 네트워크에서의 MEC 활용 (기술보고서)	admin	2021-12-07	5
38	[2021 포럼표준]ICTFR.KO-1023 하이브리드 클라우드 구축 실무 가이드(기술보고서)	admin	2021-12-07	4
37	[2021 포럼표준]ICTF.KO-1022 서버 전력 및 열 관리를 위한 베이스보드 관리 컨트롤러(BMC)…	admin	2021-12-07	6
36	[2021 포럼표준]ICTF.KO-1021 데이터처리용 컴퓨터서버의 성능 요구사항 및 성능 시험 가이드라인	admin	2021-12-07	5
35	[2021 포럼 표준]ICTF.KP-1021_R1, API Driven 마이크로 서비스 기반 클라우드엣지SW…	admin	2021-12-07	6
34	2021 지능형컴퓨팅기술 웨비나 발표자료	admin	2021-07-05	68

[ICTFR.KO-1024, 지능형 컴퓨팅 기술 포럼 홈페이지 게재]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



ICTF Technical Report

기술보고서
ICTFR.KO-1024 제정일: 2021년 12월 01일

5G 네트워크에서의 MEC
(Multi-access Edge Computing)
(기술보고서)

MEC(Multi-access Edge Computing) in
5G networks
(Technical Report)

ICTF 지능형컴퓨팅기술포럼
Intelligent Computing Technology Forum

	성명	소 속	직위	위원회 및 직위	기술보고서번호
기술보고서(과제) 제안	오동하	한국클라우드컴퓨팅연구조합	연구원	지능형컴퓨팅 기술포럼 위원	
기술보고서 초안 작성자	오동하	한국클라우드컴퓨팅연구조합	연구원	지능형컴퓨팅 기술포럼 위원	ICTFR.KO-1024
	김화성	에스피테크놀로지	팀장	-	ICTFR.KO-1024
	김화성	전자부품연구원	팀장	-	ICTFR.KO-1024
사무국 담당	권유진	ICTF	-	-	

본 문서에 대한 저작권은 ICTF에 있으며, ICTF의 사전 허락 없이 이 문서의 전체 또는 일부를 상업적 목적으로 복제 또는 배포하여서는 안 됩니다.

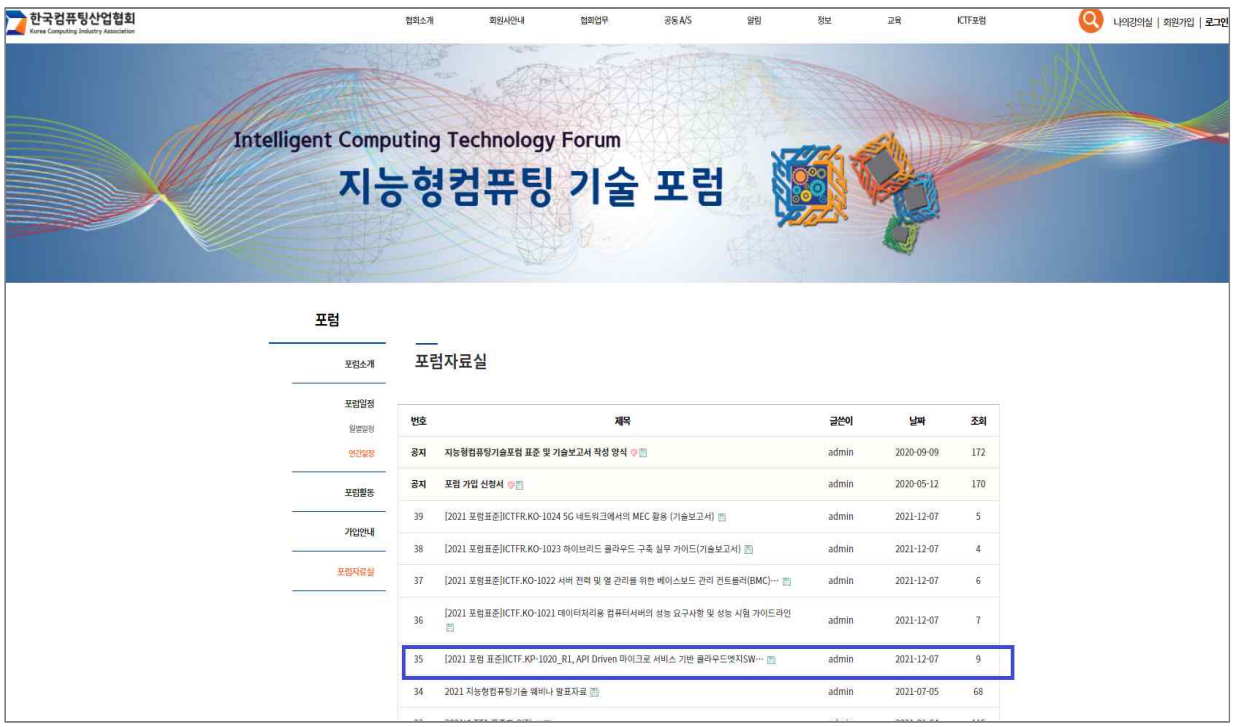
본 표준 발간 이전에 검수된 지식재산권 확약서 정보는 본 표준의 "부록(지식재산권 확약서 정보)"에 명시하고 있으며, 이후 검수된 지식재산권 확약서는 ICTF 웹사이트에서 확인할 수 있습니다.
본 표준과 관련하여 검수된 확약서 외의 지식재산권이 존재할 수 있습니다.

발행인 : 지능형컴퓨팅기술포럼 회장
발행처 : 지능형컴퓨팅기술포럼 사무국
13449, 경기도 성남시 수정구 대왕판교로 815, 기업지원허브 276
Tel : 031-5182-9053, Fax : 031-5182-9054
발행일 : 2021.12.01

[ICTFR.KO-1024, 5G 네트워크에서의 MEC 표준 기술 보고서]

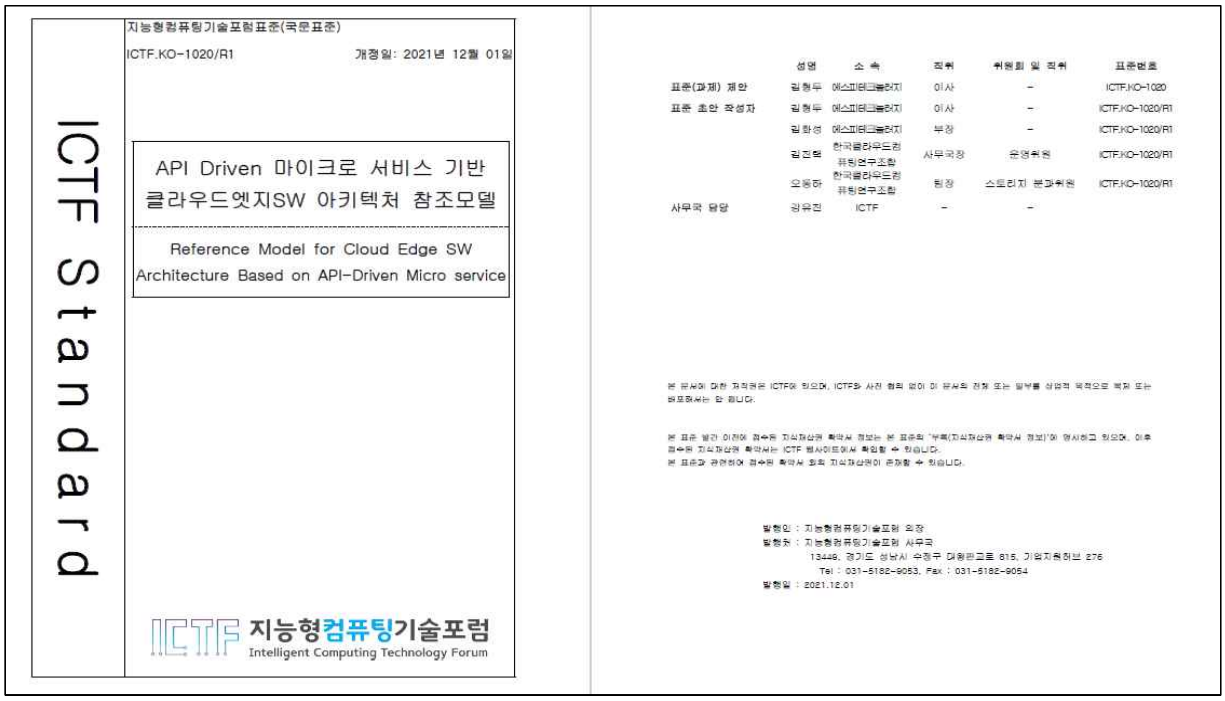
○ 포럼 표준 추진(개정)

- TTA의 전략 포럼인 지능형 컴퓨팅 기술포럼 기술표준 개정(ICTF.KO-1020-R1, API Driven 마이크로 서비스 기반 클라우드 엣지 SW 아키텍처 참조모델, '21.12.01)



[ICTF.KO-1020-R1, 지능형 컴퓨팅 기술 포럼 홈페이지 게재]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다



ICTF Standard

지능형컴퓨팅기술포럼표준(국문표준)

ICTF.KO-1020/R1 개정일: 2021년 12월 01일

API Driven 마이크로 서비스 기반 클라우드엣지SW 아키텍처 참조모델

Reference Model for Cloud Edge SW Architecture Based on API-Driven Micro service

ICTF 지능형컴퓨팅기술포럼 Intelligent Computing Technology Forum

	성명	소 속	직위	위원회 및 직위	표준번호
표준(과제) 제안	김형두	에스피테크놀러지	이사	-	ICTF.KO-1020
표준 초안 작성자	김형두	에스피테크놀러지	이사	-	ICTF.KO-1020/R1
	김한성	에스피테크놀러지	부장	-	ICTF.KO-1020/R1
	김진혁	한국클라우드컴퓨팅연구조합	사무국장	운영위원	ICTF.KO-1020/R1
	오동하	한국클라우드컴퓨팅연구조합	팀장	스토리지 분과위원	ICTF.KO-1020/R1
사무국 담당	장유진	ICTF	-	-	

본 문서에 대한 저작권은 ICTF에 있으며, ICTF와 사전 협의 없이 이 문서의 전체 또는 일부를 상업적 목적으로 복제 또는 배포해서는 안 됩니다.

본 표준 발간 이전에 접수된 지식재산권 확약서 정보는 본 표준의 '부록(지식재산권 확약서 정보)'에 명시하고 있으며, 이후 접수된 지식재산권 확약서는 ICTF 웹사이트에서 확인할 수 있습니다.
본 표준과 관련하여 접수된 확약서 외의 지식재산권이 존재할 수 있습니다.

발행인 : 지능형컴퓨팅기술포럼 의장
발행처 : 지능형컴퓨팅기술포럼 사무국
13449, 경기도 성남시 수정구 대왕판교로 815, 기업지원허브 276
Tel : 031-5182-9053, Fax : 031-5182-9054
발행일 : 2021.12.01

[ICTF.KO-1020-R1, API Driven 마이크로서비스 기반 클라우드 엣지 SW 아키텍처 표준]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

기술문서

번호	기술문서명	문서번호	작성기관
1	SPTek_01_요구사항 정의서	SPTek_01	스트라토
2	SPTek_02_상위 설계서	SPTek_02	스트라토
3	SPTek_03_기술 표준 항목 선정 검토 자료	SPTek_03	스트라토
4	SPTek_04_테이블 명세서	SPTek_04	스타라토
5	SPTek_05_인터페이스 정의서	SPTek_05	스타라토
6	SPTek_06_Redis Key 정의서	SPTek_06	스타라토
7	KETI_01_요구사항 정의서	KETI_01	KETI
8	KETI_02_상위 설계서	KETI_02	KETI
9	SPTek_06_LCM API 정의서	SPTek_06	스트라토
10	SPTek_07_관리포탈 화면설계서	SPTek_07	스트라토
11	SPTek_08_사용자 포탈 화면설계서	SPTek_08	스트라토
12	KETI_03_CRD버전 마이그레이션 매니저_설계	KETI_03	KETI
13	KETI_04_LSTM_워크로드예측모델_설계	KETI_04	KETI
14	KETI_05라이브마이그레이션_설계	KETI_05	KETI
15	SPTek_09_edge portal 관리자 매뉴얼	SPTek_09	스트라토
16	SPTek_10_edge portal 사용자 매뉴얼	SPTek_10	스트라토
17	Sptek_11_edge_portal_테이블명세서	SPTek_11	스트라토
18	KETI_06_CloudEdge_LSTM_예측모델_워크로드분석최적화_정의서	KETI_06	KETI
19	KETI_07_CloudEdge_Migration_환경별최적도_정의서	KETI_07	KETI
20	KETI_08_CloudEdge_IoT_Collector_연동서	KETI_07	KETI
21			
22			

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 연구개발과제의 수행 결과 및 목표 달성 정도

1) 연구수행 결과

(1) 정성적 연구개발성과

세부연구명	세부 연구 목표	연구 결과
마이크로서비스 기반의 엣지 서비스 관리기술 개발	엣지 환경/특성에 따른 마이크로서비스 기반의 엣지 서비스 조합과 연결 기술 개발	• API 단위 서비스 처리를 위한 마이크로서비스 기반의 엣지 플랫폼 아키텍처 설계 및 개발 • API Driven 마이크로 서비스 기반 클라우드 엣지 SW 아키텍처 참조모델 표준 제정(ICTF.KO-1020/R1) • 엣지 서비스 라우팅 정보 등록 관리 기능 개발 • Client-Server 형태의 서비스 연결 라우팅 기능 개발 • 복수 엣지 서비스 인스턴스에 대한 로드밸런싱 기능 개발
	엣지 앱의 API 요청에 따른 트래픽 조절과 결함 격리 기술 개발	• 엣지 앱의 API별 일 단위 사용량 기준에 따른 트래픽 조절 기능 개발 • 엣지 앱 Access Token Validation 실패에 따른 트래픽 조절 기능 개발 • 엣지 앱 인증 실패에 따른 트래픽 조절 기능 개발 • Circuit Breaker를 이용한 엣지 서비스 결함 대비 격리 기술 개발
	Kubernetes 기반의 엣지 서비스 컨테이너 관리 구조 개발	• Kubernetes Name Space 기반의 엣지 앱, 엣지 서비스, Edge Service Framework 구성 모듈 배포 관리 기능 개발 • Kubernetes Deployment 기반의 엣지 앱, 엣지 서비스 인스턴스 관리 기능 개발 • Kubernetes NameS pace, Limit Range, Resource Quota 기반의 엣지 앱/서비스 리소스 사용 제한 관리 기능 개발
	엣지 앱과 엣지 플랫폼 구간 오픈 연동 기술 개발	• 3rd 엣지 응용을 위한 마이크로 서비스 기반의 플랫폼 표준 오픈 인터페이스 개발
	엣지 서비스 API 단위의 권한/허가 및 보안 인증 기술 개발	• OAuth2 기반의 인증 서버와 연동을 통한 인증 정보 생성 기능 개발 • 엣지 앱 인증 정보 유효성 체크 및 Access Token 발급 기능 개발 • 엣지 앱 Access Token에 대한 유효성 체크 기능 개발 • 엣지 앱의 엣지 서비스 API 단위 접근 권한 관리 기능 개발 • 엣지 앱의 엣지 서비스 API 접근 권한 체크 기능 개발
클라우드 엣지 플랫폼 라이프 사이클 관리 기술 개발	패키지 및 인스턴스 배포/실행을 위한 라이프 사이클 관리 모델 연구	• 엣지 앱/서비스 패키지 구조 설계 및 적용 • 엣지 앱/서비스 패키지 라이프 사이클 모델 구조에 맞춘 라이프 사이클 관리 기능 개발 • 엣지 앱/서비스 인스턴스 라이프 사이클 관리 기능 개발

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	엣지 앱 배포/실행 관리를 위한 네트워크 서비스 디스크립터 기술 개발	• 엣지 서비스/앱 서비스 디스크립터 설계 및 적용 • 엣지 서비스/앱 서비스 디스크립터 자동 생성 기능 개발
	엣지 플랫폼과 엣지 플랫폼 관리 구간 연동 기술 개발	• 엣지 플랫폼과 엣지 플랫폼 관리 구간 오픈 인터페이스 25종 개발
	엣지 앱 및 서비스 라이스 사이클 관리를 위한 운용자 및 사용자 포털 제공 기술 개발	• 엣지 앱/서비스 등록, 배포 , 인스턴스 관리, 인증 관리 및 API 이용 통계 정보를 제공하는 엣지 운용자/사용자 포털 개발
고확장성 및 고가용성 제공을 위한 클라우드 엣지 자원 자동 관리 기술 개발	자동 스케줄링을 위한 자원 워크로드 및 사용량 이벤트 수집 기술 개발	• 엣지 앱/서비스 인스턴스의 리소스 사용량 관련 이벤트 정책 관리 기능 개발 • 리소스 사용량 관련 메트릭 정의 • 리소스 사용량 수집 기능 개발 • 사용량 관련 이벤트 발생 판단 기능 개발
	API 실행을 위한 엣지 서비스 인스턴스 자원 할당 및 스케줄링 기술 개발	• 자원 스케줄링 정책 관리 기능 개발 • 요청 및 정책에 따른 컨테이너 및 인프라 오케스트레이션 기술 개발 • 다차원 메트릭 복합 분석을 통한 자원 자동 스케줄 기술 개발
	엣지 앱, 엣지 서비스의 부하 기반 자원 용량 프로비저닝 및 자동 조정 기술 개발	• 엣지 서비스 인스턴스 리소스 사용량에 따른 Scale In/Out 처리 기능 개발 • Self Healing 처리 기능 개발
Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발	마이크로서비스 기반 Zone In/Out 여부 판단 기술 개발	• 클라우드 엣지 자원 계층별(Node, VM, Container, Application) 다차원 메트릭 기반 워크로드 분석 기술 개발 • 클라우드 엣지 자원 계층별 워크로드 예측 기술 개발 • 엣지 서비스 워크로드에 따른 Live Migration Decision 기술 개발 • 클라우드 엣지 간 라이브 마이그레이션 정책 코어 개발
	마이크로서비스 기반의 Zone 이탈 여부 정보 전달 기술 개발	• 라이브 서비스 마이그레이션을 위한 이전 목적지 자원 요청 개발 • 실시간 마이그레이션 인증/인가/접근제어 개발 • 클라우드 엣지 Controller 기반 엣지 서비스 마이그레이션 Push(Alert) 기술 개발
	Zone 이동 시, 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발	• 엣지 서비스 자원 사용 특성(Memory 중심) 분석 기술 개발 및 배포 모듈 개발 • 플랫폼 간 State-Full & Stateless 서비스 라이브 마이그레이션 기술 개발 및 배포 모듈 개발 • 엣지 서비스 자원 특성에 따른 라이브 마이그레이션 알고리즘 추천 기술 개발 및 배포 모듈 개발 • 엣지 서비스 자원 특성에 따른 플랫폼간 라이브 마이그레이션 기술 최적화 및 배포 모듈 개발 • On-demand 마이그레이션 기술 개발 및 플랫폼 연동 • 주관 기관의 상용 서비스인 Smart Helper 서비스 시험 환경에서의 통합 연동

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

클라우드 엣지 컨테이너 기반 IoT 응용 서비스 설계 및 프로토타입 개발	대규모 IoT 디바이스 데이터 수집 서비스 응용 클러스터 개발	• 클라우드 엣지 플랫폼 MQTT, CoAP, HTTP 프로토콜 지원 IoT 데이터 Collector 개발 완료 • 다중 IoT 데이터 통합을 위한 Data Hub 기술 개발 및 플랫폼 연동 안정화
	컨테이너 기반 OneM2M(Mobius 플랫폼 or WSO2) 기반 IoT 플랫폼 연동 기술 개발	• IoT 디바이스 Discovery 기술 개발 및 플랫폼 연동 안정화 • 대규모 IoT 디바이스 데이터 저장을 위한 Repository 기술 개발
	클라우드 엣지 컨테이너 기반 IoT 응용 서비스 시각화 기술 개발	• 개발 컨테이너 기반 OneM2M 연결 지원을 위한 시각화 안정화

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

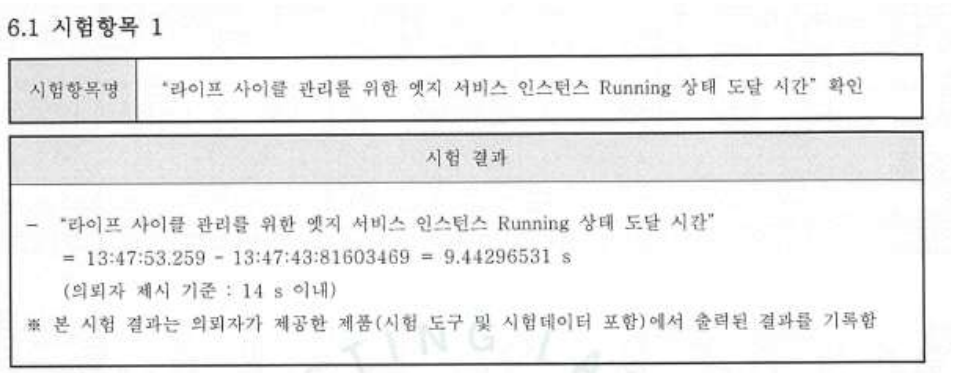
(2) 정량적 연구개발성과

▢ 정량적 목표 및 달성결과

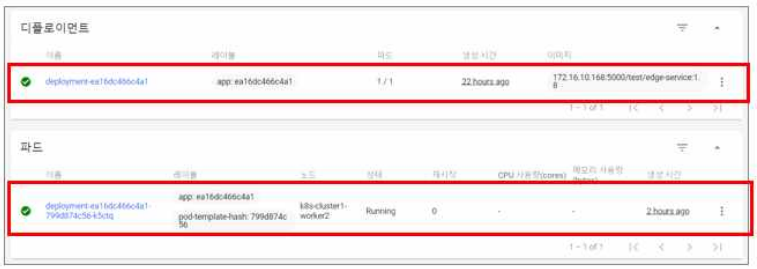
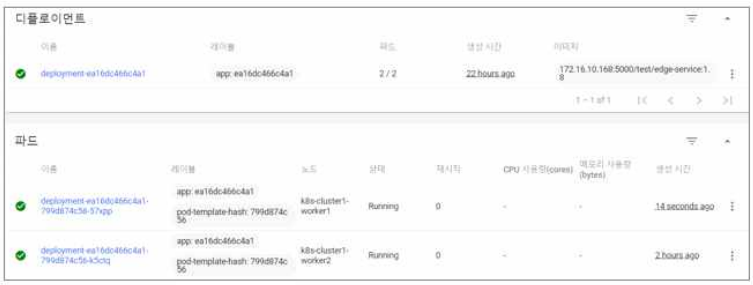
평가 항목 (주요성능)		단위	전체 항목에서 차지하는 비중(%)	세계 최고		연구개발 전 국내 성능수준	연구개발 목표치	최종 결과
				보유국/ 보유기업	성능수준	성능수준	1단계 (19~21)	
1	라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간	sec		미국 /Amazon	< 17	N/A	< 14	9.4
2	자원 자동 스케줄링 관리를 복합분석 정책 메트릭 항목 수	종		미국/ Amazon	2	N/A	5	6 (누적)
3	엣지 앱과 엣지 서비스 Life Cycle Management API 수	종		N/A	N/A	N/A	22	25 (누적)
4	클라우드 엣지 플랫폼 정보보호 및 보안인증 지원 항목 수	종		N/A	N/A	N/A	30	30 (누적)
5	라이브 마이그레이션 수행시 최대 Freeze Time	sec		미국/ RedHat	22	N/A	19	13.8
6	클라우드 엣지간 라이브 마이그레이션 정책 항목 수	종		미국/ Google	2	N/A	8	8 (누적)
7	워크로드에 따른 Live Migration Decision API 수	종		N/A	N/A	N/A	5	5 (누적)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

▢ 정량적 목표 항목별 평가방법

1. 라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간	
평가방법	• 정의 : 라이프 사이클 관리 및 자동 확장 시, 엣지 서비스 인스턴스가 기동한 후 Runnig 상태까지 도달하는데 걸리는 시간 • 성능 측정 수식 : ∑(AutoScaling으로 엣지 서비스 인스턴스가 기동하여 Running 상태 도달 시간)/Auto Scaling에 의한 엣지 서비스 인스턴스 기동 수 • 최종 성능 목표치 : 14sec • 성능 목표치 근거 : AWS Lamda SpringBoot 인스턴스의 2GB 메모리 환경에서 시작 시간(Cold Start) < 17sec 유지 (https://blog.octo.com/en/cold-start-warm-start-with-aws-lambda) • 성능 평가 도구 : 공인시험 확인 (라이프 사이클 관리 툴을 통한 측정 시간 표시) • 성능 평가 방법 : 최종 성능 목표치 대비 성능 비교
평가환경	• 개발 클라우드 엣지 플랫폼에서 라이프 사이클 관리 및 자동 확장 시, 엣지 서비스 인스턴스가 기동하여 Running 상태까지 도달하는데 걸리는 최대 시간 측정
공인시험결과	 [Scale Out 요청과 엣지 서비스 Registry 요청 시간]  6.1 시험항목 1 시험항목명 "라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간" 확인 시험 결과 – "라이프 사이클 관리를 위한 엣지 서비스 인스턴스 Running 상태 도달 시간" = 13:47:53.259 - 13:47:43:81603469 = 9.44296531 s (의뢰자 제시 기준 : 14 s 이내) ※ 본 시험 결과는 의뢰자가 제공한 제품(시험 도구 및 시험데이터 포함)에서 출력된 결과를 기록함 [KTL 공인 시험 결과] • 위 사진은 엣지 서비스에 대한 Auto Scale Out 수행 결과임 • LifeCycle Manager에서 엣지 서비스에 대해 Scale Out을 요청한 시간과 Scale Out에 의해 생성된 엣지 서비스 인스턴스가 Registry에 자신의 라우팅 정보를 등록한 시간차를 시간 측정 • 엣지 서비스 인스턴스가 기동하여 Running 상태까지 도달하는데 걸리는 시간이 약 9.45 sec로 당해 연도 목표치(<14 sec) 달성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 자원 자동 스케줄링 관리를 복합분석 정책 메트릭 항목 수	
평가방법	• 정의 : 자원 자동 스케줄링을 통한 엣지 인스턴스 Scale-Out/In을 결정하는 정책 메트릭 항목 수 • 성능 측정 수식 : ∑(스케줄링 정책 항목 수) • 최종 성능 목표치 : 5종 이상 • 성능 목표치 근거 : Amazon, Azure 등 퍼블릭 클라우드는 과금 기반의 메모리 리소스와 시스템 호출 빈도만을 활용하나, 본 과제에서는 다차원 복합분석을 위해 정책 메트릭 항목 수를 성능지표로 설정하여 자동 스케줄링 수행 • 성능 평가 도구 : 공인시험 확인 • 성능 평가 방법 : 최종 성능 목표치 대비 성능 비교
평가환경	• 개발 클라우드 엣지 플랫폼 자원 스케줄링 정책 항목 수 측정
공인시험결과	 네트워크 부하 발생 전 (Scale Out 이전 K8S 대시보드)  Monitoring의 network_receive 정보 수집로그  네트워크 부하 발생 후 (Scale Out 후 K8S 대시보드) [Network receive량에 따른 엣지 서비스 Scale Out 테스트 화면] 6.2 시험항목 2 시험항목명 "Kubernetes(K8S) 1.17.3" 의 "파드" 수 확인 시험 결과 - 시험 진행 전 "파드" 수 : 1 개 - "network_receive" 값이 2Mbyte/sec 이상 3 회 연속 출력되었을 때 "파드" 수 : 2 개 - "network_receive" 값이 1Mbyte/sec 이하 3 회 연속 출력되었을 때 "파드" 수 : 1 개 - "cpu" 값이 50 %이상, "network_receive" 값이 2Mbyte/sec 이상 3 회 연속 출력되었을 때 "파드" 수 : 2 개 - "cpu" 값이 20 %이하, "network_receive" 값이 1Mbyte/sec 이하 3 회 연속 출력되었을 때 "파드" 수 : 1 개 ※ 본 시험 결과는 의뢰자가 제공한 제품(시험 도구 및 시험데이터 포함)에서 출력된 결과를 기록함 [2021년 KTL 공인 시험 결과] • 위 사진은 network 사용량과 CPU 사용량에 따른 엣지 서비스의 Scale In/Out 수행 결과임 • network_receive량에 따른 엣지 서비스의 Scale In/Out 진행과 network_receive량과 CPU 사용량 조합에 따른 엣지 서비스의 Scale In/Out 진행 결과를 확인 • 엣지 서비스에 대해 CPU 사용률의 임계치 일정 횟수 연속 초과, CPU 사용률의 임계치 일정 횟수 비연속 초과, 메모리 사용률의 임계치 초과,network_receive양 임계치 초과, network_receive양과 CPU 사용률의 동시에 임계치 초과 등 총 5가지 스케줄링 정책 검증으로 최종 목표치(5종)를 달성함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

3. 엣지 앱과 엣지 서비스의 Life Cycle Management API 수	
평가방법	• 정의 : 엣지 플랫폼에서 엣지 앱과 엣지 서비스의 Life cycle 관리를 위해 제공하는 API 수 • 성능 측정 수식 : ∑(엣지 앱 & 엣지 서비스 Life Cycle Management API 수) • 최종 성능 목표치 : 22종 이상 • 성능 목표치 근거 : 유사 관련 데이터가 존재하지 않아 개발사에서 엣지 앱과 엣지 서비스의 Life cycle 관리를 위해 엣지 플랫폼에서 제공해야할 것으로 판단되는 API 수 산정 • 성능 평가 도구 : 공인시험 확인 • 성능 평가 방법 : 최종 성능 목표치 대비 연동 지원수 비교
평가환경	• 개발 클라우드 엣지 플랫폼에서 엣지 앱과 엣지 서비스의 life cycle Management 관련해서 지원하는 API 수 측정

공인시험결과	23) 엣지 플랫폼 namespace 내 특정 resourceQuota 정보 조회
	URL : http://210.217.178.182:30093/api/k8s/clusters/1/namespaces/test-ns/resource-quota/rq-1
	Method : GET
	ResponseBody :
	{
	"api_version": "v1",
	"kind": "ResourceQuota",
	"metadata": {
	"creation_timestamp": "2021-12-22 05:12:37",
	"name": "rq-1",
	"namespace": "test-ns",
	"resource_version": "4878753",
	"self_link": "/api/v1/namespaces/test-ns/resourcequotas/rq-1",
	"uid": "26fb5b47-bdc9-4809-a527-b1363d804c0f"
	},
	"spec": {
	"hard": {
	"limits.cpu": "2",
	"limits.memory": "2G",
	"requests.cpu": "1",
	"requests.memory": "1G"
	}
	},
	"status": {
	"hard": {
	"limits.cpu": "2",
	"limits.memory": "2G",
	"requests.cpu": "1",
	"requests.memory": "1G"
	},
	"used": {
	"limits.cpu": "0",
	"limits.memory": "0",
	"requests.cpu": "0",
	"requests.memory": "0"
	}
	}
	}
	[Resource Quota 정보 조회 API 호출 결과]
	6.3 시험항목 3
	시험항목명 | "api-test.sh" 스크립트 실행 후 출력문 확인
	시험결과 | * 하단 칸 참조
	확인 내용
	가상 서버 9에서 "./api-test.sh" 명령문 실행 시 출력문 확인
	결과
	==========
	1) 클러스터 조회
	URL : http://210.217.178.182:30093/api/k8s/clusters?page=1&size=10&sort=DESC
	Method : GET
	ResponseBody :
	[
	{
	"id": 1,
	"name": "cluster1"
	},
	{
	"id": 2,
	"name": "cluster2"
	}
	]
	[KTL 공인 시험 결과]

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	• 위 사진은 엣지 앱과 엣지 서비스의 life cycle Management 관련해서 지원하는 API 호출 수행 결과임 • 클라우드 엣지 플랫폼에서 엣지 앱과 엣지 서비스 life cycle Management 관련한 API 총 25개를 호출하여 정상 동작을 확인함 • 엣지 앱과 엣지 서비스의 life cycle management 관련 API 총 25개를 개발하여 당해연도 목표치(22종 이상) 달성 – 클러스터 조회 – 엣지 플랫폼 Node 동기화 – 엣지 플랫폼 Node 전체 조회 – 엣지 서비스 인스턴스 전체 정보 조회 – 엣지 서비스 인스턴스 상세 조회 – 엣지 서비스 인스턴스 상태 조회 – 엣지 서비스 인스턴스 생성 – 엣지 서비스 인스턴스 기동 – 엣지 서비스 인스턴스 업그레이드 – 엣지 서비스 인스턴스 Scale Out – 엣지 서비스 인스턴스 중지 – 엣지 서비스 인스턴스 삭제 – 엣지 서비스 인스턴스 재시작 – 엣지 플랫폼 namespace 생성 – 엣지 플랫폼 namespace 전체 조회 – 엣지 플랫폼 namespace 정보 조회 – 엣지 플랫폼 LimitRange 생성 – 엣지 플랫폼 namespace 내 LimitRange 목록 조회 – 엣지 플랫폼 namespace 내 특정 LimitRange 정보 조회 – 엣지 플랫폼 LimitRange 삭제 – 엣지 플랫폼 namespace내 Resource Qutoa 생성 – 엣지 플랫폼 namespace 내 Resource Quota 목록 조회 – 엣지 플랫폼 namespace 내 특정 ResourceQuota 정보 조회 – 엣지 플랫폼 namespace 내 ResourceQuota 삭제 – 엣지 플랫폼 namespace 삭제

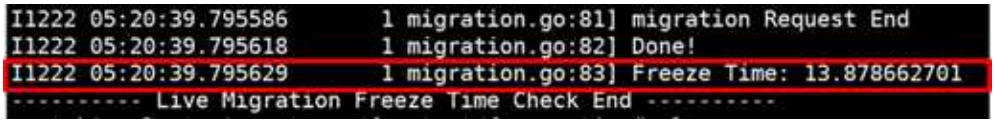
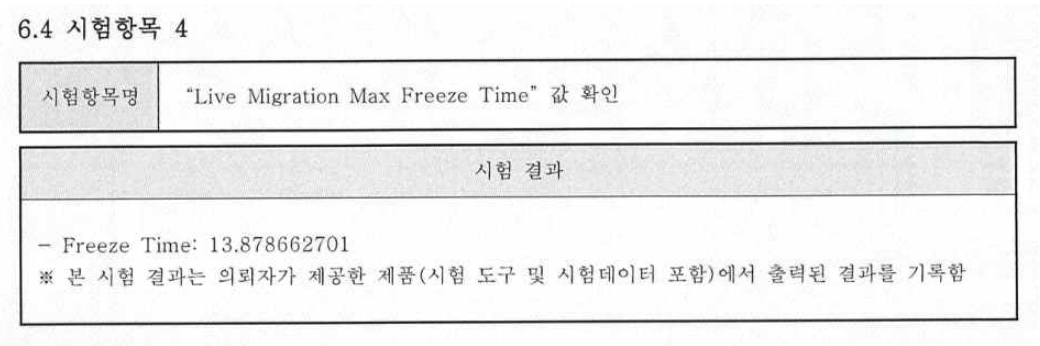
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 클라우드 엣지 플랫폼 정보보호 및 보안인증 지원 항목 수	
평가방법	• 정의 : KISA의 클라우드 컴퓨팅 정보 보호 기준 항목 중 서비스 보안을 위해 엣지 플랫폼에서 지원하는 항목의 수 • 성능 측정 수식 : ∑(엣지 플랫폼에서 지원하는 클라우드 보안 인증 항목 수) • 최종 성능 목표치 : 30종 이상 • 성능 목표치 근거 : 클라우드 SaaS 서비스에서 서비스 보안을 위해 지원되어야 하는 KISA의 클라우드 컴퓨팅 정보 보호 기준 항목 수 산정 • 성능 평가 도구 : 자체 시험 확인 • 성능 평가 방법 : 최종 적용된 보안 인증 항목 수 비교
평가환경	• 개발 클라우드 엣지 플랫폼에서 지원하는 KISA 클라우드 컴퓨팅 정보 보호 기준 항목 수 측정
자체시험결과	 [엣지 플랫폼 관리 포털 2단계 로그인]  [엣지 플랫폼 관리 포털 Https를 통한 접근] • 각 항목별로 보호 조치 방법 적용 여부 확인을 통해 자체 검증 • KISA의 클라우드 컴퓨팅 정보 보호 기준 항목 중 총 30종을 적용하여 목표치(30종 이상) 달성 ▪ 9.1.1 가상자원(가상머신, 가상 스토리지, 가상 소프트웨어 등)의 생성, 변경, 회수 등에 대한 관리 방안을 수립하여야 한다. ▪ 9.1.5. 공개서버 보안 : 가상자원을 제공하기 위한 웹사이트와 가상소프트웨어(앱, 응용프로그램)를 배포하기 위한 공개서버에 대한 물리적, 기술적 보호대책을 수립하여야 한다. ▪ 9.2.2 인터페이스 및 API 보안 : 가상 환경(가상PC, 가상 서버, 가상 소프트웨어 등)접근을 위한 인터페이스 및 API에 대한 보안 취약점을 주기적으로 분석하고, 이에 대한 보호방안을 마련하여야 한다. ▪ 9.2.3 데이터 이전 : 이용자가 기존 정보시스템 환경에서 클라우드 컴퓨팅 서비스의 가상환경으로 전환 시 안전하게 데이터를 이전하도록 암호화 등의 기술적인 조치 방안을 제공하여야 한다. ▪ 9.2.4 가상소프트웨어 보안 : 클라우드 컴퓨팅 서비스 제공자는 출처, 유통경로 및 제작자가 명확한 소프트웨어로 구성된 가상환경을 제공하여야 한다. ▪ 10.1.1 비인가자의 접근을 통제할 수 있도록 접근 통제 영역 및 범위, 접근통제 규칙, 방법 등을 포함하여 접근 통제 정책을 수립하여야 한다. ▪ 10.1.2 접근 기록 대상을 정의하고 서비스 통제, 관리, 사고 발생 책임 추적성 등을 보장할 수 있는 형태로 기록되고 유지하여야 한다. ▪ 10.2.1 클라우드 시스템 및 중요 정보에 대한 접근을 통제하기 위하여 공식적인 사용자 등록 및 해지 절차를 수립하고 업무 필요성에 따라 사용자 접근 권한을 최소한으로 부여하여야 한다. ▪ 10.2.2 관리자 및 특수 권한 관리 : 클라우드 시스템 및 중요정보 관리 및 특수 목적을 위해 부여한 계정 및 권한을 식별하고 별도 통제하여야 한다. ▪ 10.2.3 접근 권한 검토 ▪ 클라우드 시스템 및 중요 정보에 대한 접근을 관리하기 위하여 접근 권한 부여, 이용(장기간 미사용), 변경(퇴직 및 휴직, 직무 변경, 부서 변경)의 적정성 여부를 정기적으로 점검하여야 한다. ▪ 10.3.1 클라우드 시스템에서 사용자를 유일하게 구분할 수 있는 식별자를 할당하고 추측 가능한 식별자 사용을 제한하여야 한다. ▪ 10.3.2 클라우드 시스템에 대한 접근은 사용자 인증, 로그인 횟수 제한, 불법 로그인 시도 경고 등 안전한 사용자 인증 절차에 의해 통제하여야 한다. ▪ 10.3.3 이용자가 클라우드 컴퓨팅 서비스에 대해 다중 요소 인증 등 강화된 인증 수단을 요청하는 경우 이를 제공하기 위한 방안을 마련하여야 한다. ▪ 10.3.4 법적 요구사항, 외부 위협 요인 등을 고려하여 패스워드 복잡 기준, 초기 패스워드 변경,

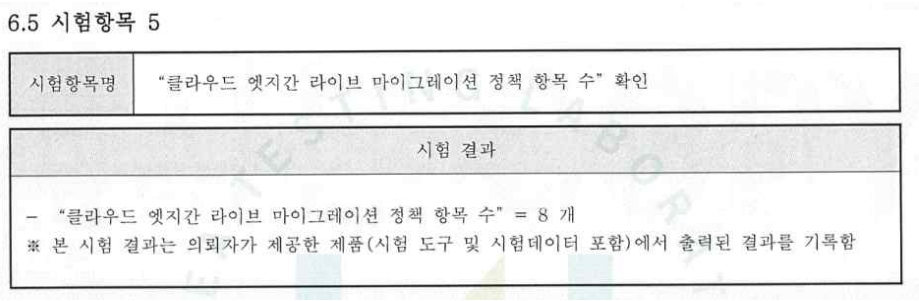
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	변경 주기 등 사용자 패스워드 관리 절차를 수립.이행하고 패스워드 관리 책임이 사용자에게 있음을 주지시켜야한다. ▪ 10.3.5 고객, 회원 등 외부 이용자가 접근하는 클라우드 시스템 또는 웹 서비스의 안전한 이용을 위하여 계정 및 패스워드 등의 관리 절차를 마련하고 관련 내용을 공지하여야 한다. ▪ 11.1.1 네트워크 보안 정책 수립 : 클라우드 컴퓨팅 서비스와 관련된 내.외부 네트워크에 대한 보안 정책과 절차를 수립하여야한다. ▪ 11.1.2 네트워크 모니터링 및 통제 : DDos, 비인가 접속 등으로 인한 서비스 중단 및 중요 정보 유출 등을 막기 위해 네트워크를 모니터링하고 통제하여야한다. ▪ 11.1.3 네트워크 정보보호 시스템 운영 : 클라우드 컴퓨팅 서비스와 관련된 내.외부 네트워크를 보호하기 위하여 정보보호시스템(방화벽, IPS, IDS, VPN 등을) 운영하여야 한다. ▪ 11.1.4 클라우드 시스템에서 중요 정보가 이동하는 구간에 대해서는 암호화된 통신 채널을 사용하여야 한다. ▪ 11.1.5 클라우드 컴퓨팅 서비스 제공자의 관리 영역과 이용자의 서비스 영역, 이용자가 서비스 영역의 네트워크 접근은 물리적 또는 논리적으로 분리하여야 한다. ▪ 12.1.2 이용자와 서비스 수준 협약 단계에서 데이터의 소유권을 명확하게 확립하여야 한다. ▪ 12.3.2 암호키 관리 : 암호키 생성, 이용, 보관, 배포, 파기에 관한 안전한 절차를 수립하고, 암호키는 별도의 안전한 장소에 보관하여야 한다. ▪ 13.1.3 클라우드 시스템 설계 시 사용자의 인증, 권한 변경, 중요 정보 이용 및 유출 등에 대한 감사 증적을 확보할 수 있도록 하여야 한다. ▪ 13.1.4 클라우드 시스템 설계시 업무의 목적 및 중요도에 따라 접근 권한을 부여할 수 있도록 하여야 한다. ▪ 13.1.5 로그기록의 정확성을 보장하고 법적인 자료로서 효력을 지니기 위해 클라우드 시스템 시각을 공식 표준시각으로 정확하게 동기화하여야 한다. ▪ 13.2.1 구현 및 시험 : 안전한 코딩 방법에 따라 클라우드 시스템을 구현하고 , 분석 및 설계 과정에서 도출한 보안요구사항이 정보시스템에 적용되었는지 확인하기 위하여 시험을 수행하여야 한다. ▪ 13.2.2 개발 및 시험 시스템은 운영 시스템에 대한 비 인가 접근 및 변경의 위험을 감소하기 위해 원칙적으로 분리하여야 한다. 단, 분리하여 운영하기 어려운 경우 그 사유와 타당성을 검토하고 안정성 확보 방안을 마련하여야 한다. ▪ 13.2.3 시험데이터 보안 : 시스템 시험 과정에서 운영 데이터 유출을 예방하기 위해 시험데이터 생성, 이용 및 관리, 파기, 기술적 보호조치에 관한 절차를 수립하여 이행하여야한다. ▪ 13.2.4 소스 프로그램에 대한 변경 관리를 수행하고 인가된 사용자만이 소스 프로그램에 접근할 수 있도록 통제 절차를 수립하여 이행하여야 한다. ▪ 13.3.1 외주 개발 보안 : 클라우드 시스템 개발을 외주 위탁하는 경우 분석 및 설계 단계에서 구현 및 이관까지의 준수해야할 보안 요구사항을 계약서에 명시하고 이행여부를 관리,감독하여야 한다.

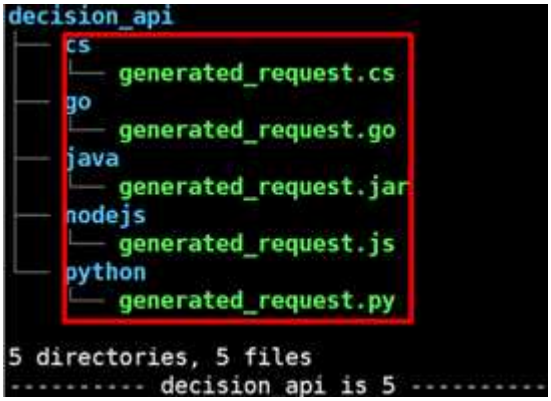
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

5. 마이그레이션 수행시 최대 Freeze Time	
평가방법	• 성능 측정 수식 : - • 최종 성능 목표치 : 19 • 성능 목표치 근거 : ReadHat Container Migration 의 최대 22초 지원 (https://rhelblog.redhat.com/2017/10/12/container-migration-around-the-world/) • 성능 평가 도구 : 공인시험 확인 (마이그레이션 툴을 통한 측정 시간 표시) • 성능 평가 방법 : 최종 성능 목표치 대비 성능 비교 → 공인(자체) 시험 시 개발 클라우드 엣지 플랫폼 환경에서 출발지에서의 라이브 마이그레이션 시작 시점과 도착지에서의 라이브 마이그레이션 완료까지의 Freeze Time을 카운터 출력값의 비교를 통한 성능 측정 표시 혹은 타이머를 통한 성능 측정 표시 (최종 정략적 목표 19초 이내 만족 여부 확인)
평가환경	• 라이브마이그레이션 수행 시 최대 Freeze Time 시간 측정
공인시험결과	 [Migration 수행 시 최대 Freeze Time 측정 결과]  [KTL 공인 시험 결과] • 위 사진은 Cluster1에서 Cluster2로의 라이브 마이그레이션 수행 결과임 • 자체 시험 도구(live-migration-max-freeze-time-checking.sh)를 통해 최대 Freeze Time 시간 측정 • Freeze Time 13.878662701으로 당해연도 목표치(<19 sec) 달성

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

6. 클라우드 엣지 간 라이브 마이그레이션 정책 항목 수	
평가방법	• 정의 : 클라우드 엣지 간 라이브 마이그레이션 수행을 시 자원간 이동을 위한 학습형 정책 제공 수 • 성능 측정 수식 : ∑(라이브 마이그레이션 정책 수) • 최종 성능 목표치 : 8종 이상 • 성능 목표치 근거 : Google Cloud 의 2종 지원 (https://cloud.google.com/compute/docs/instances/setting-instance-scheduling-options) • 성능 평가 도구 : 공인시험 확인 • 성능 평가 방법 : 최종 성능 목표치 대비 연동 지원수 비교
평가환경	• 개발 클라우드 엣지 플랫폼 기반 학습형 라이브 마이그레이션 정책 항목 수 측정
공인시험결과	 [Pod 정책 조회 결과]  [KTL 공인 시험 결과] ◦ **Pod 정책 8종 지원** ▪ Anything : True면 Pod의 정책 중 어떤 것도 포함되지 않음, False면 포함 ▪ CriticalPod : True면 Pod의 마이그레이션 시 위험성을 고려해 마이그레이션 리스트에서 제외, False면 Pod의 마이그레이션 수행 가능 ▪ GeoMode : 마이그레이션 시 자원 사용량과 거리에 따른 가중치로 마이그레이션 할 수 있게 되는 정책 ▪ IsSystemPod : 마이그레이션을 위한 모듈 또는 Kube-System 네임스페이스에 있는 Pod는 Migration을 실행하면 안되기에 해당 정책 명시 ▪ MigrationAlgorithm : 마이그레이션 알고리즘 설정을 위한 정책 ▪ MigrationPod : Pod와 MigrationPod의 정보를 탐색하는 Client가 다르기 때문에 해당 Policy는 변경할 수 없으며, 이를 바탕으로 Migration 대상 Pod의 정보 탐색 ▪ MigrationProperty : 마이그레이션 시 Pod의 자원 사용 특성을 고려하기 위한 정책 ▪ ServiceType : 서비스의 종류 및 특성을 고려하기 위한 정책 ◦ 당해 년도 정책 항목수 (8개) 만족

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<table>
<tr><th colspan="2">7. 워크로드에 따른 Live Migration Decision API 수</th></tr>
<tr><td>평가방법</td><td>• 정의 : 클라우드 엣지 플랫폼 환경에서 지원 가능한 Decision API 지원의 수
• 성능 측정 수식 : ∑(클라우드 엣지 플랫폼 환경에서 연동 지원 Decision API)
• 최종 성능 목표치 : 5종
• 성능 목표치 근거 : AWS Lambda 4종 지원(Java, Node.js, C#, Python) (https://aws.amazon.com/ko/lambda/features/)
• 성능 평가 도구 : 공인시험 확인
• 성능 평가 방법 : 최종 성능 목표치 대비 연동 지원수 비교</td></tr>
<tr><td>평가환경</td><td>• 개발 클라우드 엣지 플랫폼 기반 Live Migration Decision API 항목 수 측정</td></tr>
<tr><td>공인시험결과</td><td> 

[Live Migration Decision API 목록]
6.6 시험항목 6
시험항목명 | “워크로드에 따른 Live Migration Decision API 수” 확인
시험 결과
– “워크로드에 따른 Live Migration Decision API 수” = 5 개
※ 본 시험 결과는 의뢰자가 제공한 제품(시험 도구 및 시험데이터 포함)에서 출력된 결과를 기록함
[KTL 공인 시험 결과]
• Go, Python, Java, Nodejs, C# 언어로 Migration 요청 가능
• 당해 년도 Decision API 수 (5개) 만족</td></tr>
</table>

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(3) 세부 정량적 연구개발성과(해당되는 항목만 선택하여 작성하되, 증빙자료를 별도 첨부해야 합니다)

[과학적 성과]

□ 논문(국내외 전문 학술지) 게재

번호	논문명	학술지명	주저자명	호	국명	발행기관	SCIE 여부 (SCIE/비SCIE)	게재일	등록번호 (ISSN)	기여율
1	엣지 컴퓨팅 기반 서비스 자원 분석을 통한 라이브 마이그레이션 설계 및 구현	2019 한국인터넷정보학회 추계학술발표	한재승,안재훈	-	대한민국	한국 인터넷정보학회	비SCI	2019.11.01		
2	클라우드 엣지 플랫폼 기반 마이그레이션 성능을 위한 캐싱 기법 설계 및 구현	다중 요소 기반 마이크로 데이터센터 워크로드에 따른 전력 사용량 예측 향상	허태혁 안재훈 김영환	-	대한민국	한국차세대컴퓨팅학회	비SCI	2020.08.20		
3	Design and Implementation of Migration Manager between Cloud Edge Platforms	2020 Research in Adaptive and Convergent Systems (RACS 2020)	Tadhyuk Heo, Jae-Hoon An, Younghwan Kim	-	Korea	RACS 2020	비SCI	2020.10.13		
4	대규모 클라우드 서비스 워크로드 분석을 위한 메트릭 데이터 예측 기법	2021 한국컴퓨터종합학술대회 (KCC2021)	허태혁안재훈 김영환	-	대한민국	한국정보과학회	비SCI	2021.08.20		
5	클라우드 엣지 환경에서 마이그레이션 결정 과정에서의 경고 리스트 구조 설계	대한전기학회 CICS'21 정보 및 제어 학술대회	윤제병 안재훈 김영환	-	대한민국	대한전기학회	비SCI	2021.10.20		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 국내 및 국제 학술회의 발표

번호	회의 명칭	발표자	발표 일시	장소	국명
1	한국정보과학회 KCC2021 (대규모 클라우드 서비스 워크로드 분석을 위한 메트릭 데이터 예측 기법)	허태혁	2021.06.23	ICC 제주 & 씨에스호텔 & 온라인	대한민국
2	CICS`21 정보 및 제어 학술대회 (클라우드 엣지 환경에서 마이그레이션 결정 과정에서의 경고 리스트 구조 설계)	윤제병	2021.10.20	베네치아 호텔&리조트(여수)	대한민국

□ 기술 요약 정보

연도	기술명	요약 내용	기술 완성도	등록 번호	활용 여부	미활용사유	연구개발기관 외 활용여부	허용방식

□ 보고서 원문

연도	보고서 구분	발간일	등록 번호
2019	연차보고서	2019.11.29	
2020	연차보고서	2020.11.27	
2021	최종보고서	2022.03.01	

□ 생명자원(생물자원, 생명정보)/화합물

번호	생명자원(생물자원, 생명정보)/화합물 명	등록/기탁 번호	등록/기탁 기관	발생 연도

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[기술적 성과]

□ 지식재산권(특허, 실용신안, 의장, 디자인, 상표, 규격, 신품종, 프로그램)

번호	지식재산권 등 명칭 (건별 각각 기재)	국명	출원				등록			기여율	활용 여부
			출원인	출원일	출원 번호	등록 번호	등록인	등록일	등록 번호		
1	클라우드 엣지 시스템 및 그 제어방법	대한민국	스트라토	2019.11.04	10-2019-0139745					100	
2	클라우드 엣지 환경에서 예측 기반 마이그레이션 후보 및 대상 선정 방법	대한민국	한국전자기술연구원	2019.11.11	10-2019-0143369		한국전자기술연구원	2021.10.27	10-2320317-0000	100	
3	엣지 서비스 제어 처리 시스템 및 그 제어방법	대한민국	스트라토	2020.10.27	10-2020-0139984					100	√
4	엣지 서비스 증설 제어 시스템 및 그 제어방법	대한민국	스트라토	2020.11.25	10-2020-0159828					100	√
5	클라우드 엣지 내 워크로드 분산을 위한 예측 모델 적용 방법	대한민국	한국전자기술연구원	2020.09.11	10-2020-0116922		한국전자기술연구원	2021.04.22	10-2245341-0000	100	
6	클라우드 엣지 환경에서 컨테이너의 메모리 라이브 마이그레이션 처리 방법	대한민국	한국전자기술연구원	2020.09.11	10-2020-0116925		한국전자기술연구원	2021.04.22	10-2245340-0000	100	
7	엣지 서비스 관리 장치 및 그 제어방법	대한민국	스트라토	2021.12.29	10-2021-0190569					100	
8	엣지 서비스 인스턴스 배포 장치 및 그 제어방법	대한민국	스트라토	2021.10.12	10-2021-0135276					100	
9	클라우드 엣지 플랫폼 적용에 따른 마이그레이션 우선 대상 선정 방법	대한민국	한국전자기술연구원	2021.11.12	10-2021-0155566					100	

ㅇ 지식재산권 활용 유형

※ 활용의 경우 현재 활용 유형에 √ 표시, 미활용의 경우 향후 활용 예정 유형에 √ 표시합니다(최대 3개 중복선택 가능).

번호	제품화	방어	전용실시	통상실시	무상실시	매매/양도	상호실시	담보대출	투자	기타
3	√	√								
4	√	√								

□ 저작권(소프트웨어, 서적 등)

번호	저작권명	창작일	저작자명	등록일	등록 번호	저작권자명	기여율
1	Edge Computing API Gateway	2019.11.11	스트라토	2019.11.14	C-2019-033501	스트라토	100
2	Edge Compute LifeCycle Manager	2019.11.11	스트라토	2019.11.14	C-2019-033503	스트라토	100
3	Edge Compute	2019.11.11	스트라토	2019.11.14	C-2019-033504	스트라토	100

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	Resource Manager						
4	Edge Compute Auth Server	2019.11.11	스트라토	2019.11.14	C-2019-033502	스트라토	100
5	클라우드 엣지 환경에서의 IoT Gateway 인터페이스 처리 모듈	2019.11.08	한국전자 기술연구원	2019.11.13	C-2019-033399	한국전자 기술연구원	100
6	클라우드 엣지 환경에서의 마이그레이션 결정 처리 모듈	2019.11.08	한국전자 기술연구원	2019.11.13	C-2019-033400	한국전자 기술연구원	100
7	엣지 컴퓨트 쿠버네티스 어댑터	2020.11.04	스트라토	2020.11.17	C-2020-042391	스트라토	100
8	엣지 컴퓨트 모니터링	2020.11.04	스트라토	2020.11.17	C-2020-042392	스트라토	100
9	엣지 컴퓨트 엣지 포탈	2020.11.04	스트라토	2020.11.17	C-2020-042390	스트라토	100
10	클라우드엣지 기반 다중 매트릭 기반 부하 예측 처리 모듈	2020.09.22	한국전자 기술연구원	2020.09.29	C-2020-033817	한국전자 기술연구원	100
11	클라우드엣지 환경에서의 고속 라이브 마이그레이션 처리 모듈	2020.09.22	한국전자 기술연구원	2020.09.29	C-2020-033818	한국전자 기술연구원	100
12	클라우드엣지 환경에서의 마이그레이션 커맨드 라인 인터페이스 모듈	2020.09.22	한국전자 기술연구원	2020.09.29	C-2020-033819	한국전자 기술연구원	100
13	클라우드엣지 환경에서의 정책 배포 및 관리 모듈	2021.05.20	한국전자 기술연구원	2021.05.28	C-2021-021891	한국전자 기술연구원	100
14	클라우드엣지 기반 IoT플랫폼 연동 모듈	2021.05.20	한국전자 기술연구원	2021.05.28	C-2021-021889	한국전자 기술연구원	100
15	클라우드엣지 환경에서의 마이그레이션을 위한 오토커밋(Auto-Commit) 모듈	2021.05.20	한국전자 기술연구원	2021.05.28	C-2021-021890	한국전자 기술연구원	100
16	Edge Compute Event Generator(엣지 컴퓨트 이벤트 제네레이터)	2021.12.21	스트라토	2021.12.31	C-2021-060283	스트라토	100
17	Edge Compute Policy Manager(엣지 컴퓨트 폴리시 매니저)	2021.12.21	스트라토	2021.12.31	C-2021-060284	스트라토	100
18	Edge Compute Registry(엣지 컴퓨트 레지스트리)	2021.12.21	스트라토	2021.12.31	C-2021-060285	스트라토	100

□ 신기술 지정

번호	명칭	출원일	고시일	보호 기간	지정 번호

□ 기술 및 제품 인증

번호	인증 분야	인증 기관	인증 내용		인증 획득일	국가명
			인증명	인증 번호		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 표준화

ㅇ 국내표준

번호	인증구분[1]」	인증여부[2]」	표준명	표준인증기구명	제안주체	표준종류[3]」	제안/인증일자
1	단체규격	제안	클라우드 엣지 플랫폼 참조모델	TTA	스트라토,한국전자기술연구원		2019.11.12
2	단체규격	제안	5G 네트워크에서의 MEC(기술보고서)	TTA	스트라토,한국전자기술연구원		2019.11.12
3	단체규격	제정	API Driven 마이크로서비스 기반 클라우드 엣지 SW 아키텍처 참조모델	ICTF 지능형 컴퓨팅 기술포럼	스트라토,한국전자기술연구원	신규	2020.11.25
4	단체규격	제정	5G 네트워크에서의 MEC	ICTF 지능형 컴퓨팅 기술포럼	스트라토,한국전자기술연구원	신규	2021.12.01
5	단체규격	제정	API Driven 마이크로서비스 기반 클라우드 엣지 SW 아키텍처 참조모델	ICTF 지능형 컴퓨팅 기술포럼	스트라토,한국전자기술연구원	개정	2021.12.01

* 1」 한국산업규격(KS) 표준, 단체규격 등에서 해당하는 사항을 기재합니다.
* 2」 제안 또는 인증 중 해당하는 사항을 기재합니다.
* 3」 신규 또는 개정 중 해당하는 사항을 기재합니다.

ㅇ 국제표준

번호	표준화단계구분[1]」	표준명	표준기구명[2]」	표준분과명	의장단 활동여부	표준특허 추진여부	표준개발 방식[3]」	제안자	표준화 번호	제안일자

* 1」 국제표준 단계 중 신규 작업항목 제안(NP), 국제표준초안(WD), 위원회안(CD), 국제표준안(DIS), 최종국제표준안(FDIS), 국제표준(IS) 중 해당하는 사항을 기재합니다.
* 2」 국제표준화기구(ISO), 국제전기기술위원회(IEC), 공동기술위원회1(JTC1) 중 해당하는 사항을 기재합니다.
* 3」 국제표준(IS), 기술시방서(TS), 기술보고서(TR), 공개활용규격(PAS), 기타 중 해당하는 사항을 기재합니다.

[경제적 성과]

□ 시제품 제작

번호	시제품명	출시/제작일	제작 업체명	설치 장소	이용 분야	사업화 소요 기간	인증기관 (해당 시)	인증일 (해당 시)
1	라우팅 및 트래픽 제어 모듈	2019	스트라토	스트라토 서버실	OpenAPI 서비스			
2	인증 및 API 접근관리 모듈	2019	스트라토	스트라토 서버실	OpenAPI 서비스			
3	Live Migration Decision 모듈	2019	KETI	스트라토 서버실	SDDC 가상서버/컨테이너 운영 관리			
4	LTSM 기반 워크로드 예측 모듈	2019	KETI	스트라토 서버실	SDDC 자원 운영 관리			
5	시각화 연동 모듈	2019	KETI	스트라토 서버실	SDDC 자원 운영 관리			
6	라이프사이클 관리 코어 모듈	2020	스트라토	스트라토 서버실	엣지 서비스/앱 패키지 및 인스턴스 배포 관리			
7	엣지 자원 자동 관리 모듈	2020	스트라토	스트라토 서버실	엣지 서비스/앱 인스턴스 자원			

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

					운영 관리			
8	Migration 프레임워크	2020	KETI	스트라토 서버실	SDDC 가상서버/컨테이너 운영 관리			
9	Service Live Migration 모듈	2020	KETI	스트라토 서버실	SDDC 가상서버/컨테이너 운영 관리			
10	IOT Collector 모듈	2020	KETI	스트라토 서버실	IOT Device 정보 수집 관리			
11	Edge Portal	2021	스트라토	스트라토 서버실	엣지 서비스/앱 패키지 및 인스턴스 배포 운영 관리			
12	Resource Manager 모듈	2021	스트라토	스트라토 서버실	K8S 환경에서의 리소스 관리			
13	IoT 플랫폼 연동 모듈	2021	KETI	스트라토 서버실	IoT 플랫폼 연동 처리			
14	마이그레이션을 위한 오토커밋 모듈	2021	KETI	스트라토 서버실	SDDC 가상서버/컨테이너 운영 관리			

□ 기술 실시(이전)

번호	기술 이전 유형	기술 실시 계약명	기술 실시 대상 기관	기술 실시 발생일	기술료 (해당 연도 발생액)	누적 징수 현황

* 내부 자금, 신용 대출, 담보 대출, 투자 유치, 기타 등

□ 사업화 투자실적

번호	추가 연구개발 투자	설비 투자	기타 투자	합계	투자 자금 성격*
1	데브옵스 자동화 패키지		350,000	350,000	기존 제품 개선
2	OpenAPI 플랫폼		300,000	300,000	기존 제품 개선

□ 사업화 현황

번호	사업화 방식[1」]	사업화 형태[2」]	지역[3」]	사업화명	내용	업체명	매출액		매출 발생 연도	기술 수명
							국내 (천원)	국외 (달러)		
1	자기실시	기존 제품 개선	국내	하나금융그룹 오픈 플랫폼	하나금융 오픈 API 플랫폼 기술	스트라토	193,900		2020	
2	자기실시	기존 제품 개선	국내	KB 국민은행 리브온	마이크로서비스 아키텍처 기술	스트라토	1,901,505		2020	
3	자기실시	기존 제품 개선	국내	신한금융투자 ICT 시스템 현대화 실증사업	마이크로서비스 아키텍처 기술	스트라토	868,181		2021	
4	자기실시	기존 제품 개선	국내	현대차 ZET 서비스 플랫폼 개선	현대차 오픈 API 플랫폼 기술	스트라토	1,315,000		2021	

* 1」 기술이전 또는 자기실시
* 2」 신제품 개발, 기존 제품 개선, 신공정 개발, 기존 공정 개선 등
* 3」 국내 또는 국외

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 매출 실적(누적)

사업화명	발생 연도	매출액		합계	산정 방법
		국내(천원)	국외(달러)		
데브옵스 자동화 패키지 사업	2020	1,901,505	-	1,901,505	
오픈 API 플랫폼 사업	2020	193,900	-	193,900	
데브옵스 자동화 패키지 사업	2021	868,181	-	868,181	
오픈 API 플랫폼 사업	2021	1,315,000	-	1,315,000	
합계		4,278,586	-	4,278,586	

□ 사업화 계획 및 무역 수지 개선 효과

성과					
사업화 계획	사업화 소요기간(년)				
	소요예산(천원)				
	예상 매출규모(천원)		현재까지	3년 후	5년 후
	시장 점유율	단위(%)	현재까지	3년 후	5년 후
		국내			
		국외			
	향후 관련기술, 제품을 응용한 타 모델, 제품 개발계획				
무역 수지 개선 효과(천원)	수입대체(내수)		현재	3년 후	5년 후
	수출				

□ 고용 창출

순번	사업화명	사업화 업체	고용창출 인원(명)		합계
			2020년	2021년	
1	데브옵스 자동화 패키지 사업	스트라토	2	1	3
2	오픈 API 플랫폼 사업	스트라토	1	1	2
합계			3	2	5

□ 고용 효과

구분			고용 효과(명)
고용 효과	개발 전	연구인력	1
		생산인력	0
	개발 후	연구인력	5
		생산인력	0

□ 비용 절감(누적)

순번	사업화명	발생연도	산정 방법	비용 절감액(천원)
합계				

□ 경제적 파급 효과

(단위: 천원/년)

구분	사업화명	수입 대체	수출 증대	매출 증대	생산성 향상	고용 창출 (인력 양성 수)	기타
해당 연도							
기대 목표							

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 산업 지원(기술지도)

순번	내용	기간	참석 대상	장소	인원

□ 기술 무역

(단위: 천원)

번호	계약 연월	계약 기술명	계약 업체명	계약업체 국가	기 징수액	총 계약액	해당 연도 징수액	향후 예정액	수출/ 수입

[사회적 성과]

□ 법령 반영

번호	구분 (법률/시행령)	활용 구분 (제정/개정)	명 칭	해당 조항	시행일	관리 부처	제정/개정 내용

□ 정책활용 내용

번호	구분 (제안/채택)	정책명	관련 기관 (담당 부서)	활용 연도	채택 내용

□ 설계 기준/설명서(시방서)/지침/안내서에 반영

번호	구 분 (설계 기준/설명서/지침/안내서)	활용 구분 (신규/개선)	설계 기준/설명서/ 지침/안내서 명칭	반영일	반영 내용

□ 전문 연구 인력 양성

번호	분류	기준 연도	현황										
			학위별				성별		지역별				
			박사	석사	학사	기타	남	여	수도권	충청권	영남권	호남권	기타

□ 산업 기술 인력 양성

번호	프로그램명	프로그램 내용	교육 기관	교육 개최 횟수	총 교육 시간	총 교육 인원

□ 다른 국가연구개발사업에의 활용

번호	중앙행정기관명	사업명	연구개발과제명	연구책임자	연구개발비

□ 국제화 협력성과

번호	구분 (유치/파견)	기간	국가	학위	전공	내용

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

□ 홍보 실적

번호	홍보 유형	매체명	제목	홍보일
1	신문기사	전자신문	미래 먹거리 '클라우드 에지 플랫폼' 우리손으로 만든다	2019.9.28
2	제품 전시	2020 ICT 기기산업 페스티벌(코엑스 전시장)	클라우드 엣지 플랫폼 기술 개발	2020.12.16
3	제품 전시	2021 ICT 기술사업화 페스티벌(코엑스 전시장)	엣지 마이크로서비스 관리 플랫폼	2021.4.21
4	온라인 브러셔	CCCR TaB IR	마이크로서비스 기반의 클라우드 엣지 플랫폼	2022.02

□ 포상 및 수상 실적

번호	종류	포상명	포상 내용	포상 대상	포상일	포상 기관

[인프라 성과]

□ 연구시설・장비

구축기관	연구시설/ 연구장비명	규격 (모델명)	개발여부 (○/×)	연구시설・장비 종합정보시스템* 등록여부	연구시설・장비 종합정보시스템* 등록번호	구축일자 (YY.MM.DD)	구축비용 (천원)	비고 (설치 장소)

* 「과학기술기본법 시행령」 제42조제4항제2호에 따른 연구시설・장비 종합정보시스템을 의미합니다.

[그 밖의 성과](해당 시 작성합니다)

해당 사항 없음

(4) 계획하지 않은 성과 및 관련 분야 기여사항(해당 시 작성합니다)

해당 사항 없음

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2) 목표 달성 수준

추 진 목 표	달 성 내 용	달성도 (%)
마이크로서비스 기반의 엣지 서비스 관리 기술 개발	• 엣지 환경/특성에 따른 마이크로서비스 기반의 엣지 서비스 조합과 연결 기술 개발 • 엣지 앱의 API 요청에 따른 트래픽 조절과 결함 격리 기술 개발 • 엣지 앱과 엣지 플랫폼 구간 오픈 연동 기술 개발 • 엣지 서비스 API 단위의 권한/허가 및 보안 인증 기술 개발 • 실사용량 측정을 위한 API 단위 호출 및 로깅 기반 미터링 기술 개발	100
클라우드 엣지 플랫폼 라이프 사이클 관리 기술 개발	• 패키지 및 인스턴스 배포/실행을 위한 라이프 사이클 관리 모델 설계 및 적용 • 엣지 앱 배포/실행 관리를 위한 네트워크 서비스 디스크립터 기술 개발 • 엣지 플랫폼과 엣지 플랫폼 관리 구간 연동 기술 개발 • Region 및 엣지 플랫폼 간 이동을 위한 엣지 호스트 선택 기술 개발 • 엣지 앱 및 서비스 라이프 사이클 관리를 위한 운용자 및 사용자 포탈 개발	100
고확장성 미 고가용성 제공을 위한 클라우드 엣지 자원 자동 관리 기술 개발	• 자동 스케줄링을 위한 자원 워크로드 및 사용량 이벤트 수집 기술 개발 • API 실행을 위한 엣지 서비스 인스턴스 자원 할당 및 스케줄링 기술 개발 • 엣지 앱, 엣지 서비스의 부하 기반 자원 용량 프로비져닝 및 자동 조정 기술 개발	100
Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발	• 마이크로서비스 기반 Zone In/Out 여부 판단 기술 개발 • 마이크로서비스 기반 Zone 이탈 여부 정보 전달 기술 개발 • Zone 이동 시, 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술 개발	100
클라우드 엣지 컨테이너 기반 IoT 응용 서비스 설계 및 프로토타입 개발	• 대규모 IoT 디바이스 데이터 수집 서비스 응용 클러스터 기술 개발 • 컨테이너기반 OneM2M(Mobius 플래폼 or WSO2)기반 IoT 플랫폼 연동 기술 개발 • 클라우드 엣지 컨테이너기반 IoT 응용 서비스 시각화 기술 개발	100
전체 통합 및 Edge 플랫폼 환경을 활용한 Smart Helper 시범 적용	• Gitlab을 활용한 소스 통합 • K8S 기반의 클라우드 엣지 플랫폼 환경 구축 및 해당 환경에서 최종 개발 결과물에 대한 공인 인증 시험 수행 • 상용 서비스인 Smart Helper를 대상으로 한 파일럿 검증 진행	100

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 목표 미달 또는 미흡한 사항에 대한 분석

1) 목표 미달 또는 미흡한 연구결과에 대한 원인(사유) 자체분석 내용

2) 미달 또는 미흡한 결과물에 대한 보완 활동

3) 연구개발 과정의 성실성

5. 연구개발성과의 관련 분야에 대한 기여 정도

▢ 산업 환경에서 클라우드 엣지 활용 확장을 지원하기 위한 클라우드 엣지 플랫폼의 구조 및 구성 요소 관련 기술의 국내 표준 기고 및 제정

○ 정보통신단체표준(TTA) 표준화 제안서 제출 2건

- “5G 네트워크에서의 MEC(기술보고서)” 기고서 제출(‘19.11.12, 제안번호 : 2019-P1424)
- “클라우드 엣지 플랫폼 참조 모델” 기고서 제출(’19.1.12, 제안번호 : 2019-P-1425)

○ 지능형 컴퓨팅 기술 포럼 표준안 제출 및 제정 3건

- “API Driven 마이크로 서비스 기반 클라우드 엣지 SW 아키텍처 참조모델”기술 표준 제안 및 제정 (‘20.11.25, 표준번호 : ICTF.KO-1020))
- “5G 네트워크에서의 MEC” 기술 표준 제안 및 제정(’21.12.01, 표준번호: ICTFR.KO-1024)
- “API Driven 마이크로 서비스 기반 클라우드 엣지 SW 아키텍처 참조모델”기술 표준 개정안 제출 및 제정 (‘21.12.01, 표준번호 : ICTF.KO-1020-R1))

▢ Region 및 엣지 플랫폼 간 서비스 라이브 마이그레이션 기술

○ State-Full 및 Stateless 기반으로 마이그레이션 시 서비스가 멈추는 Freeze Time을 최소화하였고 Pod의 마이그레이션 정책 기반으로 Copy 알고리즘을 선정 가능하며, 자체 Custom Kubelet의 CRU를 통해 Migration Pod 생성 및 관리가 가능함

▢ 클라우드 엣지 자원 사용률 기반 워크로드 예측 기술

○ 예측 모델 생성 시 LSTM 알고리즘을 활용하여 예측모델을 생성하고 예측을 할 때 Node와 Pod의 Running State를 확인하고 진행하여 불필요한 예측 진행을 방지함. 생성된 모델을 기반으로 워크로드 예측을 통해 분석을 한 후 과부하 대상을 결정하고 목적지를 선정함

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

6. 연구개발성과의 관리 및 활용 계획

< 연구개발성과 활용계획표 >

구분(정량 및 정성적 성과 항목)		연구개발 종료 후 5년 이내
국외논문	SCIE	매년 목표치
	비SCIE	-
	계	-
국내논문	SCIE	-
	비SCIE	-
	계	-
특허출원	국내	1
	국외	-
	계	1
특허등록	국내	-
	국외	-
	계	-
인력양성	학사	-
	석사	-
	박사	-
	계	-
사업화	상품출시	-
	기술이전	-
	공정개발	-
제품개발	시제품개발	-
국제 표준화	기고서 제출	-
	기고서 채택	-
	표준안 채택	-
국내 표준화	기고서 제출	-
	기고서 채택	-
	표준안 채택	-
성과홍보		-
포상 및 수상실적		-
정성적 성과 주요 내용		• 클라우드 엣지 플랫폼 핵심 기술 확보 • 외산 서비스 수입 대체로 국내 시장 진입 • 글로벌 시장 진출을 위한 기술 경쟁력 확보 • 지능 정보, 스마트 팩토리등과 연계하여 국민 편의성 제공

1) 향후 5년간 성과활용·확산 활용방안 및 계획(활동계획)

o 클라우드 엣지 플랫폼의 자체 사업화

- Business Model

구분	내용
상용화 모델 및 형태	• 클라우드 엣지 플랫폼 공급 및 구축
예상 수요처	• SK텔레콤 : 기업 고유 영역의 엣지 서비스인 스마트팩토리 사업을 위한 솔루션 개발 및 공개에도 많은 투자를 하고 있음. 2018년 12월 '스마트제조혁신센터'에서 5G 스마트팩토리 확산 전략을 발표. 5G와 첨단 ICT를 접목한 솔루션 및 향후 스마트팩토리 규격 표준화를 주도해 생태계를 빠르게 확산 예정임. SK텔레콤 스마트팩토리 솔루션에 클라우드 엣지 플랫폼 기술을 적용하여 본 과제 결과에 대한 사업화 가능성을 실증함. • 클라우드 사업자 : KT, 네이버 등 기존 클라우드 사업자들도 IoT, 드론, AR/VR 등 다양한 형태의 클라우드 사업 확대에 따른 네트워크 속도이슈를 가지고 있으며 엣지 컴퓨팅에 대한 니즈 발생. 본 과제 결과인 클라우드 엣지 플랫폼의 사업화 가능성 높음.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

	예상 수요처	
	IoT 사업자	SKT, KT, LG U+ 등 대형 서비스 사업자
	중소형 사업자	IoT 서비스를 하는 공공기관
예상 단가	• 플랫폼 납품 및 구축 시, 협의예정	
용도 및 기능	• 엣지 응용 서비스를 구축 및 실행하는데 사용할 수 있는 일련의 완전 관리형 서비스를 제공 • 서비스 개발을 위한 API 단위 응용 개발/프로토타입/배포/반복 등 일련의 개발 및 운영 과정의 자동화 • **(클라우드 서비스/인프라 사업자는)** IoT 등 API 서비스를 지원하는 차세대 클라우드 엣지 컴퓨팅 및 엣지 서비스 상품을 제공 • **(IT 응용 서비스 사업자 또는 개발사는)** API 단위 기능 조합을 통한 다양한 엣지 응용 서비스 개발 환경을 제공	

– 사업화 계획 및 일정

- AR/VR 기반의 원격 작업 지시 서비스인 Smart Helper 서비스에 대한 검증과 사업화 관련 개선사항 도출 및 보완
- 사업 초기 경쟁력 확보를 위해 원격 작업 지시 서비스의 활용도가 높은 사업 영역 공략에 집중 (공장, 건설 현장, 학교 등)
- Smart Helper 시범서비스 적용 및 실증으로 엣지 서비스 활용에 관한 성공사례를 확보
- 엣지 서비스 생태계 조성을 위해 관련 협회 및 커뮤니티를 활용한 적용사례 홍보를 진행하여 저변을 확대하고 관련시장 확산을 유도

(1단계) Smart Helper 서비스(AR/VR) 검증

◦ Smart Helper 서비스 적용 및 실증
◦ 적용 결과 분석을 통한 사업화 개선사항 도출

↓

(2단계) 주요 수요처 중심의 시범서비스

◦ SK텔레콤 5G 스마트 팩토리 시범적용 및 실증
◦ 실증 결과 분석을 통한 상용화 개선사항 도출
◦ 니치마켓 타겟의 요구사항과 도출된 개선사항 반영
◦ SK텔레콤과의 기술적용 및 상용화 검증

↓

(3단계) 본격 상용 서비스 추진 및 생태계 응용 확산

◦ 스마트 팩토리, 스마트 시티 적용을 위한 표준 레퍼런스 모델 확보
◦ 클라우드 서비스 사업자, IoT사업자, 제조 및 유통사업자 등을 중심으로 본격 사업추진
◦ SK텔레콤 스마트 팩토리 상품 런칭 및 운영

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

◦ 초기 활성화 및 채널 확대를 위한 클라우드 엣지 플랫폼 개발 교육 추진 (3rd 참여를 위한 기술세미나 및 커뮤니티 활동)

◦ IoT 및 클라우드 전시회, 포럼을 통한 적용사례 지속적 홍보

- 홍보 및 판매 전략

• 한국사물인터넷협회에서 주체하는 IoT 전문 세미나, 전시회 및 컨퍼런스 등에서 엣지 컴퓨팅 기술 소개 및 효과 홍보

◦ 국내 성공적인 서비스 구축 사례 확보 후, 해외 전시회 및 컨퍼런스 참가하여 엣지 컴퓨팅 기술 및 솔루션 홍보

• 기술개발 결과물의 수요처 대상 협력 체계 구축 및 홍보

제조 및 공장자동화 솔루션 사업자	도시 인프라 솔루션 및 서비스 사업자
현대자동차 등 커넥티드 카 사업자	SKT, KT 등 대형 클라우드 서비스 사업자
중소형 IoT 사업자	네이버, 카카오 등 포털 사업자

• 주요 기업, 공공기관 및 클라우드 사업자 대상으로 클라우드 엣지 플랫폼 판매 전략

◦ 한국클라우드컴퓨팅연구조합 등 엣지 컴퓨팅 기술 유관기업을 대상으로 엣지 컴퓨팅 기술 세미나 및 시연 행사 추진

◦ 시범서비스 구축 및 테스트 센터 구축 등을 통해 신제품 도입을 검토하는 고객들에게 자유로운 체험 기회를 제공하며, 이를 통해 엣지 컴퓨팅 기술 확산을 위한 국내 생태계 확보

• 언론사 대상 메일링, 홍보 및 판촉 활동

<연도별 주요 활동계획(요약표)>

구분	종료+1 (2022년)	종료+2 (2023년)	종료+3 (2024년)	종료+4 (2025년)	종료+5 (2026년)
클라우드 엣지 플랫폼 자체 사업화	- Smart Helper 서비스 적용 및 실증으로 성공 사례 확보 - 전시회 및 포럼을 통한 적용 사례 발표 중심의 홍보	- SKT 5G 스마트팩토리 적용 및 실증 - 전시회 및 포럼에서 시연 중심의 솔루션 홍보	- 스마트팩토리 적용을 위한 표준 레퍼런스 모델 확보 - 제조 및 공장 자동화 솔루션 사업자 대상 협력 체계 구축 및 홍보	- 스마트시티 적용을 위한 표준 레퍼런스 모델 확보 - 도시 인프라 솔루션 및 서비스 사업자 대상 협력 체계 구축 및 홍보	- 클라우드 엣지 플랫폼 어플라이언스 솔루션 확보 - 해외 고객을 대상으로 한 솔루션 홍보 및 해외 고객 확보

2) 주요 성과활용 (기대)성과목표

(1) 정성적 목표

o 기술적 측면

- 4차 산업혁명과 5G 기술 상용화로 클라우드 엣지 기술에 대한 중요성이 부각되고 있으나 국내 관련 핵심 기술은 부재한 상황으로 대부분이 글로벌 외산 기업이 클라우드 엣지 기술을 선점하고 있으나 아직 상용화 수준에는 기술적 제약이 있어 글로벌 업체들도 최근에야 대처하기 시작한 상황으로 본 과제에서 제안하는 기술 개발을 통해 기술 선도 가능
- 외산 상용 기술에 절대 의존적인 클라우드 엣지 기술(Lock-In) 대신 공개 SW 기

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

반 클라우드 엣지 기술(Kubernetes, OpenStack 등)을 개발 환경으로 활용함으로써 공개 SW의 기술적 가능성 확보 및 공개 SW의 확산 기반 마련

o 경제적, 산업적 측면

- 국내에 수입되고 있는 외산 클라우드 엣지 서비스(예: MS 에저 IoT 엣지 서비스)에 대한 수입 대체로 국내 시장 진입
- 제안 기술을 개발 완료한 후 '2024년 약 104억 원 외산 엣지 솔루션 수입 대체 가능
- 외국 클라우드 엣지 서비스를 도입하여 단순 활용하는 국내기업의 해외 기술 종속성을 개선하고 국내 클라우드 SW 기업의 기술 및 국내 시장 경쟁력을 확보하고 글로벌 시장 진출을 위한 경쟁력 확보

o 사회적 측면

- 4차 산업혁명의 핵심기술인 클라우드 컴퓨팅 활성화를 위한 공공부문 기본계획을 마련하고 민간 클라우드서비스를 공공부문에 적극 도입하고, 민간 클라우드 이용 범위를 대폭 확대하기 위한 노력을 진행하고 있는 정부 정책에 부합
- 지능정보, 스마트팩토리 등 차세대 융합 서비스와 연계하여 단말과 물리적으로 가까운 위치에 있는 엣지에서 데이터를 처리함으로서 지연시간 최소화를 통해 소비자에 삶의 편의성을 제공하는 다양한 응용 서비스를 실시간 제공이 가능해짐
- 외국 클라우드 엣지 서비스를 도입하여 단순 활용하는 국내기업의 해외 기술 종속성을 개선하고 국내 클라우드 SW 기업의 기술 및 국내 시장 경쟁력을 확보하고 글로벌 시장 진출을 위한 경쟁력 확보

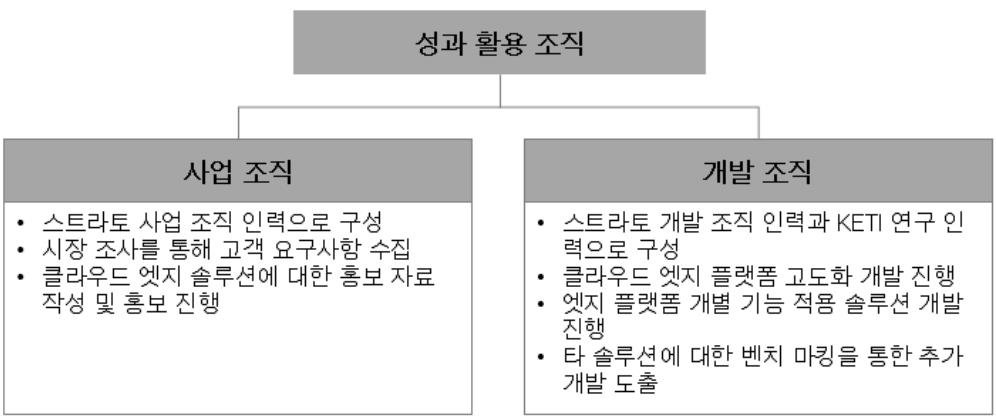
(2) 정량적 목표

<연도별 성과활용 (기대)성과목표(요약표)>

구분	종료+1 (2022년)	종료+2 (2023년)	종료+3 (2024년)	종료+4 (2025년)	종료+5 (2026년)
매출액	7억	35억	105억	140억	170억
고용	-	2	3	1	1

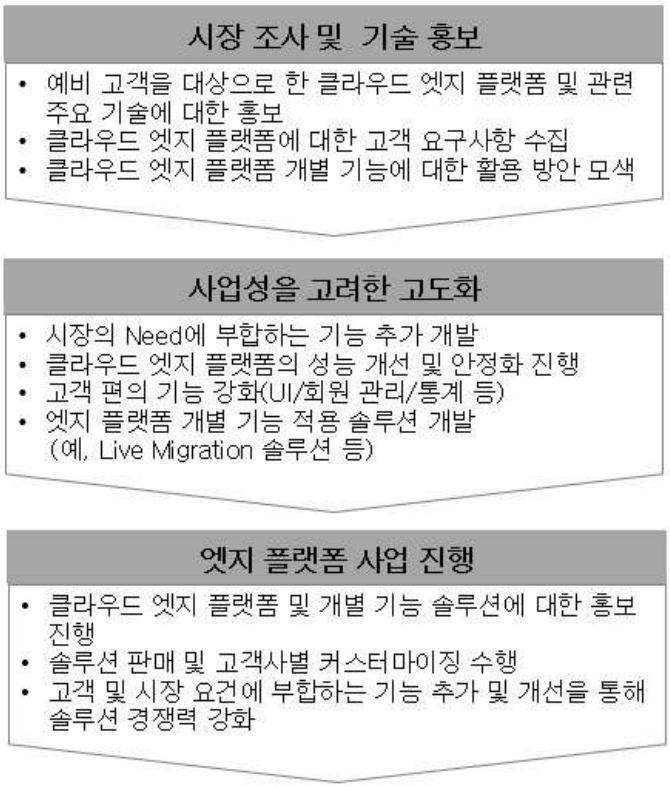
3) 성과활용·관리 추진체계

(1) 성과활용 조직 구성



국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(2) 성과활용 프로세스



4) 기타

해당 사항 없음

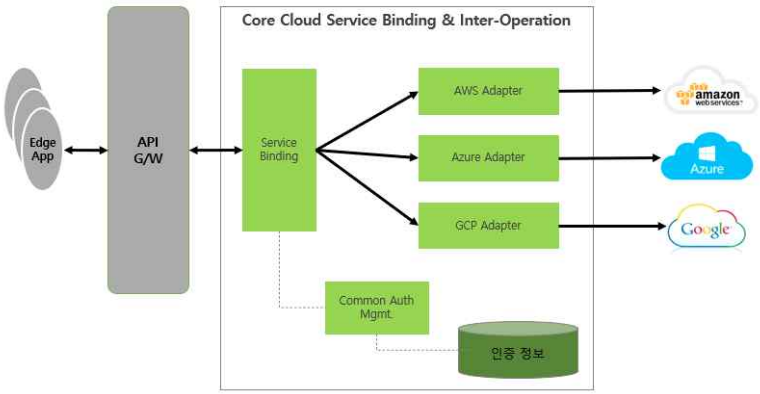
국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

7. 연구개발성과에 대한 후속연구 및 추가 개발 계획

○ 다양한 Public Cloud와의 연계 지원 기술 개발 예정

Core Cloud와 Edge Cloud간 연계를 통한 서비스 구성을 위해 Public Cloud(AWS, Azure, GCP 등)에서 제공하는 다양한 API를 이용할 수 있도록 관련 기능 고도화 진행 예정

- Azure, AWS, Google 등 벤더별 API 분석을 통해 클라우드 엣지 환경의 서비스에서 사용될 IoT, 빅데이터, API 등의 API 단일 규격화 진행
- Public Cloud 공통 인증 토큰 관리 기능 개발

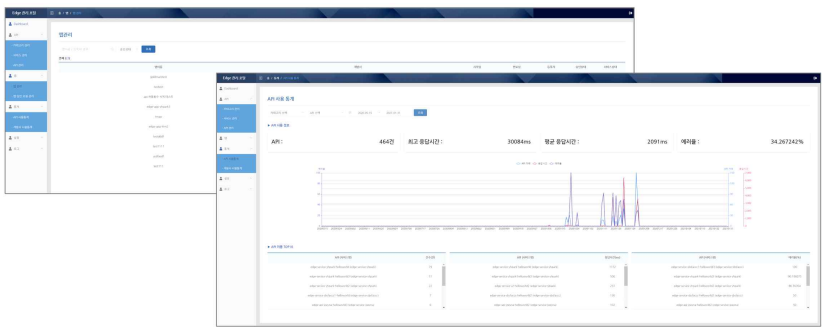


[Public Cloud 연계 구조]

○ 클라우드 엣지 포털 고도화 진행 예정

클라우드 엣지 플랫폼의 고객 편의성 강화를 통한 상품성 개선을 위해 필요한 기능을 지속적으로 개발 진행 예정

- 엣지 앱/서비스 Docker Image 파일 업로드를 통한 Registry 등록 기능 개발
- 서비스 디스크립터 자동 생성 기능 개선
  - K8S 신규 Object, Controller 추가 지원
- 서비스 디스크립터 파일 업로드 시 정합성 체크 기능 개발
- 상용 환경의 Need를 고려한 통계 기능 추가
- 사용자 관리 체계 고도화



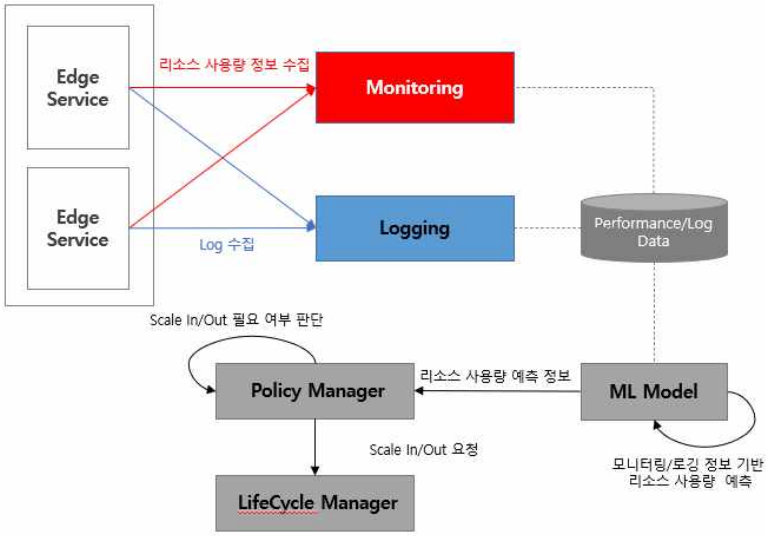
[클라우드 엣지 포털]

○ 자원 사용량/로그 기반의 예측을 통한 자원 자동 스케줄링 기술 개발 예정

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

엣지 서비스 인스턴스의 자원 사용량과 로그 정보를 기반으로 Machine Learning을 통해 리소스 사용량을 예측하고 예측된 사용량 정보를 기반으로 엣지 서비스에 대한 Auto Scale In/Out 처리를 수행 예정

- CPU/메모리/Network 자원 사용량 기반 리소스 사용량 예측 모델 개발
- 로그 정보를 기반으로 한 자원 사용량 예측 모델 개발
- 사용량 예측을 기반으로 한 엣지 서비스 Auto Scale In/Out 처리 기능 개발



[리소스 사용량 예측에 의한 자원 자동 스케줄링 흐름]

○ 계층별 자원 모니터링 및 학습 모델을 통한 라이브 마이그레이션 지원형 자원 재배치 기술 상용 제품화 예정임

○ 현재는 다차원 메트릭 수집(서버와 컨테이너의 CPU,메모리 자원 사용량, 네트워크, 스토리지) 기반 학습 모델 생성 및 이를 통한 워크로드 예측 기술을 적용했으며, 향후 클라우드 엣지 레벨에서의 구동 애플리케이션 별 특성에 대한 즉시성의 실시간 보장을 위한 현장 학습이 가능한 부하 예측 모델을 적용하기 위한 추가 모니터링 데이터 수집 적용 기능과 현장 수집 데이터를 통한 애플리케이션 특성 별 워크로드 예측 기능을 개발할 예정임

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

8. 연구개발비 사용실적

① 총괄

1) 연구개발비 사용명세서

(단위 : 원, %)

항목 \ 구분		연구개발비 현황					사용금액 ⑥	사용잔액 (⑤-⑥)	사용률 (⑥/⑤ ×100)	불용금액 (해당 시)
		전단계 이월액 (해당 시) ①	당초 연구개발비 ②	변경 연구개발비 ③	발생이자 중 연구개발비 산입금 ④	소계 ⑤ (①+②+③+④)				
가. 직접비	현금	-	624,804,000	0	-	624,804,000	615,479,996	9,324,004	98.51%	5,104,004
	현물	-	90,000,000	0	-	90,000,000	90,000,000	0	100%	-
	소계	-	714,804,000	0	-	714,804,000	705,479,996	9,324,004	98.70%	5,104,004
1) 인건비	현금	-	430,097,000	19,880,000	-	449,977,000	446,042,880	3,934,120	99.13%	3,934,120
	현물	-	90,000,000	0	-	90,000,000	90,000,000	0	100%	-
2) 학생인건비	현금	-	0	0	-	0	0	0	-	-
3) 연구시설・장비비	현금	-	19,900,000	-4,600,000	-	15,300,000	15,248,000	52,000	99.66%	52,000
	현물	-	0	0	-	0	0	0	-	-
4) 연구재료비	현금	-	39,788,000	10,720,000	-	50,508,000	50,507,930	70	100%	70
	현물	-	0	0	-	0	0	0	-	-
5) 위탁연구개발비[1」]	현금	-	0	0	-	0	0	0	-	-
6) 국제공동연구개발비	현금	-	0	0	-	0	0	0	-	-
7) 연구개발부담비	현금	-	0	0	-	0	0	0	-	-
8) 연구활동비	현금	-	56,300,000	-21,500,000	-	34,800,000	30,248,610	4,551,390	86.92%	331,390
	현물	-	0	0	-	0	0	0	-	-
9) 연구수당	현금	-	76,819,000	-2,600,000	-	74,219,000	73,432,576	786,424	98.94%	786,424
나. 간접비	현금	-	54,096,000	-1,900,000	-	52,196,000	51,773,400	422,600	99.19%	-
	소계	-	54,096,000	-1,900,000	-	52,196,000	51,773,400	422,600	99.19%	-
합계	현금	-	677,000,000	0	-	677,000,000	667,253,396	9,746,604	98.56%	5,104,004
	현물	-	90,000,000	0	-	90,000,000	90,000,000	0	100%	-
	합계	-	767,000,000	0	-	767,000,000	757,253,396	9,746,604	98.73%	5,104,004

* 1」 위탁연구개발비 현황 및 사용금액은 주관연구개발기관만 기재합니다.

2) 발생이자 사용명세서

(단위 : 원)

구분	발생금액 (A)	사용용도	사용금액 (B)	사용잔액 (A-B)
연구개발기간 중 발생이자	299,433	국고반납	-	299,433
연구개발기간 후 발생이자	-	-	-	-
합계	299,433	국고반납	-	299,433

3) 반납액 명세서

(단위 : 원, %)

연구개발비 반납액				발생이자 반납액		총 반납액 (C+E)
구분	현금	현물	소계			
사용잔액	9,746,604	-	9,746,604	연구개발기간 중	299,433	10,046,037
불인정 사용금액	-	-	-	연구개발기간 후	-	-
소계(A)	9,746,604	-	9,746,604	소계(D)	299,433	10,046,037
정부지분율(B)	100%	-	100%	정부지분율(B)	100%	100%
정부지분액(C=A×B)	9,746,604	-	9,746,604	정부지분액(E=D×B)	299,433	10,046,037

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

② 주관연구개발기관 : 스트라토 (구 에스피테크놀러지(주))

1) 연구개발비 사용명세서

(단위 : 원, %)

구분 / 항목		연구개발비 현황					사용금액 ⑥	사용잔액 (⑤-⑥)	사용률 (⑥/⑤ ×100)	불용금액 (해당 시)
		전단계 이월액 (해당 시) ①	당초 연구개발비 ②	변경 연구개발비 ③	발생이자 중 연구개발비 산입금 ④	소계 ⑤ (①+②+③+④)				
가. 직접비	현금	-	407,200,000	0	-	407,200,000	402,980,000	4,220,000	98.96%	-
	현물	-	90,000,000	0	-	90,000,000	90,000,000	0	100%	-
	소계	-	497,200,000	0	-	497,200,000	492,980,000	4,220,000	99.15%	-
1) 인건비	현금	-	296,000,000	32,880,000	-	328,880,000	328,880,000	0	100%	-
	현물	-	90,000,000	0	-	90,000,000	90,000,000	0	100%	-
2) 학생인건비	현금	-	-	-	-	-	-	-	-	-
3) 연구시설・장비비	현금	-	9,600,000	-9,600,000	-	0	-	0	-	-
	현물	-	-	-	-	-	-	-	-	-
4) 연구재료비	현금	-	5,400,000	-3,780,000	-	1,620,000	1,620,000	0	100%	-
	현물	-	-	-	-	-	-	-	-	-
5) 위탁연구개발비[1」]	현금	-	-	-	-	-	-	-	-	-
6) 국제공동연구개발비	현금	-	-	-	-	-	-	-	-	-
7) 연구개발부담비	현금	-	-	-	-	-	-	-	-	-
8) 연구활동비	현금	-	44,300,000	-17,600,000	-	26,700,000	22,480,000	4,220,000	84.19%	-
	현물	-	-	-	-	-	-	-	-	-
9) 연구수당	현금	-	50,000,000	0	-	50,000,000	50,000,000	0	100%	-
나. 간접비	현금	-	4,700,000	-1,900,000	-	2,800,000	2,377,400	422,600	84.91%	-
	소계	-	4,700,000	-1,900,000	-	2,800,000	2,377,400	422,600	84.91%	-
합계	현금	-	410,000,000	0	-	410,000,000	405,357,400	4,642,600	98.87%	-
	현물	-	90,000,000	0	-	90,000,000	90,000,000	0	100%	-
	합계	-	500,000,000	0	-	500,000,000	495,357,400	4,642,600	99.07%	-

* 1」 위탁연구개발비 현황 및 사용금액은 주관연구개발기관만 기재합니다.

2) 발생이자 사용명세서

(단위 : 원)

구분	발생금액 (A)	사용용도	사용금액 (B)	사용잔액 (A-B)
연구개발기간 중 발생이자	299,433	국고반납	0	299,433
연구개발기간 후 발생이자	-	-	-	-
합계	299,433	국고반납	0	299,433

3) 반납액 명세서

(단위 : 원, %)

연구개발비 반납액				발생이자 반납액		총 반납액 (C+E)
구분	현금	현물	소계			
사용잔액	4,642,600	0	4,642,600	연구개발기간 중	299,433	4,942,033
불인정 사용금액	0	0	0	연구개발기간 후	0	0
소계(A)	4,642,600	0	4,642,600	소계(D)	299,433	4,942,033
정부지분율(B)	100	0	100	정부지분율(B)	100	100
정부지분액(C=A×B)	4,642,600	0	4,642,600	정부지분액(E=D×B)	299,433	4,942,033

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

③ 공동연구개발기관 : 한국전자기술연구원

1) 연구개발비 사용명세서

(단위 : 원, %)

항목 \ 구분		연구개발비 현황					사용 금액 ⑥	사용 잔액 (⑤-⑥)	사용률 (⑥/⑤ ×100)	불용금액 (해당 시)
		전단계 이월액 (해당 시) ①	당초 연구개발비 ②	변경 연구개발비 ③	발생이자 중 연구개발비 산입금 ④	소계 ⑤ (①+②+③+④)				
가. 직접비	현금	-	217,604,000	0	-	217,604,000	212,499,996	5,104,004	97.65%	5,104,004
	현물	-	-	-	-	-	-	-	-	-
	소계	-	217,604,000	-	-	217,604,000	212,499,996	5,104,004	97.65%	5,104,004
1) 인건비	현금	-	134,097,000	-13,000,000	-	121,097,000	117,162,880	3,934,120	96.75%	3,934,120
	현물	-	-	-	-	-	-	-	-	-
2) 학생인건비	현금	-	-	-	-	-	-	-	-	-
3) 연구시설·장비비	현금	-	10,300,000	5,000,000	-	15,300,000	15,248,000	52,000	99.66%	52,000
	현물	-	-	-	-	-	-	-	-	-
4) 연구재료비	현금	-	34,388,000	14,500,000	-	48,888,000	48,887,930	70	99.99%	70
	현물	-	-	-	-	-	-	-	-	-
5) 위탁연구개발비 [1」]	현금	-	-	-	-	-	-	-	-	-
6) 국제공동연구개발비	현금	-	-	-	-	-	-	-	-	-
7) 연구개발부담비	현금	-	-	-	-	-	-	-	-	-
8) 연구활동비	현금	-	12,000,000	-3,900,000	-	8,100,000	7,768,610	331,390	95.91%	331,390
	현물	-	-	-	-	-	-	-	-	-
9) 연구수당	현금	-	26,819,000	-2,600,000	-	24,219,000	23,432,576	786,424	96.75%	786,424
나. 간접비	현금	-	49,396,000	-	-	49,396,000	49,396,000	-	100%	-
	소계	-	49,396,000	-	-	49,396,000	49,396,000	-	100%	-
합계	현금	-	267,000,000	-	-	267,000,000	261,895,996	5,104,004	98.09%	5,104,004
	현물	-	-	-	-	-	-	-	-	-
	합계	-	267,000,000	-	-	267,000,000	261,895,996	5,104,004	98.09%	5,104,004

* 1」 위탁연구개발비 현황 및 사용금액은 주관연구개발기관만 기재합니다.

2) 발생이자 사용명세서

(단위 : 원)

구분	발생금액 (A)	사용용도	사용금액 (B)	사용잔액 (A-B)
연구개발기간 중 발생이자	-	-	-	-
연구개발기간 후 발생이자	-	-	-	-
합계	-	-	-	-

3) 반납액 명세서

(단위 : 원, %)

연구개발비 반납액				발생이자 반납액		총 반납액 (C+E)
구분	현금	현물	소계			
사용잔액	5,104,004	-	5,104,004	연구개발기간 중	-	5,104,004
불인정 사용금액	-	-	-	연구개발기간 후	-	-
소계(A)	5,104,004	-	5,104,004	소계(D)	-	5,104,004
정부지분율(B)	100%	-	100%	정부지분율(B)	-	100%
정부지분액(C=A×B)	5,104,004	-	5,104,004	정부지분액(E=D×B)	-	5,104,004

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

붙임 2. 자체 보안관리 진단표

1) 스트라토

구분	체크항목	결과 체크 (√표)	비고 (미실시 사유)
보안관리 체계	o 기관 내 보안관리규정을 제정/적용하고 있다	O(√), X( )	
	o 보안관리 조직이 있으며, 자체 보안점검실시 등 잘 운영되고 있다	O(√), X( )	
	o 보안교육을 정기적(1회이상/연)으로 실시하고 있다	O(√), X( )	
	o 보안사고에 대한 방지대책 및 비상시 대응계획이 준비되어 있다	O(√), X( )	
참여연구원 관리	o 참여연구원에 대하여 보안서약서를 받았다	O(√), X( )	
	o 참여연구원에게 보안관리의 중요성 등을 인식시키고 있다	O(√), X( )	
연구개발 내용/결과 관리	o 주요 연구자료 및 성과물의 무단유출 방지대책을 수립하고 있다	O(√), X( )	
	o 보안성 검토 방법 및 절차를 이행하고 있다	O(√), X( )	
	o 기술이전 관련 내부규정 및 절차를 준수하고 있다	O(√), X( )	
연구시설 관리	o 연구시설 보안관련 내부규정 또는 지침을 이행하고 있다	O(√), X( )	
	o 주요 시설에는 보안장비가 설치되어 있다	O(√), X( )	
	o 보호구역이 지정되어 있다	O(√), X( )	
정보통신망 관리	o 정보통신망 보안관련 내부규정 또는 지침이 구비되어 있다	O(√), X( )	
	o 보안관리 책임자의 승인 항목이 구분되어 있다	O(√), X( )	
	o 주요 데이터에 대해 백업을 실시하고 있다	O(√), X( )	
	o 개인용 정보통신장비(노트북, USB메모리)에 대하여 인가/관리중이다	O(√), X( )	
	o 전산망 보호를 위한 HW 및 SW 등을 도입하여 적용하고 있다	O(√), X( )	
	o 직책, 임무별 열람 권한을 차등화하여 부여하고 있다	O(√), X( )	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2) 한국전자기술연구원

구분	체크항목	결과 체크 (√표)	비고 (미실시 사유)
보안관리 체계	o 기관 내 보안관리규정을 제정/적용하고 있다	O(√), X( )	
	o 보안관리 조직이 있으며, 자체 보안점검실시 등 잘 운영되고 있다	O(√), X( )	
	o 보안교육을 정기적(1회이상/연)으로 실시하고 있다	O(√), X( )	
	o 보안사고에 대한 방지대책 및 비상시 대응계획이 준비되어 있다	O(√), X( )	
참여연구원 관리	o 참여연구원에 대하여 보안서약서를 받았다	O(√), X( )	
	o 참여연구원에게 보안관리의 중요성 등을 인식시키고 있다	O(√), X( )	
연구개발 내용/결과 관리	o 주요 연구자료 및 성과물의 무단유출 방지대책을 수립하고 있다	O(√), X( )	
	o 보안성 검토 방법 및 절차를 이행하고 있다	O(√), X( )	
	o 기술이전 관련 내부규정 및 절차를 준수하고 있다	O(√), X( )	
연구시설 관리	o 연구시설 보안관련 내부규정 또는 지침을 이행하고 있다	O(√), X( )	
	o 주요 시설에는 보안장비가 설치되어 있다	O(√), X( )	
	o 보호구역이 지정되어 있다	O(√), X( )	
정보통신망 관리	o 정보통신망 보안관련 내부규정 또는 지침이 구비되어 있다	O(√), X( )	
	o 보안관리 책임자의 승인 항목이 구분되어 있다	O(√), X( )	
	o 주요 데이터에 대해 백업을 실시하고 있다	O(√), X( )	
	o 개인용 정보통신장비(노트북, USB메모리)에 대하여 인가/관리중이다	O(√), X( )	
	o 전산망 보호를 위한 HW 및 SW 등을 도입하여 적용하고 있다	O(√), X( )	
	o 직책, 임무별 열람 권한을 차등화하여 부여하고 있다	O(√), X( )	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

주 의

1. 이 보고서는 과학기술정보통신부에서 시행한 정보통신·방송연구개발사업의 ○○연구개발과제 최종보고서이다.

2. 이 연구개발내용을 대외적으로 발표할 때에는 반드시 과학기술정보통신부(정보통신기획평가원)에서 시행한 정보통신·방송연구개발사업의 결과임을 밝혀야 한다.

3. 국가과학기술 기밀 유지에 필요한 내용은 대외적으로 발표 또는 공개하여서는 안 된다.